

BECKMAN COULTER LIFE SCIENCES

Biomek Instrument & Project Reference

Instrument Settings, Deck Layouts & Project Items

JSON format version 1.0

20 documents

Companion volumes: Biomek JSON Authoring Guide (in *Biomek JSON Authoring Guide*) · Biomek JSON Add-on Steps (in *Biomek JSON Add-on Steps*)

Table of Contents

Instruments & Deck Formats

- Instrument & Project-Item Reference
- Pod Settings Format Specification
- Deck Layouts Format Specification
- ALP Catalog
- Reachability and Access
- Pod Reach Envelopes (i7 worked example)
- Deck Layout: Biomek i3
- Deck Layout: Biomek i5 Multichannel
- Deck Layout: Biomek i5 Span8
- Deck Layout: Biomek i7 DualMultichannel
- Deck Layout: Biomek i7 Hybrid
- Deck Layout: Biomek i7 Multichannel
- Deck Layout: Biomek i7 Span8

Project Items

- Project Items Format Specification
- Catalog: Labware Classes
- Catalog: Labware Patterns
- Catalog: Liquid Types
- Catalog: Techniques
- Catalog: Pipetting Templates
- Catalog: Tip Classes

Instrument & Project-Item Reference

Reference material describing what Biomek expects for instrument configuration and project items, so that while developing a JSON method you can understand what your parameters, deck positions, techniques, and labware must match — and what the software validates against. The per-step docs assume you are familiar with your own instrument's configuration: export your instrument's settings from the Biomek software (**File** ▶ **Export** ▶ **Instrument** ▶ **Save As JSON**) and read them alongside the specs, sample exports, and catalogs here to interpret what you see. A method that targets a configuration your instrument does not actually have will likely fail to enqueue or run.

Each subarea is self-contained; read the spec for the format you need, and consult the neighboring sample or catalog file for a worked reference.

This is one of three companion volumes. The Biomek JSON Authoring Guide (in *Biomek JSON Authoring Guide*) covers method structure, pipetting techniques, worked examples, and the per-step parameter reference; Biomek JSON Add-on Steps (in *Biomek JSON Add-on Steps*) covers step types that are only available when their add-on is installed.

instrument-settings/

The JSON export produced by the Biomek Instrument Settings feature — pod settings and deck layouts, per instrument model. See Pod Settings Format Specification for the pod-settings portion of the format, including a roster of seven representative instrument configurations (i3, i5 multichannel, i5 span-8, i7 multichannel, i7 span-8, i7 hybrid, i7 dual multichannel).

deck-layouts/

The deck-layout portion of an Instrument Settings export — how to read a deck out of `deckLayouts.<deckKey>` in a Biomek Instrument Settings JSON export. See Deck Layouts Format Specification for the format spec (including the reference model, `$id / $ref` handling, and the per-position record schema), ALP Catalog for the common default ALP types that can appear in a deck, Reachability and Access and Pod Reach Envelopes (i7 worked example) for which pod can reach which position (per-pod X travel envelopes and an i7 reach table), and [deck-layouts/samples/](#) for an illustrative resolved view of each default deck (seven files, one per default instrument).

project-items/

A planned/future summary format for Biomek project items (labware classes, labware patterns, liquid types, tip classes, pipetting techniques and templates). Biomek currently emits no JSON export for project items, so this format documents a derived summary shape rather than a current export. See Project Items Format Specification for the format, and the six per-category catalog files (Catalog: Labware Classes, Catalog: Labware Patterns, Catalog: Liquid Types, Catalog: Tip Classes, Catalog: Techniques, Catalog: Pipetting Templates) for the defaults included with Biomek.

[†] Table of contents

Pod Settings Format Specification

What this document is. A single spec for the **pod-settings** portion of a Biomek **Instrument Settings** JSON export: what a pod-settings record looks like, and how to read facts out of it. The umbrella export (`format:"Biomek Instrument Settings"`) bundles two categories in one file — **pod settings** and deck layouts. This doc owns the **pod settings** category (`podSettings`), plus the shared envelope and the `$id / $ref` reference mechanics you need to read *any* part of the file. Part 1 defines the source format (envelope, category keys, the pod record, value encoding, `$id / $ref`). Part 2 is the extraction recipe, worked end-to-end against a sample export.

The deck portion is documented separately. Deck layouts (`deckLayouts`) live in the same file but are specified — with their own conversion recipe — in Deck Layouts Format Specification. This doc does not duplicate it; when you need decks, go there.

Part 1 — the source format

What pod settings are (read this first)

`podSettings` is the per-pod hardware configuration of the instrument: one entry per installed pod, carrying that pod's **class** (Multichannel / Span-8 / Fixed-8), head type, axis travel limits, calibration/gripper offsets, speed limits, and current tip state.

Why this format is useful:

- To learn **how many pods** the instrument has and **what each pod is** (Multichannel / Span-8 / Fixed-8) before authoring pod-specific steps.
- To read a pod's **axis limits**, calibration, and gripper offsets.
- To cross-check a Multichannel or Span-8 pod's identity against its human-readable `asText` summary. **Fixed-8's `asText` is a stub** (just the literal `"Fixed-8 pod settings"`), so do not rely on it for that class — read `settings` directly.

Where pod settings come from: the Instrument Settings JSON export

Pod settings are **not** a standalone file. They live inside a **"Biomek Instrument Settings"** JSON export.

It is produced by the Import/Export Utility's "Save As JSON" action: in the Editor, *File > Export > Instrument Settings*, select folders in the instrument tree (Pod Settings / Deck Layouts), *Export*, then *Save As JSON*.

Contrast with method JSON

	Method JSON (<code>Method-JSON-Structure.md</code>)	Instrument-settings JSON (this doc)
<code>format</code> value	<code>"Biomek Method"</code>	<code>"Biomek Instrument Settings"</code>
Purpose	Ordered tree of steps to run	The machine : pods and deck layouts
Top-level keys	<code>format</code> , <code>formatVersion</code> , <code>author?</code> , <code>description?</code> , <code>steps[]</code>	<code>format</code> , <code>formatVersion</code> , <code>podSettings?</code> , <code>deckLayouts?</code>
Value encoding	<code>_biomekType</code> provider stack	Same stack + instrument-only providers + <code>\$id / \$ref</code> refs

There is no `steps` , `stepType` , or `subSteps` here. Do not look for a method envelope.

Top-level envelope

```
{
  "format": "Biomek Instrument Settings",
  "formatVersion": "1.0",
  "podSettings": { ... },
  "deckLayouts": { ... }
}
```

Key	Always present?
<code>format</code>	Yes — literal <code>"Biomek Instrument Settings"</code>
<code>formatVersion</code>	Yes — the Instrument Settings format version, currently <code>"1.0"</code> (independent of the Biomek Method JSON <code>formatVersion</code>)
<code>podSettings</code>	Only if Pod Settings were exported and non-empty
<code>deckLayouts</code>	Only if Deck Layouts were exported and non-empty (spec'd in the deck doc)

Key points:

- The category keys are the camelCase renamings of the instrument-tree folder names: `"Pod Settings"→podSettings` , `"Deck Layouts"→deckLayouts` . The original spaced names never appear in the JSON.
- **A category appears only if the user selected it AND the instrument has something for it.** You can export just one folder, and empty categories are dropped. So a file may have `podSettings` and no `deckLayouts` , or vice-versa.
- **First step when reading any file:** check `format` , then list which of the category keys are present. That tells you what the export covers before you dive in.

The pod-settings category (`podSettings`)

`podSettings` is a JSON object **keyed by pod name** (`pod1` , `pod2` , ...). Each entry has exactly **three** top-level fields:

Field	Type	Meaning
<code>asText</code>	string	Human-readable summary of the pod. On Multichannel and Span-8 it carries validation time, head type, axis ranges — a useful cross-check. On Fixed-8 it is a stub (just <code>"Fixed-8 pod settings"</code>) with none of that; read <code>settings</code> directly for Fixed-8.
<code>settings</code>	object	The structured pod record — class, head type, limits, offsets, tips (see below).
<code>timestamp</code>	object	A <code>_biomekType:"dateTime"</code> object (see Timestamps) marking when this settings snapshot was taken , i.e. export time — not when the pod was last configured.

Not every key is necessarily an installed pod. The pods physically installed on the instrument are named `pod1` and `pod2` . If the operator has *saved pod settings* under their own names in Hardware Setup, those saved profiles are exported into the same `podSettings` object, keyed by their saved names (letters, digits and underscores only) with the same camelCase key policy applied. A saved profile has the same three-field shape as an installed pod, so structure alone will not distinguish them — **identify installed pods by the `pod1` / `pod2` names rather than by counting keys**. None of the seven default instrument exports contains a saved profile, so the examples below all show the installed-pod case.

The pod class: read `settings.type` , and know the two-key nuance

- `settings.type` is the **pod class string** — one of `Pod96` , `Pod8Span` , or `Fixed8` . *This* is what tells you what the pod is; read it first.
- `settings.podType` is a **head-family code**: `MC` (multichannel), `SPAN8` , or `Fixed8` . For Multichannel and Span-8 it is a *different* value from `type` (`MC` vs `Pod96` , `SPAN8` vs `Pod8Span`). For **Fixed8** the two collapse — both fields carry `"Fixed8"` . It is present on every pod, which is why treating `settings.podType` as guaranteed-present is safe across pod classes.

Pod class	<code>settings.type</code>	<code>settings.podType</code>
Multichannel (96-head OR 384-head)	<code>Pod96</code>	<code>MC</code>
Span-8	<code>Pod8Span</code>	<code>SPAN8</code>
Fixed-8	<code>Fixed8</code>	<code>Fixed8</code>

The `settings` field set — per pod class

`settings` is a plain dictionary (no `_biomekType`). The key set differs substantially by pod class, so a doc built around only one class will mislead readers of another. The table below is grounded in seven representative instrument configurations (see Representative instrument configurations).

Common to every pod class (13 keys):

Key	Meaning
type	Pod class string (Pod96 / Pod8Span / Fixed8)
podType	Head-family code (MC / SPAN8 / Fixed8)
cmdGen	Command-generator identity for this pod
additionalSafeHeight	Safe-height margin above deck for moves (cm)
alwaysMaxZ	If true, always raise to Z-max between motions
currentSpeed	Current speed setting — a percentage, clamped by the pod to speedLimit
speedLimit	Maximum allowed speed as a percentage (0–100) of the axis maxima, never mm/s. The pod's asText renders it as e.g. Speed Limit: 100%. The other speed fields (discardSpeed , tipLoadSpeed , tipUnloadShuckSpeed) are percentages too.
rT_CurrentSpeed	Runtime-state mirror of currentSpeed
gripperXOffset , gripperYOffset , gripperZOffset	Gripper calibration offsets (cm). These are variant-typed, so a whole-number value may arrive without a decimal — Fixed-8 exports carry "gripperXOffset": 0 while 96-head exports carry "gripperYOffset": 0.0 . Accept both forms (see Value encoding).
max , min	Per-axis travel limit dictionaries — see Axis limits

Multichannel + Span-8 shared (5 keys, absent on Fixed-8):

Key	Meaning
additionalSafeHeightGripper	Extra Z margin when the gripper is deployed
gripperModel	Gripper model identifier
headSerialNumber	Serial number of the mounted head assembly
sensorEnabled	Whether the pod's tip-presence / other sensor is enabled
timeoutSlop	Motion timeout tolerance

Fixed-8 + Span-8 shared (17 keys, absent on Multichannel — these are the per-probe fields):

Key	Meaning
axes	Motion-parameters nested dictionary. Both the keying and the inner field set differ by class — see Axis motion parameters.
tip1 ... tip8	Per-probe tip descriptors — see Per-probe tips and tip state
rT_Tip1 ... rT_Tip8	Runtime per-probe tip state (same shape as tipN)

Pod96 (Multichannel) only (15 keys):

Key	Meaning
headType	Head type code
headTypeLabel	Human-readable head label (e.g. "300 µL MC-96 Head")
framingProbe	Reference probe used for head framing
perActionOverhead	Per-motion time overhead (calibration constant)
tips	Tip descriptor for the whole 96-head (single collection, not per-probe)
rT_Tips	Runtime tip state for the whole head
tipSeatingDepth	Depth to press tips into the labware for seating
tipLoadForceLimitOffset , tipLoadForceLimitSlope	Tip-load force-limit calibration
tipLoadPushOffDistance	Distance the head backs off during tip load

Key	Meaning
tipLoadSettlingTime	Wait time after tip pickup
tipLoadSpeed	Z-axis speed during tip pickup
tipLoadXRangePadding	X-axis padding at tip-load boundaries
tipUnloadShuckSpeed , tipUnloadZOffset	Tip-unload motion parameters

Which head is mounted

Two export fields identify the head, and a third gives its volume ceiling as a number:

- **settings.headType** (code) and **settings.headTypeLabel** (human) name the head. The μL figure in the name is the head's plunger stroke — a "300 μL MC-96 Head" tops out at 300 μL .
- **settings.max.d** is that same figure as a number, in μL , and is the value to read. It matches the mounted head's rating in every configuration in the sample exports, but importing a saved pod-settings file can overwrite it, so read the export rather than trusting the label.

What that ceiling is measured against, and why it means something different on the other two pod classes, is in What **max.d** means, and when it is not the pipetting ceiling.

Read the head and its ceiling for each MC pod straight from the export:

```
jq -r '.podSettings | to_entries[]
| select(.value.settings.podType=="MC")
| "\(.key): \(.value.settings.headTypeLabel) → ceiling \(.value.settings.max.d)  $\mu\text{L}$ "' \
  biomek-i7-hybrid.json
# pod1: 300  $\mu\text{L}$  MC-96 Head → ceiling 300  $\mu\text{L}$ 
```

Fixed-8 only (2 keys):

Key	Meaning
gripper	Fixed-8 pod's integrated gripper descriptor
podSerialNumber	Serial number of the Fixed-8 pod

Pod8Span (Span-8) only (13 keys):

Key	Meaning
syringes	Per-probe syringe configuration (size and state)
probeSize	Probe geometry / size class
probesSFOffset	Scale-factor offset per probe
loadingOffsets	Per-probe Z-offset used at tip load
dAxisCycles	Cycle count for the D (dispense) axes
discardSpeed	Speed used to discard tips
tipLoadTime	Wait time during tip load
detectTips	Whether tip-presence detection is enabled
brokenTips	Per-probe broken/damaged flag mask
postRunWashVolume	Volume used to wash probes after a run
systemTrailingAirgap	System-level trailing air-gap volume
calibrateVolumeForProbes	Whether per-probe volume calibration is enabled
lastValidation	Timestamp of last validation run

Axis limits (max / min)

max and min are per-axis dictionaries. The axis set varies by pod class, not just X/Y/Z:

- **Pod96:** x , y , z , d (dispense) plus gripper axes gg , gr , gy , gz .
- **Pod8Span:** adds per-probe travel limits z1 .. z8 and d1 .. d8 , plus a span axis (probe spacing), and an internal zMaxMC limit used when the Span-8 must reach an MC head's zone. zMaxMC appears in max **only** — max and min are not guaranteed to have the same key set, so do not iterate one and index the other blindly.
- **Fixed-8:** x , y , z , d **only** — the pod moves as a single block, so there are no per-probe axis limits, and **no gripper axes**. A Fixed-8 with an integrated gripper still shows only d / x / y / z here: the gripper is described by the top-level gripper block and is driven through the **D axis**, using the gripper offsets/angles inside cmdGen.conversion (gripperDegreesPerDAxisTravel , gripperFingerLength , restingGripperAngle , ...).

A consumer looking only at x/y/z will silently miss the gripper and per-probe limits — always enumerate the actual keys under .settings.max / .settings.min for the pod at hand. The asText summary is **not** a substitute: even on a 96-head pod its "Axis Limit Settings" block lists only X, Y, Z and D.

What max.d means, and when it is not the pipetting ceiling

d is not a distance. On every pod class the D axis is denominated in µL: its position is the plunger or syringe position expressed as the volume currently drawn. So max.d is a volume — but which volume depends on the pod class, and on one of the three it is not a pipetting limit at all.

Pod class	What max.d is	The pipetting ceiling?
Pod96 (Multichannel)	The mounted head's plunger stroke	Yes
Pod8Span	The installed syringe's capacity, per probe (max.d1 ... max.d8 ; max.d mirrors probe 1)	Yes
Fixed-8	The far end of the D axis, where the gripper is closed	No — see below

Where it is the ceiling, the check is on the **plunger position the move ends at**, not on the move's own volume, so whatever the tips already hold counts against it: a 200 µL aspirate on top of 150 µL already in the tips overruns a 300 µL head. It most often bites when a *computed* volume — a base volume plus conditioning, excess, or reagent terms — pushes one aspirate over the top. The overrun is caught at enqueue, before any motion, and the two families word it differently:

```
Move of <pod> to D of <n> µL violates D axis maximum of <max> µL.
Move of <pod> syringe #<n> to <v> µL violates the syringe maximum of <max> µL.
```

The first is Multichannel. The second is Span-8, and it names the offending **probe**, because Span-8 limits are per-probe — read max.d1 ... max.d8 rather than assuming one value covers the pod.

Fixed-8 is the exception. One motor drives both the plunger and the gripper, so the D axis runs past the pipetting range and on into the gripper's travel. max.d is the far end of that combined travel — on the sample i3 export, roughly a third above the largest volume the pod can actually pipette. Past the pipetting range the pipetting seals disengage, so the pipetting path enforces its own, lower maximum and rejects anything beyond it at enqueue:

```
The D axis position of <n> µL is outside the supported pipetting range of the pod.
```

That lower maximum is not in the export, under any key: the export carries the settings that are editable in Hardware Setup, and this one is not among them. There is therefore no way to compute a Fixed-8 pod's true pipetting ceiling from an instrument settings file. Do not read max.d as the ceiling on a Fixed-8 pod, and do not present it to a user as one — in practice a Fixed-8 tip's capacity is well below either number, so tip capacity is the limit a method meets first.

Note that for a Span-8 pod, the system trailing air gap must be accounted for in D axis movements. For example, when using a 1mL syringe size, the maximum draw of the D axis is 1mL minus the system trailing air gap amount.

Axis motion parameters

`axes` (Fixed-8 and Span-8 only) holds the per-axis motion profile — velocity, acceleration, jerk and their resolutions. **Both the keying and the inner field set differ by pod class**, so enumerate the inner keys instead of assuming a fixed set:

	Span-8	Fixed-8
Outer keys	Per probe: <code>z1 ... z8</code> , <code>d1 ... d8</code>	Whole pod: <code>x</code> , <code>y</code> , <code>z</code> , <code>d</code>
Velocity keys	<code>vmax</code> , <code>vmin</code> , <code>vres</code> (all lowercase)	<code>vMax</code> , <code>vMin</code> , <code>vres</code> (note the capital M)
Acceleration keys	<code>amax</code> , <code>amin</code> , <code>ares</code>	<code>amax</code> , <code>amin</code> , <code>ares</code>
Jerk keys	<code>jmax</code> , <code>jmin</code> , <code>jres</code> — z-axes only	<code>jmax</code> , <code>jmin</code> , <code>jres</code> — on every axis
Extra keys	<code>startDelay</code> on z-axes; <code>backlash</code> and <code>vstart</code> on d-axes	none
Always present	<code>setupTime</code>	<code>setupTime</code>

The mixed casing on Fixed-8 (`vMax` next to `amax`) is correct and intentional: each JSON key is the camelCase rendering of the key as it is stored, and the stored names really are `vMax` / `Amax` . Do not normalize one family's spelling into the other's.

Per-probe tips and tip state

On Fixed-8 and Span-8, `tip1 ... tip8` (and their runtime mirrors `rT_Tip1 ... rT_Tip8`) describe what is on each probe. Two things trip up readers:

- **They are `null` when nothing is mounted.** A Fixed-8 export and a Span-8 export configured for disposable tips both show `"tip1": null ... "tip8": null` and `"rT_Tip1": null ...`; a Span-8 with fixed tips installed shows populated objects. `null` means "no tip on this probe", not "field missing".
- **The tip discriminator is not on the outer object.** When populated, `tipN` contains a single `class` sub-object. `class` carries the tip-class scalars (`name`, `capacity`, `filtered`, `lls`, `geometry`) and usually a `$id`, because it is shared. The `_biomekType:"tip"` marker lives one level further down, on `class.template`, whose own `class` key is a `$ref` back to the enclosing class. Looking for `_biomekType:"tip"` at the top of `tipN` will always miss it.

For a Pod96 there are no per-probe tip fields at all — the whole head has one `tips` / `rT_Tips` value instead (also `null` when no tips are loaded).

Fixed vs. disposable probes

The discriminator is `tipN.class.fixed`, not whether `tipN` is populated. `null` only means "no tip on this probe right now", which is the normal resting state of a *disposable* probe. A probe configured for **fixed** tips carries a populated descriptor whose class has `"fixed": true` — this is what `biomek-i5-span8.json` shows on all eight probes, with the class named `Fixed100` . A disposable-tip Span-8 export instead shows `null` (or, if tips happened to be loaded at export time, a class with `"fixed": false`). Compare `biomek-i7-span8.json`, the same instrument with the tip configuration swapped to disposable.

The instrument's probe configuration wins; a method cannot override it. The pod reads each probe's fixed/disposable state from that per-probe tip object's class `Fixed` flag when it initializes, and no method JSON key feeds into that read. A protocol's tip choice does not change probe hardware — it is only checked against it:

- **Loading disposable tips onto a fixed probe fails at enqueue** with `Cannot load tips on fixed tips.` — raised for any selected probe that is configured fixed. Against an all- `Fixed100` export like `biomek-i5-span8.json` this means *every* probe, so no probe selection avoids it.
- **Unloading silently skips fixed probes** rather than failing: the discard mask is ANDed with "not fixed" and "has a tip", so an unload against an all-fixed pod is a no-op.
- The **Span-8 Load Tips / Unload Tips steps are offered at all** only when at least one Span-8 probe is non-fixed, which is why a disposable-tip method authored against a fixed-tip instrument looks valid on paper and fails on the machine.

How to request disposable tips against a fixed-tip export: you cannot, from method JSON. The probe configuration is instrument state, changed in Hardware Setup (and, on real hardware, by physically swapping the mandrels) — not by any key in a method. Either run the method on an instrument whose Span-8 probes are configured disposable, or change that instrument's probe configuration first. Read the target instrument's `podSettings.<pod>.settings.tip1...tip8` before authoring, and treat an all- `fixed: true` Span-8 as incompatible with a

disposable-tip protocol. The pod's `asText` carries the same fact in readable form — `Probe Configurations: 1: Fixed100 tips, 1 mL syringes.`

Runtime-state (`rT_*`) fields

The `rT_` prefix marks **runtime state** — a live snapshot at export time, not static configuration. The name varies by pod class:

- **Pod96** → `rT_Tips` (single tip collection for the whole 96-head).
- **Pod8Span, Fixed-8** → `rT_Tip1` ... `rT_Tip8` (per-probe).
- **All classes** also carry `rT_CurrentSpeed` .

Treat every `rT_*` value as "current at export time" — do not read one as fixed configuration.

Timestamps

`timestamp` is a `_biomekType:"dateTime"` object, not a bare string:

```
"timestamp": { "_biomekType": "dateTime", "value": "2026-08-20T13:49:57.0670000" }
```

Read the ISO-8601 string from `.value` . The same encoding applies to any other `dateTime` in the file.

What the pod `timestamp` actually means. It is generated when the export is produced, not when the pod was last edited in Hardware Setup: each pod's settings snapshot is stamped as it is extracted, so in a two-pod export `pod1` and `pod2` typically differ by a millisecond or two. Use it to date the *export*; do not read it as a configuration-change time. (`lastValidation` on a Span-8 is a different, genuinely persisted timestamp.)

The other category (brief)

- `deckLayouts` — decks defined on the instrument. **Fully specified, with its own JSON→map conversion recipe, in Deck Layouts Format Specification.** Not repeated here.

Value encoding (shared stack + instrument extensions)

Values are encoded exactly as in the method format, so nothing here is instrument-specific except the extra aggregate types listed below. The rules you need to read a pod-settings record are repeated in full here; the same rules are covered from the authoring side in Method JSON Structure §"Parameter Value Encoding" (in *Biomek JSON Authoring Guide*) / §"JSON Parsing Behavior".

Primitive encoding

- Strings, booleans and `null` are ordinary JSON primitives. Numbers are written locale-invariantly (always `.` for the decimal separator, never a thousands separator).
- **Numbers carry their COM variant type in their spelling.** A floating-point value always gets a decimal point, even when whole (`10` → `10.0`). An integer value is written bare (`100`). Because instrument values are variant-typed, *the same field can be a float on one pod class and an integer on another* — `gripperXOffset` is `0.29129999999999967` on a 96-head pod and a bare `0` on Fixed-8. Parse every numeric field as a number and never key off the presence of a decimal point.
- Floats are written at full round-trip precision — expect values like `6.375400000000001` . Do not assume the exporter rounds.
- A JSON object with **no** `_biomekType` is a plain nested dictionary.

`_biomekType` **discriminators from the shared stack**, all of which can appear in an instrument export:

<code>_biomekType</code>	Shape	Meaning
<code>"dateTime"</code>	<code>{ "_biomekType": "dateTime", "value": "<ISO 8601>" }</code>	A date/time. <code>value</code> is the ISO 8601 round-trip form, e.g. <code>2026-08-20T13:49:57.0670000</code> .
<code>"empty"</code>	<code>{ "_biomekType": "empty" }</code> — no other keys	The COM "empty" variant. Distinct from JSON <code>null</code> , which is the COM "null" variant.

<code>_biomekType</code>	Shape	Meaning
"missing"	<code>{ "_biomekType": "missing" }</code> — no other keys	The COM "missing"/parameter-not-supplied variant. Also distinct from <code>null</code> .
"nonFiniteNumber"	<code>{ "_biomekType": "nonFiniteNumber", "value": "NaN" }</code>	A non-finite float. <code>value</code> is the string <code>Infinity</code> , <code>-Infinity</code> , or <code>NaN</code> — JSON has no literal for these.
"comArray"	<code>{ "_biomekType": "comArray", "arraySubtype": "double", "values": [...] }</code>	A COM array, written as an object so its element type survives. <code>arraySubtype</code> is one of <code>boolean</code> , <code>byte</code> , <code>dateTime</code> , <code>double</code> , <code>integer</code> , <code>short</code> , <code>single</code> , <code>string</code> , <code>variant</code> . Optional <code>dimensionCount</code> (only when > 1) and <code>lowerBound</code> (only when non-zero) describe multidimensional arrays; <code>values</code> nests one array per dimension.
"labware", "technique", "SILASMessage"	see the method structure doc	Method-domain values that can be reached from instrument state.

A plain JSON **array** is an ordinary Biomek list, not a COM array — the two are different types and are spelled differently on purpose.

Instrument-only `_biomekType` values:

<code>_biomekType</code>	What it is
"device"	A device — also carries <code>_deviceType</code> = aggregate class name, normally a ProgID (see the <code>device</code> field schema below for the observed value).
"conversion"	Deck↔pod geometry / gripper conversion block.
"deckPosition"	A deck position (see the deck spec).
"tip"	A tip / tip-class definition.

An object aggregated with a class **not** in this table makes the export **fail loudly** rather than silently degrade — so if a file exists, all its aggregate objects were representable.

Field schemas for the instrument-only aggregates other than `deckPosition` (which is fully covered in the deck spec):

- **device** — appears on deck positions wired to a hardware device (trash and similar fixtures). Carries the standard `_biomekType:"device"` plus a `_deviceType` string identifying the device's aggregate class name — normally a ProgID. The only value present in the exports included with Biomek is on the trash devices attached to `TR*` positions:

```
json { "_biomekType": "device", "_deviceType": "Biomek5.TrashDevice" }
```

Other device classes (including devices not implemented in Biomek itself) can appear. Compare the string whole and treat it as an opaque identifier — do not build behavior on how the ProgID is spelled. Any additional keys under a `device` object are internal wiring state; treat them as opaque.

- **conversion** — a block encoding a deck↔pod geometry / gripper correlation table (used when reconstructing pod-relative coordinates). Its inner keys are geometry/calibration coefficients tuned per pod install. **Treat as opaque:** it plays no part in method authoring — do not read its fields. Note that the `_biomekType:"conversion"` marker is **not** universal: 96-head and Span-8 pods carry it on `settings.cmdGen.conversion`, but the Fixed-8's equivalent block is a plain dictionary with no `_biomekType` at all. Locate the block by path, not by discriminator.
- **tip** — a tip-class definition. It is not the per-probe wrapper under `settings.tip1 ... tip8`; it is what those wrappers bottom out in, at `settings.tipN.class.template` (see Per-probe tips and tip state). Carries the tip class name and its behavioral scalars (capacity, filtered flag, LLS capability, geometry). The `LLS` scalar records whether the tip class itself is LLS-capable; **liquid-level sensing is only supported by the Span-8 pod** — Fixed-8 and Multichannel pods do not perform LLS, regardless of the tip's `LLS` flag. Prefer the same tip-class facts from the project-items export (`../project-items/`) when authoring; the copy inside pod settings is a runtime snapshot.

If you encounter an unfamiliar key under any of these aggregates, treat it as opaque rather than inventing a meaning — the categories above are not exhaustively field-schemad because their contents are internal instrument state, not method-authoring input.

References: \$id / \$ref — you MUST resolve these

The instrument object graph is **cyclic and full of shared objects** (a pod settings object may be reached from two places; a deck position points at its group). JSON cannot nest a cycle, so the serializer uses JSON-Reference metadata:

- An object reached **once** is written **inline** with no metadata.
- An object reached **more than once** is written **in full at its first occurrence** with an added `"$id"`, and **every later occurrence becomes** `{ "$ref": "<pointer>" }`.
- `$id / $ref` values are **JSON-Pointer-style strings** rooted at `#`, each segment URL-escaped and camelCased (space → `%20`, `/` → `~1`, `~` → `~0`), with numeric list indices. A real pointer looks like `#/podSettings/pod1/settings/tip1/class`. The escaping is **full** URL escaping, not just spaces — `#` → `%23`, `?` → `%3F`, `%` → `%25`, `é` → `%C3%A9` — so decode with a general URL-decoder rather than special-casing `%20`. Pod keys themselves never contain a space, so `%20` will not appear in a pod pointer — but implement the unescaping anyway: property names deeper in the graph do contain spaces (e.g. the deck-position `characteristics` keys `can Discard Tips` and `disallow AutoTeach`), and so can names in the other two categories.
- References can **cross categories** — the same object can be reachable from both `podSettings` and `deckLayouts`, and the second occurrence becomes a `$ref` into the first.
- Only **objects** carry reference metadata. A shared **array** is written inline at each occurrence (duplicated), never as a `$ref`.
- `$id / $ref` are **reserved** — real payload never uses those keys.

How to resolve a \$ref while reading: treat the pointer as a path from the document root (`#`), split on `/`, URL-decode each segment (`%20` → space, `~1` → `/`, `~0` → `~`), match keys case-insensitively (they are camelCased), and read the object carrying the matching `$id`. E.g. `#/podSettings/pod1/settings` = the `settings` object under `pod1`. An interpreter that does not follow `$ref` will read incomplete records.

Part 2 — reading pod settings out of the export

This is the extraction recipe. It uses `jq` paths for concreteness; `jq` does **not** follow `$ref`, so when a value is `{ "$ref": "#/..." }` read the target path directly (see the resolution rule above).

Worked-example note. The snippets and values below are from a **sample i7-Hybrid export** (two pods: Pod1 = 96-head, Pod2 = Span-8). Use the values as *representative* shapes; field names carry over directly from the live object graph, but per-instrument values differ. See Representative instrument configurations below for the roster.

Step 1 — confirm the envelope and list categories

```
jq -r 'keys[]' <export>.json
# → deckLayouts, format, formatVersion, podSettings
jq -r '.format, .formatVersion' <export>.json
# → "Biomek Instrument Settings" "1.0"
```

If `podSettings` is absent, that folder was not exported (or the instrument had nothing) — it does **not** mean the machine has no pods.

Step 2 — enumerate pods and read each pod's class

`podSettings` is keyed by pod name. Read the **class** from `settings.type` (not `settings.podType`):

```
jq -r '.podSettings | to_entries[]
      | "{key}: type={value.settings.type} podType={value.settings.podType}"' \
    <export>.json
# pod1: type=Pod96 podType=MC (example)
# pod2: type=Pod8Span podType=SPAN8 (example)
```

Sample result: two pods — `pod1 = Pod96`, `pod2 = Pod8Span` — matching the i7-Hybrid mapping (Pod1 = 96-head, Pod2 = Span-8). Read the installed pods from the `pod1 / pod2` entries; any other key is a saved pod-settings profile, not an installed pod (see The pod-settings

category).

Cross-check with the human summary — often the fastest sanity check on Pod96 and Pod8Span:

```
jq -r '.podSettings.pod1.asText' <export>.json
# Pod96 → "Validation Time: Not Validated ... Head Type: 300 µL MC-96 Head ... X: [ 10.52 , 108.27 ] ..."
# Pod8Span → "Validation Time: Not Validated ... Probe Configurations: 1: Fixed100 tips, 1 mL syringes. ..."
# Fixed-8 → "Fixed-8 pod settings" (stub – read settings directly instead; note Fixed-8 has no headType)
```

Step 3 — read a pod's structured settings

```
jq -c '.podSettings.pod1.settings
| {type, podType, headTypeLabel, min, max,
  gripperXOffset, gripperYOffset, gripperZOffset, speedLimit}' \
<export>.json
```

- `type` → pod class; `headType` / `headTypeLabel` → mounted head.
- `min` / `max` → axis travel limits; `gripper*Offset` → gripper calibration.
- `rT_*` fields (e.g. `rT_Tips`) are **runtime state**, current at export time — don't treat them as static configuration.

Step 4 — read the export timestamp

```
jq -r '.podSettings.pod1.timestamp.value' <export>.json
# → 2026-08-20T13:49:57.0670000
```

(`timestamp` is a `_biomekType:"dateTime"` object; the ISO string is under `.value`. It records when the export was produced, not when the pod was last configured — see Timestamps.)

Worked example — an i7-Hybrid pod-settings record (representative)

A single pod entry looks like this (trimmed to the load-bearing fields):

```
"pod1": {
  "asText": "Validation Time: Not Validated ... Head Type: 300 µL MC-96 Head ... X: [ 10.52 , 108.27 ] ...",
  "settings": {
    "type": "Pod96",           // ← pod class: read THIS
    "podType": "MC",          // ← pod-family code, NOT the class
    "headType": "MC96_300",   // ← head code
    "headTypeLabel": "300 µL MC-96 Head",
    // Note the gripper axes: a reader that only looks at x/y/z misses gg/gr/gy/gz.
    "min": { "d": -82.66380166, "gg": 6.039, "gr": -370.0, "gy": 15.834,
              "gz": 6.943, "x": 10.52, "y": 14.596, "z": 10.996 },
    "max": { "d": 300.0, "gg": 15.739, "gr": 190.0, "gy": 62.683,
              "gz": 37.949, "x": 108.2689, "y": 59.591, "z": 39.863 },
    "gripperXOffset": 0.29129999999999967, "gripperYOffset": 0.0,
    "gripperZOffset": -7.2799999999999999,
    "speedLimit": 100.0,           // ← percent, not mm/s
    "rT_Tips": null               // ← runtime tip state; null = no tips loaded
  },
  "timestamp": { "_biomekType": "dateTime", "value": "2026-08-20T13:49:57.0670000" }
}
```

→ Read as: **Pod1 is a Pod96 (96-channel) pod with a 300 µL MC-96 head**, snapshotted 2026-08-20. The companion `pod2` in this sample is a `Pod8Span`. Values shown are from a sample i7-Hybrid export — regenerate against your own file for real numbers.

Reading pitfalls (common mistakes)

- **Using `settings.podType` for the pod class.** It is a pod-family code (`MC` / `SPAN8`), not the class. Read `settings.type` (`Pod96` / `Pod8Span` / `Fixed8`).
- **Not following `$ref`.** Shared/cyclic objects live at their `$id` site and appear elsewhere as `{ "$ref": ... }`. `jq` won't follow them — resolve the pointer yourself. Shared **arrays** are duplicated inline, not ref'd.
- **Treating `rT_*` fields as static config.** They are runtime state, current at export time.

- **Assuming a missing category means a missing capability.** `podSettings` / `deckLayouts` are dropped when not selected or empty — absence is not evidence.
- **Trying to import the file.** It is export-only (`FromJson` throws); treat it as a read-only snapshot.
- **Looking for decks here.** Deck layouts are in the same file but specified in Deck Layouts Format Specification.

Representative instrument configurations

The pod-settings tables in this document are grounded in seven representative instrument configurations, each defined by a `.bif` instrument-configuration file. They are a sample chosen to cover the pod and chassis combinations — **not an exhaustive catalog**. A real instrument can vary in ways the export reflects: an i5 Span-8, for example, can carry either fixed or disposable tips, and the roster shows just one such variant per row. Use it to find a configuration close to the instrument you are authoring for, then read the authoritative values from that instrument's own export. Re-exporting the same instrument reproduces the same structure and the same values, with one exception: each pod's `timestamp` is stamped at export time, so it will differ (see Timestamps).

Configuration	Instrument	Pods	Deck layouts
i3	i3 (Fixed-8)	<code>pod1 = Fixed8</code>	<code>i3</code> , <code>standard</code>
i5 Multichannel	i5 Multichannel	<code>pod1 = Pod96 (MC)</code>	<code>multichannel</code> , <code>standard</code>
i5 Span-8	i5 Span-8	<code>pod1 = Pod8Span (SPAN8)</code>	<code>span8</code> , <code>standard</code>
i7 Multichannel	i7 Multichannel (single MC pod)	<code>pod1 = Pod96 (MC)</code>	<code>dualMultichannel</code> , <code>hybrid</code> , <code>multichannel</code> , <code>span8</code> , <code>standard</code>
i7 Span-8	i7 Span-8, tip-config swapped to disposable	<code>pod1 = Pod8Span (SPAN8)</code>	same as i7 Multichannel
i7 Hybrid	i7 Hybrid (MC + Span-8)	<code>pod1 = Pod96 (MC)</code> , <code>pod2 = Pod8Span (SPAN8)</code>	same as i7 Multichannel
i7 Dual-MC	i7 Dual-MC, head types differ across pods	<code>pod1 = Pod96</code> , <code>pod2 = Pod96</code>	same as i7 Multichannel

Notes on provenance:

- **All i7 configurations carry the same five default deck layouts** (`dualMultichannel` , `hybrid` , `multichannel` , `span8` , `standard`) regardless of which pod configuration is installed. The simulated pod-type change was made in Hardware Setup only; the default/standard i7 deck was not swapped even though a real hybrid deployment might do so. Treat the deck list as instrument-class geometry, not pod-derived.
- **The i7 Span-8 configuration** differs from a nominal i7-Span baseline only in the pod's tip configuration (disposable rather than fixed).
- **The i7 Dual-MC configuration** has two `Pod96` pods; their `headType` / `headTypeLabel` differ.
- i5 and i3 configurations carry only their two native decks each; i7 configurations carry all five because the chassis supports every layout.

[†] Table of contents

Deck Layouts Format Specification

What this document is. How to read a deck out of a Biomek Instrument Settings JSON export. The deck lives inside the export as `deckLayouts.<deckKey>` — this spec defines the envelope around it, the per-deck object, and the per-position record shape (fields, types, and the `$id` / `$ref` reference model). Values shown against the reference export are copied verbatim.

The export may carry more keys than are documented here, and it shares objects via `$id` / `$ref`. The keys listed below are the ones you need to read to understand a deck — what ALPs are on it, where each position sits, what labware it accepts, and how it is wired to hardware. Any additional key you see in a real export is internal book-keeping; treat it as opaque and ignore it.

What a deck is

A **deck** is a **permanent, reusable arrangement of ALPs** (Automated Labware Positioners) bolted to a Biomek instrument at **fixed physical coordinates**. It is built once in the **Deck Editor** utility (`Utilities` ▶ `Instrument` ▶ `Deck Editor`) and shared across many methods. Each ALP occupies one or more numbered **positions** (`P1` , `P2` , ... plus named fixtures like `WS1` , `TR1`). The common ALP types included with Biomek are listed in ALP Catalog. Additional ALP types beyond this list may appear on a given system (for example add-on hardware or integrated devices):

Permanent deck (this format) vs. temporary labware (NOT this format):

	Permanent deck	Temporary labware
Built in	Deck Editor utility	Instrument Setup method step
What it defines	ALPs at fixed coordinates (the board)	Plates/tip boxes/reservoirs placed on ALPs for one run (the pieces)
Lifetime	Permanent, reused by every method	Per-run, per-method
Serialized as	<code>deckLayouts</code> in an Instrument Settings export (this doc)	<code>deckItems</code> in method JSON (a different format)

One-liner: **the Deck Editor builds the board (ALPs); Instrument Setup places the pieces (labware) on it.** This document describes the *board*. When a method authors labware placement it writes `deckItems` keyed by the position names that this deck defines — it never edits the deck. `deckItems` is a method-JSON structure and is specified elsewhere: see Instrument Setup (in *Biomek JSON Authoring Guide*) for the step that writes it and Concept Guide: Labware & Position Resolution (in *Biomek JSON Authoring Guide*) for how a position name resolves to labware at run time.

Why this format is useful:

- To turn a raw exported deck into a clean `position` → `ALP type` → `coordinate` → `accepted-labware` table.
- To know which positions exist, i.e. which `Position` / `where` values are valid when authoring a method against this instrument.
- To understand that placing labware is temporary and happens via Instrument Setup (`deckItems`), not by changing the deck.

The illustrative resolved views under `samples/` show one of these reads applied to each default deck included with Biomek — same key names as the raw export, with `$ref` pointers dereferenced and the internal book-keeping keys elided.

Where deck data comes from: the Instrument Settings JSON export

Decks are **not** exported as standalone files. They live inside a **"Biomek Instrument Settings"** JSON export.

The examples throughout this document are drawn from the **i7 Hybrid** default configuration (five decks: `dualMultichannel` , `hybrid` , `multichannel` , `span8` , `standard`). The other six representative configurations cover i3, i5 Multichannel, i5 Span-8, i7 Multichannel, i7 Span-8 (disposable-tip), and i7 Dual-Multichannel — see the pod-settings spec's "Representative instrument configurations" section for the roster.

The `standard` deck is included as a read-only default. It is never the actual deck being used and should not be selected in an **Instrument Setup** step.

Top-level envelope

```
{
  "format": "Biomek Instrument Settings",
  "formatVersion": "1.0",
  "podSettings": { ... },
  "deckLayouts": { ... }
}
```

The category names are fixed: the category `"Deck Layouts"` maps to the JSON key `deckLayouts`. Both category keys — `podSettings` and `deckLayouts` — are **optional**: a category appears only if the user selected one or more objects of that type at export time, so be prepared for an Instrument Settings file that omits `deckLayouts` entirely. The reference file has `podSettings` + `deckLayouts`.

deckLayouts

`deckLayouts` is an object keyed by **deck name** (camelCased — see below). The reference export `biomek-i7-hybrid.json` contains five: `dualMultichannel`, `hybrid`, `multichannel`, `span8`, `standard`. Each deck object has:

Key	Type	Meaning
<code>name</code>	string	Original deck name, un-camelCased (e.g. <code>"Standard"</code> , <code>"DualMultichannel"</code>)
<code>isDefault</code>	bool	Whether this is the instrument's default deck
<code>isReadOnly</code>	bool	Read-only (factory) deck vs. user-editable
<code>groups</code>	object	ALP hardware definitions, keyed by group name — all \$ref (see reference model)
<code>positions</code>	object	The placed positions, keyed by position name — the authoritative per-position records

Finding and using the instrument's default deck

An instrument defines one or more decks, and one of them is its **default**. In the export, the default deck is the deck object under `deckLayouts` whose `isDefault` is `true`. Reading the export first is the reliable way to know both **which deck names an instrument accepts** (the keys of `deckLayouts`) and **which one is the default**.

- **Find the default:** scan `deckLayouts` for the entry with `"isDefault": true`; that entry's name is the instrument's default deck.
- **Author against it:** put that deck name in the Instrument Setup step's `layout` key and use it as the method's default deck (name matching is case-insensitive). Targeting a different deck the same instrument defines is fine — just use that deck's name instead. A `layout` value the instrument does not define fails at enqueue. Do not use multiple decks in the same method.

You need the instrument's **Instrument Settings JSON export** to determine its decks and its default. If you do not have that export on hand, produce one first — in the Biomek software, *File ▶ Export ▶ Instrument Settings ▶ Save As JSON* — and read the default deck from it as above; without it there is no way to know which deck names a given instrument will accept.

Key-naming and type conventions (must understand to parse)

These three rules are applied uniformly by the shared provider stack (`PersistenceFormats/TypeProviders/`) and explain everything that looks odd in the raw JSON.

1. CamelCase key conversion

Every dictionary key is passed through `JsonNamingPolicy.CamelCase`, which lowercases only the first PascalCase word and preserves the rest verbatim (see Base Keys & Serialization Conventions (in *Biomek JSON Authoring Guide*) §1). So:

- Field names: `ALPType` → `alpType`, `PermanentLabware` → `permanentLabware`, `Positions` → `positions`, `X1` → `x1`, `Framed1` → `framed1`, `TemplatePositionName` → `templatePositionName`.

- **Position/deck keys too:** `P1` → `p1`, `S1` → `s1`, `WS1` → `wS1`, `TL1` → `tL1`, `TR1` → `tR1`, `W1` → `w1`; `deck` `Standard` → `standard`, `DualMultichannel` → `dualMultichannel`. Verified in the reference file (`positions` keys of the `standard` `deck`: `p1` ... `p30`, `tL1` ... `tL5`, `tR1`, `tR2`, `w1`, `wS1` — 39 keys).

Method `deckItems` use the same convention (e.g. `deckItems.p5`), so the position keys line up between deck and method JSON — see Instrument Setup (in *Biomek JSON Authoring Guide*).

2. `_biomekType` type tag

Objects that wrap a Biomek aggregate class carry a `_biomekType` string. A plain dictionary has **no** `_biomekType` (kept concise). Friendly names seen in a deck export:

<code>_biomekType</code>	Where it appears in a deck
<code>deckPosition</code>	every entry under <code>positions</code>
<code>labware</code>	permanent labware on a wash station (<code>position.labware</code>)
<code>device</code>	positions wired to a device — carries an extra <code>_deviceType</code> naming the device class (see "The <code>device</code> block")
(<i>none</i>)	the deck object, <code>groups</code> , <code>characteristics</code> , <code>required</code> , <code>pins</code> , <code>regions</code> , ... (plain dictionary — no <code>_biomekType</code>)

3. `$id` / `$ref` reference model — resolve every reference

The instrument object graph is cyclic and shares objects, so the serializer uses reference tracking. A shared object is **materialized once** (first occurrence, tagged with `$id` = a JSON Pointer into the document) and appears everywhere else as `{"$ref": "<same pointer>"}`.

In a deck this appears in three places. An interpreter that does not follow `$ref` produces an incomplete deck.

1. **Top-level groups.* are all `$ref`.** The group object is materialized inline under its host position. Example from the `hybrid` `deck`:
`groups.static1x1_1 = {"$ref": "#/deckLayouts/hybrid/positions/p1/group"}`.
2. **position.group** holds the inline group object (with `$id`) on the host position; `position.group.positions.<name>` lists the member positions, back-referencing the host via `$ref`.
3. **Multi-position ALPs (e.g. `Static1x3`) hide their secondary members.** A `Static1x3` group spans three positions (e.g. `p3`, `p4`, `p5`). Only one member — the *host* — carries a full record at `positions.<key>`; the full records for the other two are materialized *inside* `positions.<host>.group.positions`, and their top-level entries are pure `{"$ref": "#/deckLayouts/hybrid/positions/p3/group/positions/p4"}`. In the reference `hybrid` `deck`, the secondary members of each `Static1x3` ALP appear as pure `$ref` at the top level.

The host is whichever member key sorts first in the export's key order, which is ordinal string order, **not** numeric order. In the reference `hybrid` `deck` the `p8` / `p9` / `p10` ALP is hosted by `p10` (because `"p10" < "p8"` as strings), so the full `p8` and `p9` records live under `positions.p10.group.positions`. Never assume the lowest-numbered member is the host — follow the `$ref`.

Resolution rule: if `positions.<key>` has only a `$ref`, dereference the pointer (a `#/...`-rooted JSON Pointer) to get the real record before reading its fields.

Pointer segments carry two-stage escaping — RFC 6901 (`~` → `~0`, `/` → `~1`) wrapped in full URL escaping — but no `$id` / `$ref` value in any of the seven default instrument exports contains a `~` or a `%`, so plain `/`-splitting resolves every pointer in them. Implement the full unescape only if you want to be strict about arbitrary keys. The pod-settings spec states the same rule for the shared envelope.

The position record (`positions.<key>`)

The authoritative per-position object. Fields present in the reference file:

Field	Type	Meaning / how to use
<code>_biomekType</code>	<code>"deckPosition"</code>	type tag
<code>name</code>	string	Display name, original casing (<code>"P1"</code> , <code>"WS1"</code>). Use this name to reference a position in method steps.

Field	Type	Meaning / how to use
baseName	string	Family prefix ("P" , "WS" , "TR")
alpType	string	ALP type — typically one of the common ALP-type names listed under "What a deck is", verbatim ("Static1x1" , "Static1x3" , "WashStation96" , "TrashLeftSlide" , ...); other values may appear for add-on hardware or integrated devices
templatePositionName	string	Internal template cell (often "A1"); not a reliable grid coordinate. Ignore this value.
x1 , y1 , z1	number (cm)	Pod 1 calibrated access coordinate, in centimeters
x2 , y2 , z2	number (cm)	Pod 2 calibrated access coordinate, in centimeters
xSpan , ySpan , zSpan	number (cm)	Footprint extent
framed1 , framed2	string	Framing state per pod ("Not" = not framed/calibrated). Methods should prefer to reference framed positions for a given pod when possible.
permanentLabware	bool	<code>true</code> ⇒ labware is permanently installed here (wash stations)
labware	object null	The installed permanent labware when <code>permanentLabware=true</code> (a <code>_biomekType:"labware"</code> object with <code>class</code> , <code>properties.liquidType</code> , <code>volumeType</code>); <code>null</code> otherwise
required	object	What labware this position accepts — keys are constraint names (e.g. "standard TiterPlate Size" , "no plates allowed"); values <code>null</code>
characteristics	object	Capability flags — keys are characteristic names (e.g. "multichannel Can Load Tips" , "span Can Load Tips" , "fixed-8 Can Load Tips" , "wash Station" , "can Discard Tips" , "disallow ManualTeach"). The values are always <code>null</code> in an instrument-settings export; the capability is signaled by the presence of the key , not by its value — read the key names, and ignore the <code>null</code> . A characteristic states a capability, not reachability : a pod that a position is built to serve may still be unable to reach it on a given instrument — see Reachability and Access.
adaptorPlate	object	Framing/setup instruction (<code>bitmap</code> , <code>caption / message</code> , <code>visible</code> , <code>x / y / z</code> , and on tip-load positions <code>frameWithGripper / frameWithTips</code>) — e.g. WS1: " <i>Lift the wash station off its base and replace it with the AccuFrame.</i> " Irrelevant to method authoring.
group	object	The ALP group this position belongs to (inline with <code>\$id</code> , or <code>\$ref</code>) — see "The group record" below
device	object null	Device wiring; <code>null</code> on ordinary positions — see "The device block" below
deviceIndex	int	Index of this position within the device's <code>deckPositions</code> ; -1 when there is no device
canCopyDevice	bool	Present (and <code>true</code>) on trash positions. Marks the <code>device</code> block as fixed to the deck — carried with the deck itself rather than bound to a separately-installed device
labwareOffsets	object	Per-labware-class placement corrections. Keys are camelCased labware class names ("bC1025F" , "agilentReservoir" , ...); each value is { "x":..., "y":..., "z":... } in cm. { } on most positions; populated on tip-load (<code>tl*</code>) and reservoir positions
obstacles	object	Present on trash positions. The ALP's walls, keyed <code>back</code> , <code>front</code> , <code>left</code> , <code>right</code> ; each value has <code>position</code> (a <code>\$ref</code> back to the owning position) plus <code>xoffset / yoffset / zoffset</code> and <code>xSpan / ySpan</code> in cm, relative to the position origin
wasteX	number (cm)	Present on wash stations. X distance from the center of the wash points to the center of the drain hole
tipDisposalOffsetX/Y/Z , span8TipDisposalOffsetX/Y	number (cm)	Present on trash positions. Where each pod type releases tips relative to the position
minSafeHeight , offsetX/Y/Z , labwareX/Y/Z , podAccessResource , deck , stack , rT_Labware , _RT_Stack	various	Runtime/geometry internals; not needed for a position→ALP map

Keys are omitted when they carry no value, so a position record is not a fixed shape: read defensively and treat a missing key as "not applicable to this ALP".

Tip-load characteristics are asymmetric. The two tip-load capability keys do not appear on the same set of positions: `multichannel Can Load Tips` is present **only** on the dedicated tip-load (`tl*`) positions, while `span Can Load Tips` is present on those **and** on every plain `p*` position. So a Multichannel tip box is confined to the `tl*` positions, whereas a Span-8 tip box may be placed on any `p*` position. Because these are capability flags, the `span Can Load Tips` on a `tl*` position does not mean the Span-8 can *reach* it — on a two-pod instrument it often cannot. See Reachability and Access §"Tip loading: capability is not reachability" and the i7 table in Pod Reach Envelopes (i7 worked example).

Note that for the Multichannel pod, if a tip box is placed on a position that is **not** a TipLoad1x1 ALP, the software will automatically move it to the TipLoad1x1 in order to load tips, if possible.

The `device` block

`device` is an **object**, not a string, and binds a position to installed hardware. On an ordinary position it is `null`. Positions wired to hardware (trash bins and slides, heaters/shakers, and other device ALPs) carry a full `_biomekType: "device"` object; `_deviceType` names the device class. The `standard` deck's `tr1` reads:

```
{
  "_biomekType": "device",
  "_deviceType": "Biomek5.TrashDevice",
  "deckPositions": [
    { "$ref": "#/deckLayouts/standard/positions/tr1" }
  ],
  "isReadOnly": true,
  "name": "Trash"
}
```

`name` is what a step refers to. `deckPositions` is an array of `$ref` back to every position the device covers, so the link is bidirectional; `deviceIndex` on the position is its slot in that array. `isReadOnly` marks a factory-defined device. The illustrative sample views collapse `device` to its `name` string.

Devices are installed and configured on the instrument, not created by a method. To find what yours has, scan the export for positions whose `device` is not `null` — an ALP that *can* host a device (orbital-shaker or heat/cool platform) is not the same as one being installed on it.

The default-instrument samples here carry only a trash device; a system with a shaker, Peltier, or bar-code reader shows other `_deviceType` values at those positions. The i3 sample has no `device` block on any position — its tip trash is a plain ALP.

Which steps address a device by position, and which by name

What you write depends on the step:

Step	You write	Resolved from
INHECO Peltier (in <i>Biomek JSON Authoring Guide</i>)	a deck position (<code>position</code>)	the device object on that position
Device Action (in <i>Biomek JSON Authoring Guide</i>)	a device name (<code>device</code>)	the pipettor's device collection
Fly-By Read (in <i>Biomek JSON Authoring Guide</i>)	a device name and a deck position	the collection, then checked for a position association
SILAS (in <i>Biomek JSON Authoring Guide</i>)	a module name (<code>module</code>)	the configured SILAS modules

Peltier does not take a device name: you name the position, and the step reads the device off it at enqueue. It fails in one of three ordered ways, and only the first is fixable by editing the method:

```
P3 is not a valid position name for the current deck.
P3 does not have a device associated with it.
Location does not have an Inheco Peltier device associated with it.
```

The first means the name is wrong, or right for a different deck than the one resolved. The second means the position is real but nothing is wired to it — **not fixable from the method**: placing an ALP there does not associate a device with it, and no method key binds one. The

third means a device is associated but is not a Peltier; the message says `Location` literally rather than naming your position.

Fly-By Read is the opposite: a device that exists but has no position association gives

The device named "<name>" is not associated with a deck position. Please use the deck editor to associate "<name>" with a position

Both are instrument-configuration messages — check the export before authoring a device step.

SILAS is outside this model. A module is not read off a position's `device` block; the modules are a separate configured collection.

The group record (`positions.<key>.group`)

The group is the ALP hardware definition. It is materialized inline (with `$id`) on its host position and appears as `$ref` everywhere else, including under `deckLayouts.<deck>.groups` .

You'll see a `group` on every position in your real export. It is the shared ALP definition (a *position group*) linked across the ALP's positions via `$id` / `$ref` — per-ALP hardware data repeated by reference, not per-position placement. The illustrative views under `samples/` omit it; for what each ALP type is, see ALP Catalog.

Field	Type	Meaning / how to use
<code>name</code>	string	Group instance name, original casing (<code>"Static1x1_1"</code> , <code>"TrashLeftSlide_1"</code>)
<code>type</code>	string	The ALP type — same value as the member positions' <code>alpType</code>
<code>column</code> , <code>row</code>	int	Grid indices of the ALP on the deck. Read these; do not compute letters
<code>positions</code>	object	Member positions, keyed by position name — the host as <code>\$ref</code> , the secondaries as full inline records
<code>categories</code>	object	Set membership; keys are <code>basic</code> , <code>wash</code> , <code>trash</code> , <code>span8</code> , <code>tubes</code> , <code>devices</code> (values <code>null</code>). A group may carry more than one, e.g. <code>{"span8": null, "trash": null}</code> on a trash slide
<code>simulatorRenderStyle</code>	string	How the simulator draws the ALP. Irrelevant to method authoring.
<code>pins</code>	object	The ALP's deck mounting-point footprint , not a well grid. <code>colCount</code> / <code>rowCount</code> = mounting points spanned by the pin bounding box; <code>offsetX</code> / <code>offsetY</code> = cm from the ALP origin to the top-left corner of that bounding box; <code>identifier</code> = X distance in <i>pitches</i> from the left edge of the bounding box to the pointing feature on the ALP's front face
<code>regions</code>	array	Where on the deck the ALP may be placed, expressed against the deck's own grid. Each entry has <code>offsetFromDeckBackRow</code> , <code>offsetFromDeckFrontRow</code> , <code>offsetFromDeckLeftCol</code> , <code>offsetFromDeckRightCol</code> — 0-based counts of mounting points inward from each deck edge. Each axis needs two of three keys; when only one edge offset is given, <code>absColCount</code> / <code>absRowCount</code> supplies the span (end-columns/rows inclusive), as on the trash slides
<code>trashLeftRightWallZSpan</code>	number (cm)	Trash groups only. Height from the top of the deck plate to the top of the trash bin
<code>x1</code> , <code>x2</code> , <code>y1</code> , <code>y2</code> , <code>z1</code> , <code>z2</code> , <code>xSpan</code> , <code>ySpan</code> , <code>zSpan</code>	number (cm)	The ALP exterior. These differ from the member position's own coordinates, which describe the labware seat inside the ALP

Coordinates and grid

- **Units are centimeters**, stored verbatim (no conversion). Coordinates are calibration data: read them from the file you were given, never from another instrument's export.
- **Grid indices** come from the group's explicit integer `column` / `row` — read them; do **not** try to recompute letters. The Deck Editor UI shows only *sparse* column letters (`A` , `F` , `M` , `T` , `AA` , `AH` , `A0` , `AV` , `BC` , `BJ` , `BQ`) and rows (`5` , `10` , `15` , `20` , `25` , `30`) as a display overlay; those labels are coarser than the raw indices and are not needed to identify a position.

The `samples/*.deck.md` files

The seven files under `samples/` — one per default deck included with Biomek — are **illustrative resolved views**, not raw dumps. Each file mirrors the shape of `deckLayouts.<deckKey>` in the export, with `$id / $ref` dereferenced inline and the hardware-internal and geometry-only keys elided so the deck's `positions` map fits on the screen. Coordinates are copied verbatim from the export (**cm**, not mm); `alpType`, labware `class`, `device name`, `required` and `characteristics` are copied verbatim.

Read them as **guides to the keys**, not deterministic parser output — a real export carries `$id / $ref` object sharing and additional book-keeping keys that these views deliberately do not show. When a value is load-bearing (a coordinate, a labware class, a device name), always confirm against the JSON export itself.

† Table of contents

ALP Catalog

This catalog covers the common default ALPs (Automated Labware Positioners) whose `alpType` values can appear in a Biomek deck layout — including the ALPs that no sample deck in `samples/` exercises. When you encounter an `alpType` in a `deckLayouts` JSON export, this is a good starting reference for what that ALP is and which instruments it is typically used on.

This list is not exhaustive. Additional ALP types beyond those listed here may appear on a user's system — for example, ALPs installed as part of add-on hardware kits or integrated third-party devices. Treat this catalog as a description of the common defaults, not a closed enumeration of every `alpType` you might encounter.

For the JSON field layout that surrounds `alpType`, see Deck Layouts Format Specification. For quick visual examples of the default decks included with Biomek, see `samples/`.

Hardware-compatibility conventions used below

- **Biomek i3** — compact chassis with a Fixed-8 pod. ALPs whose `alpType` ends in `_i3` are the i3-chassis variants and are only used on i3 decks.
- **i5 and i7** — Each i5/i7 instrument carries either a Multichannel pod (96-tip or 384-tip head) or a Span-8 pod, or both (Hybrid / Dual-Multichannel).
- "Any pod" means the ALP is a passive holder or device platform that can be reached by whichever pod the instrument is equipped with; a pod-specific note calls out ALPs (wash stations, tube racks) whose labware or geometry is tied to one pod family.

The compatibility column below reflects each ALP's **default applicability** as expressed by its category tags and its type-name convention; it is not an absolute mechanical constraint enforced by the instrument.

Common ALP types

<code>alpType</code>	What it is	Hardware compatibility
<code>Static1x1</code>	Passive holder that presents one titer-plate-sized position.	i5 and i7; any pod.
<code>Static1x3</code>	Passive holder with three titer-plate positions stacked front-to-back on the same footprint.	i5 and i7; any pod.
<code>Static1x5</code>	Passive holder with five titer-plate positions stacked front-to-back — the largest tandem static holder.	i5 and i7; any pod.
<code>Static1x1_i3</code>	Single-plate passive holder sized and mounted for the compact i3 deck grid.	Biomek i3 chassis (Fixed-8 pod).
<code>Short1x1_i3</code>	Low-profile single-plate holder for the i3 — reduced Z so the Fixed-8 pod can reach plate labware inside its limited Z-travel range. Used to hold tall tip boxes (such as BC1070).	Biomek i3 chassis (Fixed-8 pod).
<code>TipLoad1x1</code>	Single-plate holder designed for the Multichannel pod to load tips from. Multichannel pods cannot load tips from any other ALP type. Can also be used for pipetting or Span-8 tip loading.	i5 and i7; supports tip loading with both Multichannel and Span-8 pods.
<code>TipTrash_i3</code>	Discarded-tip trash for the i3, feeding a bag or bin. Not a labware position — accepts no plates and cannot be framed.	Biomek i3 chassis (Fixed-8 pod).
<code>WashStation96</code>	Circulating wash reservoir sized for a 96-tip Multichannel pod.	i5 or i7 with a 96-tip Multichannel pod.
<code>WashStation384</code>	Circulating wash reservoir sized for a 384-tip Multichannel pod.	i5 or i7 with a 384-tip Multichannel pod.
<code>WashStationSpan8</code>	Dispense-only wash reservoir with eight wash points for the Span-8 probe pod. Narrow tower footprint mounted off to the side of the deck.	i5 or i7 with a Span-8 pod.

alpType	What it is	Hardware compatibility
WashStationSpan8Active	Circulating wash reservoir for the Span-8 pod. Same envelope as the passive Span-8 wash station.	i5 or i7 with a Span-8 pod.
TrashLeftBin	Bin-style trash chute mounted on the left side of the deck. Accepts discarded tips and labware.	i5 and i7; default trash accessory (categorized with Span-8 in the Deck Editor).
TrashLeftSlide	Slide-style trash chute mounted on the left side of the deck — same function as <code>TrashLeftBin</code> with a slide form-factor.	i5 and i7; default trash accessory.
TrashRightBin	Bin-style trash chute mounted on the right side of the deck (mirror of <code>TrashLeftBin</code>).	i5 and i7; default trash accessory.
TrashRightSlide	Slide-style trash chute mounted on the right side of the deck (mirror of <code>TrashLeftSlide</code>).	i5 and i7; default trash accessory.
TubeRack	Holder for a tube-rack labware — the entry point for single-tube pipetting with the Span-8 probes.	i5 or i7 with a Span-8 pod.
OrbitalShaker	Mounting platform for an orbital shaker device that agitates a plate in place.	i5 and i7; any pod (device platform).
HeatOrCool	Mounting platform for a Peltier-based heat/cool device that regulates a plate's temperature.	i5 and i7; any pod (device platform). Also qualified for Span-8 tip pickup.
ReservoirTipBox	Mounting platform for a circulating-reservoir labware that can hold a tip box on top of it (for on-deck bulk-liquid supply).	i5 and i7; any pod (device platform).
PositivePositioner	Mounting platform for a clamp device that mechanically pushes a plate against known X/Y stops so its origin is exact.	i5 and i7; any pod (device platform).
FBBCR	Mounting platform for a bar-code reader that scans plate labels. Presents a scan-target position, not a labware seat.	i5 and i7; any pod (device platform).

Notes

- **Multi-position ALPs** (`Static1x3` , `Static1x5`) contribute more than one usable position on the deck; each member position has its own key in the JSON export. See the format spec's group / position sections for how the members are laid out.
- **Permanent-labware ALPs** — the active wash stations — come with a fixed piece of labware pre-installed at the position; method authors don't place labware on them via Instrument Setup.
- **Read-only trash devices** — the four `Trash*` ALPs (and `TipTrash_i3`) carry factory-defined trash devices that are fixed to the deck; they are part of the deck geometry, not something a method configures or reassigns.

[†] Table of contents

Reachability and Access

Not every deck position can be used by every pod, and a position being reachable does not mean every well of the labware on it can be pipetted, or that the labware can be moved off it. A method that enqueues cleanly on one instrument can fail on another with the same deck because of the pods installed, where a position sits in the pod's travel, or what else is on the deck at the time.

This page is a troubleshooting aid, not a specification. The one number it does publish is each pod's **X-axis travel envelope**, because that is a hard limit the export records and the single most common reason a position is unreachable — see "Reading a pod's travel envelope" below and the worked i7 figures in Pod Reach Envelopes (i7 worked example). Everything else (per-position framing, the live two-pod interaction, what else is on the deck) still depends on your chassis and calibration; use this page to work out *which* constraint you have hit and what to change, and confirm the specifics in the Deck Editor and Manual Control.

For what an ALP is and which instruments each one suits, see ALP Catalog. For the JSON fields mentioned here, see Deck Layouts Format Specification.

Two different questions

Reach is not one test. Biomek answers two questions separately, and a position can pass one and fail the other:

Question	Applies to	Typical failure
Can the pod get its tips over this position to pipette?	Transfer, Combine, and other pipetting steps	The <pod> cannot reach the deck location "<position>".
Can the pod's gripper get to this position, holding this labware, on this grip side?	Move Labware, and steps that fetch tip boxes	<pod> cannot grip labware at <position>. / <pod> cannot use this grip ...

The gripper question additionally has a coarse form and an exact form. The coarse form asks whether the move is geometrically possible on a **bare deck**; the exact form asks whether it is possible **given what is on the deck right now**. A move that the Deck Editor accepts can still fail at run time once neighboring positions are occupied.

Symptom to likely cause

The message you see often does not name the constraint you actually hit — most notably, an unreachable *column* reports a path-planning failure, not a reach failure.

Message you see	What it usually means	What to try
The <pod> cannot reach the deck location "<position>".	The position as a whole is outside that pod's travel.	Move the labware to a more central position, or use the other pod.
Unable to find a path ...	Often a <i>partial</i> reach problem: the position is reachable but the specific wells, columns, or approach are not — or the route is blocked. Read the rest of the message: if it continues ... destination <axis> <n> cm is outside of travel range, which is between <min> and <max> , it is a hard axis limit, not an obstruction.	For a travel-range clause, move the labware to a position within range — clearing neighbors cannot help. Otherwise try a different column or well selection, clear tall neighbors, or check whether a second pod is limiting travel.
Limit violation: Destination (Z: <n>) is outside the Z limits (<min>, <max>).	A hard pod Z-axis limit, reported in cm of deck height. The move would put the pod above its ceiling or — far more often — below its floor. On the i3 this usually means a low-seated position: short-ALP positions sit much closer to the deck than standard positions, and gripping or loading tips there can drive the pod under its Z floor.	Use a standard-height position instead of a short one, or shorter labware. Clearing neighbors cannot help — this is an axis limit, not an obstruction. See "The pod's Z travel band" below.
Unable to find a path to move the pipettor to a safe Z height:	A genuine obstruction , not an axis limit: a tall item — most often a tip box — sits in the pod's travel corridor. An obstacle named <position>: TipHubs is not a labware class: it is the stub of tips	Relocate the tall obstacle named in the message out of the corridor between the source and destination

Message you see	What it usually means	What to try
... would cause a collision with obstacle <position>: <class>.	standing proud of a tip box's surface, modeled separately from the box body, so the blocker is the <i>present tips</i> at that position rather than the box itself. The blocker sits between two positions and stops the safe-Z move (frequently seen prefixed <code>Error cleaning tips for transfer from <position>:).</code>	positions; this one <i>is</i> fixed by clearing the neighbor, unlike a travel-range clause.
Can't lower Pod<n> to tip load height (...) over <position> because it would cause a collision: ... (also ...to tip preload height... / Lowering the pod to tip load height would cause a collision.)	A collision while descending to pick up tips — the head cannot reach tip-load height over that tip position because a neighbor obstructs it. Distinct from the safe-Z pipetting collision above.	Clear the obstructing neighbor, or move the tip box to a tip-load position with clearance.
Unable to move <pod> gripper GZ axis to <n> to grab plate at <position>: <part> of <pod> gripper interferes with position <neighbor>. (also ... to put labware at ...)	A gripper-body collision with a neighboring deck position , not with the labware you are moving. The named part — <code>lowerhand</code> , <code>upperhand</code> , or a finger portion — sweeps into the adjacent position as the gripper descends. Positions immediately beside a wash or trash station are the common case. Read the blocker's wording closely: <code>position <name></code> is the neighboring position itself, so clearing what sits on it will not help; a blocker phrased <code>obstacle <name> on position <name></code> is an item you can remove.	Move the labware to a position that is not adjacent to the named neighbor, or grip from the other side.
The selected probes cannot reach the given section of the reservoir.	Span-8 only. The chosen probes, at their spacing, do not fit inside that reservoir section.	Select fewer probes, choose different probes, or target a wider section.
Cannot access labware of type <class> with <pod>.	The head's geometry cannot address that labware's wells at all — not a position problem.	Use a pod whose head suits the labware; low-density plates generally need Span-8.
<pod> cannot use this grip ...	No single grip side works at both ends of the move.	Try the other grip side; if neither works, change one end's position.
Cannot pick up labware that would extend ... before hitting the mandrels.	Fixed-8 only. The labware or stack is too tall for the integrated pod gripper at the requested grip height.	Use a higher grip offset, or move fewer plates at a time.
Cannot pick up labware when tips are loaded.	Tips are on the pod.	Unload tips before the move.
<pod> is unable to find a tip discard location. (Span-8) and <pod> cannot find a location to discard tips. (Multichannel)	No deck position carrying the <code>can Discard Tips</code> characteristic is reachable by that pod . The search filters on the characteristic <i>and</i> pod reach, so a trash the other pod reaches can be out of range here — see "Two pods on one bridge" below.	Place a trash inside that pod's envelope, or assign the unload to the pod that can reach one. Neither step names a destination, so this is a deck fix rather than a parameter change.
Unable to find a trash position to discard tips. (Fixed-8)	No deck position carries <code>can Discard Tips</code> at all. This path tests the characteristic alone with no reachability check, and never names a pod.	Add a trash position, or set the step's tip destination to return tips to the box they came from — see Fixed-8 Unload Tips (in <i>Biomek JSON Authoring Guide</i>).
<position> does not allow gripping.	The position is flagged against gripping regardless of geometry.	Use a different position; see the characteristics note below.

What limits reach

Pod travel and the position center

A pod's position-level reach test is evaluated at the **center of the position**, so it is all-or-nothing: it tells you whether the pod can address that position, not whether every well of the labware there is reachable. Trash and tip-disposal positions are the exception — they are

evaluated at their tip-discard point rather than their center, because they are usually placed at the end of pod travel where the center would not be reachable.

The pod's Z travel band

Reach is limited vertically too, and the band is narrower than most authors expect. Each pod publishes it in the instrument-settings export as `podSettings.<pod>.settings.min.z` and `max.z`, in cm of deck height — on an i3, Pod1 runs nominally from **9.472** to **26.286**. A move outside it fails with the `Limit violation: Destination (Z:<n>) is outside the Z limits (...)` message above.

The floor is the one that bites. A position whose ALP seats labware close to the deck leaves the pod nowhere to go: on the i3, short-ALP positions seat labware about 2.8 cm above the deck against roughly 8.6 cm at standard positions, and gripping labware or loading tips there can put the pod below 9.472. This is why the short positions carry a characteristic that disallows manual teaching — the Fixed-8 pod does not have the Z travel to under-grip labware directly off them, so they are framed by auto-teach or with a framing block instead.

Do not try to predict this by adding heights together. The number checked against the band is a pod-axis coordinate, not "seat height plus labware height", and it is computed differently for different operations: a tip load works from the ALP seat, the tip *box* class height, and a press-on offset, while a gripper move works from the ALP seat, the pod's own gripper Z offset, the stacking offset at the position, and the labware's grip offset — which is negative for an under-grip. Arithmetic on the tip class's own height does not enter either one. Treat the published band as a diagnostic for reading the error, and confirm a marginal position by trying it.

Individual columns and wells

Because the position-level test uses the center, a position near the edge of travel can pass it and still have columns the pod cannot address. On a compact chassis, labware at the far-left position may be usable for its rightmost column and not its leftmost. This surfaces as a path-planning failure rather than a reach message, so if a transfer fails only for certain column or well selections, suspect edge-of-travel geometry before suspecting the step. Narrowing the selection to the columns nearer the deck center is the quickest test.

Positions that are simply out of range

Reach is limited front-to-back as well as side-to-side, and a deck can define positions that a given pod cannot reach **at all**. This is the case worth ruling out first, because it looks like every other reach problem and responds to none of the usual remedies.

The tell is in the message. A hard axis limit reports the axis, the requested coordinate and the pod's actual range:

```
Unable to find a path for <pod> pipettor to approach position <position>:  
Specified pipettor destination Y <n> cm is outside of travel range,  
which is between <min> cm and <max> cm.
```

When you see that, stop looking for a collision. Emptying neighboring positions, changing the load order, rearranging tips and re-framing will all fail, because nothing about the deck's *contents* is being complained about. The position is out of the pod's reach and the labware has to move.

For how to read the pod name, the requested coordinate and the two limit values in that message against the pod's published envelope — and a real example of each axis form — see Pod Reach Envelopes (i7 worked example) §"The travel-range error: reading the two numbers".

Back-row positions are the usual offenders — on a multichannel layout the rearmost row of positions (including the rearmost tip-load position) can sit beyond the pod's Y travel once the head's own depth is added to the position's coordinate. They are listed in the deck layout exactly like every other position, with nothing to mark them out, so a method that spreads labware evenly across the published positions can land on one without warning. If you are allocating several tip boxes or plates across a row, prefer positions nearer the front and confirm the rearmost one in Manual Control before committing to it.

The front row positions are common offenders on a Span-8 configuration. Probe 1 cannot reach wells at the front of the front row. Probes 7-8 should be used instead if access is required, or labware placed farther back.

Note also that being out of range only matters when a pod actually has to go there. A position that is merely *named* by a non-pipetting step — a Pause that takes a deck position as a scheduling resource, say — involves no pod motion and is not subject to these limits, so an unreachable position can appear in a working method without ever failing.

Reading a pod's travel envelope

The travel range in that message is not a mystery number — the instrument records it, per pod. In a Biomek Instrument Settings export each pod carries its X (and Y, Z, ...) limits at `podSettings.<pod>.settings.min.x / .max.x`, and the same values appear in human-readable form in `podSettings.<pod>.asText` under *Axis Limit Settings* — which is also what the Pod Settings dialog shows in the software. Read those to know, before you enqueue, how far each pod can go.

The catch is the coordinate frame: those limits are **pipettor-axis** values, and a deck position's `x1` is offset from them by a fixed per-pod amount (head and mandrel geometry) that the export does not publish. So the envelope reliably tells you the *shape* of the constraint — the right-hand pod loses the left of the deck, the left-hand pod loses the right — but the exact `x1` cut-off has to be confirmed in Manual Control unless you can anchor the offset from an observed reach failure. Worked i7 envelopes are in Pod Reach Envelopes (i7 worked example).

Tip loading: capability is not reachability

The `characteristics` keys `multichannel Can Load Tips` and `span Can Load Tips` advertise which pod family a position is *built* to load tips for — a **capability**, not a promise that the pod can currently **reach** it. The two are not symmetric, and confusing them is a common trap:

- **Multichannel tip boxes are confined to the tip-load positions.** Only the `tl*` positions carry `multichannel Can Load Tips`; a plain `p*` position never does. An MC tip box has nowhere else to go.
- **Span-8 tip boxes can relocate anywhere.** Every plain `p*` position carries `span Can Load Tips`, so a Span-8 tip box is not tied to the `tl*` column — this is the escape hatch when the `tl*` positions do not work for the Span-8.
- **The trap:** the `tl*` positions *also* carry `span Can Load Tips`, so it looks like Span-8 tip loading works there. On a two-pod instrument where the Span-8 is the right-hand pod (the i7 Hybrid), the `tl*` column sits at the far left, below the Span-8's X minimum — the Span-8 cannot reach it, capability flag notwithstanding. Put the Span-8 tip box on a `p*` position inside the Span-8's reach instead. The i7 Hybrid figures are in Pod Reach Envelopes (i7 worked example).

Grip side

For the Multichannel and Span-8 pods, labware can be gripped with the A1 corner **near** or **away from** the pod. A grip side is only usable if it works at **both** the source and the destination of the move. This is the common surprise: each end may be fine on its own, but with different sides, leaving no side that serves the whole move.

Grip side is not universal. A Fixed-8 pod does not offer a grip-side choice at all.

Labware and stack height

Gripping is limited from above, not below. On a **Fixed-8** pod (i3), where the gripper is integrated directly beneath the mandrels, a stack that would extend too far up from the bottom of the gripper fingers collides with the mandrels, and the check accounts for the grip offset: an **undergrip** brings the labware closer to the mandrels, while a **high grip** buys clearance. Labware that fails to move as a stack may move when gripped higher.

On **Span-8** and **Multichannel** pods (i5/i7) the gripper is a *separate rotating gripper* that swings the labware clear of the pipettor, so height is not bounded by mandrel clearance. There the move is limited instead by the gripper's safe-Z / travel ceiling and by collision against deck obstacles and the pipettor body — gripping higher can still help, but through that clearance model rather than a fixed distance to the mandrels.

Some labware cannot be undergripped at all — where the external walls taper, there is nothing for the gripper's toe to get under, and the grip offset must be raised to catch the skirt. Class descriptions in the labware catalog call this out where it applies.

Height also constrains from the other direction on a compact chassis. The same low profile that lets a Fixed-8 pod reach *into* labware to pipette can leave it no room to reach *below* that labware for an undergrip. See "The pod's Z travel band" earlier on this page for the limit behind this and the message it raises.

Two pods on one bridge

When two pods share a bridge, neither gets its full travel: each pod's range is narrowed by where the other one is, so a position reachable on a single-pod instrument may be unreachable on a two-pod one with the same deck. Which end is clipped depends on which side the other pod is on. If a position works when a method is run one way and not another, or works after a step that parks the other pod, this is the likely cause — the answer depends on pod state, not on the deck alone.

Refer to Pod Reach Envelopes (i7 worked example).

Head geometry versus labware layout

Distinct from position reach: a head may be unable to address a labware class's wells at all, because the well spacing does not match the head's channel pitch. Low-density plates — 6-, 12-, 24-well culture plates — are the usual case. This fails on the labware class, not the position, so moving the plate elsewhere does not help.

Positions that block access outright

Some positions refuse access for reasons unrelated to geometry: a position may be flagged to disallow gripping, hold permanent labware that method authors do not place or move, or be a trash device. These appear as `characteristics` keys on the position in an instrument-settings export — see the `characteristics` row in Deck Layouts Format Specification. A position advertises which pod families can load tips from it the same way.

Depth is a separate axis

"Cannot reach" is also used for **depth** — a tip that is not long enough to get to the requested height in a deep well without the pod striking the labware, or a technique whose minimum-height setting holds the tip above the target. These report their own messages, naming tip length and depth in cm, and are properties of the tip class, labware class, and technique rather than of the deck position. If a failure names a depth or a tip length, it is not a deck-reachability problem.

Narrowing it down

1. Does it fail for the position, or only for some wells or columns? Position-wide points at pod travel; selection-specific points at edge-of-travel geometry.
2. Does it fail for pipetting, gripping, or both? They are separate tests with separate messages.
3. For a failed move, does either grip side work on its own at each end? If so, the move needs one side that serves both.
4. Does it fail only with both pods in play, or only after a particular step? That points at the two-pod travel limit rather than the deck.
5. Does the message name a labware class rather than a position? That is head-versus-labware geometry, and relocating will not help.

Notes

- Reach depends on framing. A position that has not been framed for a given pod, or has been framed to different offsets, will not behave like the same position on another instrument.
- Reach is evaluated per pod. On a two-pod instrument, "the deck" does not have one reach answer; each pod has its own, and each is affected by the other's position.
- The sample decks in samples/ show where positions sit on each default layout, which is often enough to spot an edge-of-travel position by eye.

Pod Reach Envelopes (i7 worked example)

Which pod can reach which position is decided first by each pod's **X-axis travel envelope** — a hard min/max on where the pipettor can go. This page publishes those envelopes for the default i7 instruments, shows where to read the equivalent numbers for **your** instrument, and turns them into a position-by-position reach table for the i7 Hybrid so that "I need a Span-8 tip box on an i7" has a deck position that actually works.

For the mechanism behind these numbers and every other reach failure mode, see [Reachability and Access](#). For the position coordinates, see the [samples/](#) [deck](#) views.

These numbers are instrument-specific. They are read from the default i7 exports and are correct for those files. Your instrument's envelope depends on its chassis, installed pods, and calibration — read yours as shown under "Reading your own envelope" rather than copying a value here.

Reading your own envelope

Every pod in a Biomek Instrument Settings export carries its X travel range in two places:

- **Human-readable:** `podSettings.<pod>.asText` contains an *Axis Limit Settings* block, e.g. `x: [ 48.59 , 146.34 ]`. This is the same text shown in the Biomek software's Pod Settings dialog, so you can read it without exporting.
- **Exact:** `podSettings.<pod>.settings.min.x` and `.settings.max.x` (cm), alongside `.y` and the other axes.

X travel envelopes, default i7 instruments

Read from each instrument's export (`podSettings.<pod>.settings.{min,max}.x`). Values in cm, in each pod's own pipettor (carriage-axis) frame.

Instrument (export)	Pod	Head	X min	X max
i7 Multichannel (<code>biomek-i7-multichannel.json</code>)	Pod1	MC 96	10.52	134.665
i7 Span-8 (<code>biomek-i7-span8.json</code>)	Pod1	Span-8	10.116	146.336
i7 Hybrid (<code>biomek-i7-hybrid.json</code>)	Pod1	MC 96	10.52	108.269
i7 Hybrid (<code>biomek-i7-hybrid.json</code>)	Pod2	Span-8	48.587	146.336
i7 DualMultichannel (<code>biomek-i7-dualmultichannel.json</code>)	Pod1	MC 96	10.52	95.499
i7 DualMultichannel (<code>biomek-i7-dualmultichannel.json</code>)	Pod2	MC 96	49.462	134.440

Y is far less constraining side-to-side but bounds the front-to-back reach: on the i7 Hybrid Pod1 Y is `[14.60, 59.59]` and Pod2 Y is `[15.11, 66.33]`. The rearmost deck row can exceed a pod's Y once the head's own depth is added — see the [back-row](#) note in [Reachability and Access](#).

What "two pods on one bridge" costs, in numbers

Adding a second pod narrows the first one's travel — the same instrument model loses reach on the shared side:

- **MC pod:** reaches X up to **134.665** cm alone (i7 Multichannel), but only to **95.499** cm as the left pod of the DualMultichannel — ~39 cm of right-side travel gone.
- **Span-8 pod:** reaches X down to **10.116** cm alone (i7 Span-8), but only to **48.587** cm as the right pod of the Hybrid — ~38 cm of left-side travel gone, which is why the left of a Hybrid deck is out of Span-8 reach (below).

These are the static axis limits recorded per instrument; the live two-pod interaction can narrow reach *further* still, depending on where the other pod is parked. Confirm borderline positions in Manual Control.

The travel-range error: reading the two numbers

Assigning a position to a pod that cannot reach it is caught when the method is enqueued, and the message quotes the axis limit back at you:

```
Unable to find a path for pod2 pipettor to approach position P8:  
Specified pipettor destination X 27.270 cm is outside of travel range,  
which is between 48.587 cm and 146.336 cm.
```

Read it in four parts:

- **pod2** — the pod the step was assigned to. Its envelope is the one that applies; the other pod's envelope has no bearing on this message.
- **P8** — the position the step named.
- **destination X 27.270 cm** — where that position puts the pipettor, expressed in the pod's own axis frame. This is *not* the position's `x1`; head and mandrel geometry offset it, by a fixed per-pod amount the export does not publish.
- **between 48.587 cm and 146.336 cm** — the pod's X envelope, exactly the `podSettings.<pod>.settings.min.x / .max.x` pair. Match it against the table above to confirm which pod you are looking at: on the default i7 Hybrid, `[48.587, 146.336]` is Pod2, the Span-8.

Despite the opening clause, this is **not** an obstruction and not a path-planning problem. Nothing about the deck's *contents* is being complained about, so emptying neighboring positions, changing the load order, reordering steps or re-framing will not change the outcome. The requested coordinate is outside a hard axis limit. Either move the labware to a position inside that pod's envelope, or assign the step to the pod whose envelope contains the position.

The same message is emitted per axis, with the axis letter naming which envelope row to check — X against `min.x / max.x`, Y against `min.y / max.y`. A multichannel pod reaching for a back-row position reports the Y form:

```
Unable to find a path for pod1 pipettor to approach position TL5:  
Specified pipettor destination Y 64.649 cm is outside of travel range,  
which is between 14.596 cm and 59.591 cm.
```

Each pod has its own envelope

On a single-pod instrument there is one envelope, and "can this instrument reach this position" has one answer. On a two-pod instrument there are **two envelopes, and they overlap only partially** — so a position one pod reaches comfortably is routinely outside the other pod's range. This is the normal case, not an edge case. On the default i7 Hybrid, Pod1 spans `[10.52, 108.269]` and Pod2 spans `[48.587, 146.336]`: the left of the deck belongs to Pod1 alone, the right to Pod2 alone, and only the middle is shared.

The consequence for authoring is that a deck plan for a two-pod instrument is not one allocation of labware to positions — it is **one allocation per pod**. Labware only Pod2 touches must sit inside Pod2's envelope; labware only Pod1 touches must sit inside Pod1's; labware both pods touch must sit inside the intersection. Positions have to be chosen per pod, and that makes authoring for a two-pod instrument materially harder than for a single-pod one, on the same deck, with the same labware.

The position that "looks central" is the one that most often fails. A position whose pipettor destination lands at 46.800 cm is under 2 cm short of Pod2's 48.587 cm minimum — visually mid-deck, and still unreachable. Nothing in the deck layout marks these positions out; they are listed exactly like every other position, with the same `characteristics` keys.

Before you assign a position to a pod

Do this per pod, for every pod that touches the labware — including the pod that fetches its tip box, which is easy to forget when the tip source is a different position from the pipetting target.

1. **Identify the pod.** Take the pod the step names (`pod1 / pod2`), not "the instrument".
2. **Read that pod's envelope.** `podSettings.<pod>.settings.min.x` and `.max.x` from your instrument's export, or the *Axis Limit Settings* block in `podSettings.<pod>.asText`, or the Pod Settings dialog in the software. Read the second pod's separately — do not reuse the first's.
3. **Read the position's geometry.** `x1` for pod 1, `x2` for pod 2, and `xSpan` from `deckLayouts.<layout>.positions`. Calculate the position's center by adding `xSpan / 2` to the X coordinate.

4. **Check if the position's center is outside the X range of the pod.** Depending on whether the pod is moving to the position for pipetting or for gripping, and whether the position includes offsets, there may be additional offsets that impact this result. However, if the position is well outside the range, do not use it with this pod.
5. **Repeat for Y** where the position is in the frontmost or rearmost row, against `min.y` / `max.y`.

If a position fails the check there are exactly two fixes: move the labware to a position inside that pod's envelope, or assign the step to the pod whose envelope contains the position. Changing what else is on the deck is not a fix.

Position reach table - Pipetting

In the nominal case, the following table shows which deck columns are reachable by each pod on each instrument for pipetting. To determine whether a specific position on a deck is reachable, in the instrument settings export, check the position's `group`'s `column` value.

This table shows positioning for standard static ALPs (Static1x1, Static1x3, Static1x5, TipLoad1x1, Static1x1_i3, Short1x1_i3). Other ALPs define their locations differently and these values may not apply; check the exact framed coordinates instead.

Instrument	Pod Type	Leftmost Column	Rightmost Column
i3	Fixed-8	3 (C)	22 (V)
i5	Multichannel	6 (F)	34 (AH)
i5	Span-8	6 (F)	34 (AH)
i7 Single-Arm	Multichannel	6 (F)	62 (BJ)
i7 Single-Arm	Span-8	6 (F)	62 (BJ)
i7 Hybrid	Left Multichannel	6 (F)	48 (AV)
i7 Hybrid	Right Span-8	27 (AA)	62 (BJ)
i7 Dual MC	Left Multichannel	6 (F)	41 (AO)
i7 Dual MC	Right Multichannel	27 (AA)	62 (BJ)

Position reach table - Gripping

In the nominal case, the following table shows which deck columns are reachable by each pod on the instrument for gripping. To determine the column of a specific position, in the instrument settings export, check the position's `group`'s `column` value.

This table shows positioning for standard static ALPs (Static1x1, Static1x3, Static1x5, TipLoad1x1, Static1x1_i3, Short1x1_i3). Other ALPs define their locations differently and these values may not apply; check the exact framed coordinates instead.

Note that for the Multichannel and Span-8 arm, the leftmost column can only be accessed with the "A1 away from the gripper" grip side, and the rightmost only with "A1 near the gripper".

Note that ALPs in positions left of column F are typically associated with integrations and are only accessible for gripping with "A1 away from the gripper". The exact extents of reachability on the left will depend on the particular instrument's calibrated values.

Instrument	Pod Type	Leftmost Column	Rightmost Column
i3	Fixed-8	3 (C)	22 (V)
i5	Multichannel	-2 (-C)	34 (AH)
i5	Span-8	-2 (-C)	40 (AN)
i7 Single-Arm	Multichannel	-2 (-C)	62 (BJ)
i7 Single-Arm	Span-8	-2 (-C)	68 (BP)
i7 Hybrid	Left Multichannel	-2 (-C)	48 (AV)
i7 Hybrid	Right Span-8	20 (T)	68 (BP)

Instrument	Pod Type	Leftmost Column	Rightmost Column
i7 Dual MC	Left Multichannel	-2 (-C)	41 (AO)
i7 Dual MC	Right Multichannel	20 (T)	62 (BJ)

Deck Layout: Biomek i3

- Source: `deckLayouts.i3.positions` in the Biomek Instrument Settings export.
- Illustrative view: `$id / $ref` references are resolved inline, and hardware-internal keys (ALP `group` blocks, footprint spans, framing internals, Pod-2 coordinates, geometry-only fields, runtime state) are stripped. Only the per-position keys you need to read a deck remain.
- Pod2 note: Pod2 coordinates (`x2 / y2 / z2 / framed2` in the raw export) are omitted from the illustrative view. The i3 has a single Fixed-8 pod, so Pod2 is a legacy placeholder — not a second physical pod.
- Full key reference: see Deck Layouts Format Specification.
- ALP types: see ALP Catalog.
- **No trash position.** This deck (`P1 -- P12 , S1 -- S3`) carries no tip-discard position, so `"<Any Trash>"` cannot resolve on it — return tips with `"<where they came from>"` instead. See Instrument Setup (in *Biomek JSON Authoring Guide*) for the full rules.

Illustrative, not a literal export. This is a cleaned, readability-oriented view of the deck: `$id / $ref` references are resolved inline and non-position detail (ALP hardware groups, framing/geometry internals, Pod-2 coordinates, ...) is removed. In particular, every position in your real export carries a `group` object (a *position group*) — the shared definition of the ALP that position belongs to, linked across its positions via `$id / $ref`; it is per-ALP hardware data, not position placement, so it is dropped here. Your real export looks messier — it uses `$id / $ref` object sharing and carries additional keys — so treat this as a guide to the keys, not a byte-for-byte copy or deterministic parser output. See the deck-layout format spec for the full key reference.

Visual Layout

The grid below is a **derived aid** — it is not part of the export. It mirrors the Biomek **Deck Editor**'s layout of this deck. The deck is a grid of mounting holes addressed by **column letter** and **row number**, with the origin at the back-left corner: column `A` is the left edge and row `1` is the back of the deck, so the top row here is the back and the bottom row is the front (operator side). The Deck Editor does not label every hole — it labels one column per **deck plate** (hence `A`, then `F`, `M`, `T`, ... on an i5/i7, or `A`, `H`, `O`, `V` on an i3) and one row every 5 holes — and the grid below uses exactly those labeled columns and rows. Each ALP occupies one cell; an ALP whose body genuinely covers more than one deck plate (a trash slide, which overhangs the side of the deck) is drawn as one wide cell spanning them. The real export carries the cm coordinates (`x1 / y1`) plus each ALP group's `column / row`.

Row	A	H	O	V
5	[empty]	[P1]	[P6]	[P11]
10	[empty]	[P2]	[P7]	[S1]
15	[empty]	[P3]	[P8]	[S2]
20	[empty]	[P4]	[P9]	[S3]
25	[empty]	[P5]	[P10]	[P12]

Deck (illustrative, resolved)

The block below mirrors the shape of `deckLayouts.i3` in the export, with `$ref` pointers resolved and non-load-bearing keys removed. Values shown (coordinates in cm, `alpType`, labware `class / properties.liquidType / volumeType`, device `name`, characteristic and required keys) are copied verbatim from the export; the surrounding `labware` object is reduced to those fields, and `device` is collapsed from the export's device object to just its `name` string.

```
{
  "name": "i3",
  "isDefault": true,
  "isReadOnly": false,
  "positions": {
    "p1": {
      "alpType": "Static1x1_i3",
      "name": "P1",
      "x1": 17.69580001907347,
      "y1": 5.5928997055053715,
      "z1": 8.6424,
      "framed1": "Not",
      "permanentLabware": false,
```

```

    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "fixed-8 Can Load Tips": null
    }
  },
  "p10": {
    "alpType": "Static1x1_i3",
    "name": "P10",
    "x1": 32.27540000190735,
    "y1": 47.24889970550537,
    "z1": 8.6424,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "fixed-8 Can Load Tips": null
    }
  },
  "p11": {
    "alpType": "Static1x1_i3",
    "name": "P11",
    "x1": 46.85500000190736,
    "y1": 5.5928997055053715,
    "z1": 8.6424,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "fixed-8 Can Load Tips": null
    }
  },
  "p12": {
    "alpType": "Static1x1_i3",
    "name": "P12",
    "x1": 46.85500000190736,
    "y1": 47.24889970550537,
    "z1": 8.6424,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "fixed-8 Can Load Tips": null
    }
  },
  "p2": {
    "alpType": "Static1x1_i3",
    "name": "P2",
    "x1": 17.695800001907347,
    "y1": 16.006899705505372,
    "z1": 8.6424,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "fixed-8 Can Load Tips": null
    }
  },
  "p3": {
    "alpType": "Static1x1_i3",
    "name": "P3",
    "x1": 17.695800001907347,
    "y1": 26.420899705505374,
    "z1": 8.6424,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "fixed-8 Can Load Tips": null
    }
  },
  "p4": {
    "alpType": "Static1x1_i3",
    "name": "P4",
    "x1": 17.695800001907347,

```

```
"y1": 36.83489970550538,
"z1": 8.6424,
"framed1": "Not",
"permanentLabware": false,
"required": {
  "standard TiterPlate Size": null
},
"characteristics": {
  "fixed-8 Can Load Tips": null
}
},
"p5": {
  "alpType": "Static1x1_i3",
  "name": "P5",
  "x1": 17.695800001907347,
  "y1": 47.24889970550537,
  "z1": 8.6424,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "fixed-8 Can Load Tips": null
  }
},
"p6": {
  "alpType": "Static1x1_i3",
  "name": "P6",
  "x1": 32.27540000190735,
  "y1": 5.5928997055053715,
  "z1": 8.6424,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "fixed-8 Can Load Tips": null
  }
},
"p7": {
  "alpType": "Static1x1_i3",
  "name": "P7",
  "x1": 32.27540000190735,
  "y1": 16.006899705505372,
  "z1": 8.6424,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "fixed-8 Can Load Tips": null
  }
},
"p8": {
  "alpType": "Static1x1_i3",
  "name": "P8",
  "x1": 32.27540000190735,
  "y1": 26.420899705505374,
  "z1": 8.6424,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "fixed-8 Can Load Tips": null
  }
},
"p9": {
  "alpType": "Static1x1_i3",
  "name": "P9",
  "x1": 32.27540000190735,
  "y1": 36.83489970550538,
  "z1": 8.6424,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "fixed-8 Can Load Tips": null
  }
},
},
```

```

"s1": {
  "alpType": "Short1x1_i3",
  "name": "S1",
  "x1": 46.85500000190736,
  "y1": 16.006899705505372,
  "z1": 2.8131,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "disallow ManualTeach": null,
    "fixed-8 Can Load Tips": null
  }
},
"s2": {
  "alpType": "Short1x1_i3",
  "name": "S2",
  "x1": 46.85500000190736,
  "y1": 26.420899705505374,
  "z1": 2.8131,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "disallow ManualTeach": null,
    "fixed-8 Can Load Tips": null
  }
},
"s3": {
  "alpType": "Short1x1_i3",
  "name": "S3",
  "x1": 46.85500000190736,
  "y1": 36.83489970550538,
  "z1": 2.8131,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "disallow ManualTeach": null,
    "fixed-8 Can Load Tips": null
  }
}
}
}

```

Deck Layout: Biomek i5 Multichannel

- Source: `deckLayouts.multichannel.positions` in the Biomek Instrument Settings export.
- Illustrative view: `$id / $ref` references are resolved inline, and hardware-internal keys (ALP `group` blocks, footprint spans, framing internals, Pod-2 coordinates, geometry-only fields, runtime state) are stripped. Only the per-position keys you need to read a deck remain.
- Pod2 note: Pod2 coordinates (`x2 / y2 / z2 / framed2` in the raw export) are omitted from the illustrative view. On stock decks they equal Pod1 because the deck is unframed (uncalibrated); on a framed deck they carry the Pod2 calibration.
- Full key reference: see Deck Layouts Format Specification.
- ALP types: see ALP Catalog.
- Duplicate-deck note: this configuration also exposes a `Standard` deck whose positions, coordinates, groups, and characteristics are byte-identical to this `Multichannel` deck (only `deckName` differs). No separate view is kept for it — this file is authoritative for both.

Illustrative, not a literal export. This is a cleaned, readability-oriented view of the deck: `$id / $ref` references are resolved inline and non-position detail (ALP hardware groups, framing/geometry internals, Pod-2 coordinates, ...) is removed. In particular, every position in your real export carries a `group` object (a *position group*) — the shared definition of the ALP that position belongs to, linked across its positions via `$id / $ref`; it is per-ALP hardware data, not position placement, so it is dropped here. Your real export looks messier — it uses `$id / $ref` object sharing and carries additional keys — so treat this as a guide to the keys, not a byte-for-byte copy or deterministic parser output. See the deck-layout format spec for the full key reference.

Visual Layout

The grid below is a **derived aid** — it is not part of the export. It mirrors the Biomek **Deck Editor**'s layout of this deck. The deck is a grid of mounting holes addressed by **column letter** and **row number**, with the origin at the back-left corner: column `A` is the left edge and row `1` is the back of the deck, so the top row here is the back and the bottom row is the front (operator side). The Deck Editor does not label every hole — it labels one column per **deck plate** (hence `A`, then `F`, `M`, `T`, ... on an i5/i7, or `A`, `H`, `O`, `V` on an i3) and one row every 5 holes — and the grid below uses exactly those labeled columns and rows. Each ALP occupies one cell; an ALP whose body genuinely covers more than one deck plate (a trash slide, which overhangs the side of the deck) is drawn as one wide cell spanning them. The real export carries the cm coordinates (`x1 / y1`) plus each ALP group's `column / row` .

Row	A	F	M	T	AA	AH	AO
5	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]
10	[empty]	[WS1]	[TL1]	[P1]	[P6]	[P11]	[empty]
15	[TR1	]		[TL2]	[P2]	[P7]	[P12]
20	[empty]	[empty]	[TL3]	[P3]	[P8]	[P13]	[empty]
25	[empty]	[empty]	[TL4]	[P4]	[P9]	[P14]	[empty]
30	[empty]	[empty]	[TL5]	[P5]	[P10]	[P15]	[empty]

Deck (illustrative, resolved)

The block below mirrors the shape of `deckLayouts.multichannel` in the export, with `$ref` pointers resolved and non-load-bearing keys removed. Values shown (coordinates in cm, `alpType`, `labware class / properties.liquidtype / volumeType`, `device name`, `characteristic` and required keys) are copied verbatim from the export; the surrounding `labware` object is reduced to those fields, and `device` is collapsed from the export's device object to just its `name` string.

```
{
  "name": "Multichannel",
  "isDefault": true,
  "isReadOnly": false,
  "positions": {
    "p1": {
      "alpType": "Static1x1",
      "name": "P1",
      "x1": 40.39020000190736,
      "y1": 15.547999705505372,
      "z1": 15.875,
      "framed1": "Not",
```

```

    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p10": {
    "alpType": "Static1x3",
    "name": "P10",
    "x1": 54.96980000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p11": {
    "alpType": "Static1x1",
    "name": "P11",
    "x1": 69.54940000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p12": {
    "alpType": "Static1x1",
    "name": "P12",
    "x1": 69.54940000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p13": {
    "alpType": "Static1x1",
    "name": "P13",
    "x1": 69.54940000190736,
    "y1": 36.37599970550538,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p14": {
    "alpType": "Static1x1",
    "name": "P14",
    "x1": 69.54940000190736,
    "y1": 46.78999970550537,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p15": {
    "alpType": "Static1x1",
    "name": "P15",

```

```

"x1": 69.54940000190736,
"y1": 57.203999705505375,
"z1": 15.875,
"framed1": "Not",
"permanentLabware": false,
"required": {
  "standard TiterPlate Size": null
},
"characteristics": {
  "span Can Load Tips": null
}
},
"p2": {
  "alpType": "Static1x1",
  "name": "P2",
  "x1": 40.39020000190736,
  "y1": 25.961999705505374,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p3": {
  "alpType": "Static1x3",
  "name": "P3",
  "x1": 40.39020000190736,
  "y1": 36.375999636840824,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p4": {
  "alpType": "Static1x3",
  "name": "P4",
  "x1": 40.39020000190736,
  "y1": 46.78999963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p5": {
  "alpType": "Static1x3",
  "name": "P5",
  "x1": 40.39020000190736,
  "y1": 57.20399963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p6": {
  "alpType": "Static1x1",
  "name": "P6",
  "x1": 54.96980000190736,
  "y1": 15.547999705505372,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
}

```

```

},
"p7": {
  "alpType": "Static1x1",
  "name": "P7",
  "x1": 54.96980000190736,
  "y1": 25.961999705505374,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p8": {
  "alpType": "Static1x3",
  "name": "P8",
  "x1": 54.96980000190736,
  "y1": 36.375999636840824,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p9": {
  "alpType": "Static1x3",
  "name": "P9",
  "x1": 54.96980000190736,
  "y1": 46.78999963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"tL1": {
  "alpType": "TipLoad1x1",
  "name": "TL1",
  "x1": 25.81060000190735,
  "y1": 15.547999705505372,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tL2": {
  "alpType": "TipLoad1x1",
  "name": "TL2",
  "x1": 25.81060000190735,
  "y1": 25.961999705505374,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tL3": {
  "alpType": "TipLoad1x1",
  "name": "TL3",
  "x1": 25.81060000190735,
  "y1": 36.37599970550538,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,

```

```

    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL4": {
    "alpType": "TipLoad1x1",
    "name": "TL4",
    "x1": 25.81060000190735,
    "y1": 46.78999970550537,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL5": {
    "alpType": "TipLoad1x1",
    "name": "TL5",
    "x1": 25.81060000190735,
    "y1": 57.203999705505375,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tR1": {
    "alpType": "TrashLeftSlide",
    "name": "TR1",
    "x1": 10.922000259399418,
    "y1": 30.147900088500975,
    "z1": 20.612,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "can Discard Tips": null,
      "disableFrameWithGripper": null,
      "disallow AutoTeach": null,
      "disallow ManualTeach": null,
      "disallow NormalTeach": null,
      "trash": null
    },
    "device": "Trash"
  },
  "wS1": {
    "alpType": "WashStation96",
    "name": "WS1",
    "x1": 10.33789996814728,
    "y1": 13.428300212097168,
    "z1": 8.5623,
    "framed1": "Not",
    "permanentLabware": true,
    "labware": {
      "class": "WashStation",
      "properties": {
        "liquidtype": "Water"
      }
    },
    "volumeType": "Known"
  },
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "disallow ManualTeach": null,
    "no Gripping Allowed": null,
    "wash Station": null
  }
}

```

}
}

† Table of contents

Deck Layout: Biomek i5 Span8

- Source: `deckLayouts.span8.positions` in the Biomek Instrument Settings export.
- Illustrative view: `$id / $ref` references are resolved inline, and hardware-internal keys (ALP `group` blocks, footprint spans, framing internals, Pod-2 coordinates, geometry-only fields, runtime state) are stripped. Only the per-position keys you need to read a deck remain.
- Pod2 note: Pod2 coordinates (`x2 / y2 / z2 / framed2` in the raw export) are omitted from the illustrative view. On stock decks they equal Pod1 because the deck is unframed (uncalibrated); on a framed deck they carry the Pod2 calibration.
- Full key reference: see Deck Layouts Format Specification.
- ALP types: see ALP Catalog.
- Duplicate-deck note: this configuration also exposes a `Standard` deck whose positions, coordinates, groups, and characteristics are byte-identical to this `span8` deck (only `deckName` differs). No separate view is kept for it — this file is authoritative for both.

Illustrative, not a literal export. This is a cleaned, readability-oriented view of the deck: `$id / $ref` references are resolved inline and non-position detail (ALP hardware groups, framing/geometry internals, Pod-2 coordinates, ...) is removed. In particular, every position in your real export carries a `group` object (a *position group*) — the shared definition of the ALP that position belongs to, linked across its positions via `$id / $ref`; it is per-ALP hardware data, not position placement, so it is dropped here. Your real export looks messier — it uses `$id / $ref` object sharing and carries additional keys — so treat this as a guide to the keys, not a byte-for-byte copy or deterministic parser output. See the deck-layout format spec for the full key reference.

Visual Layout

The grid below is a **derived aid** — it is not part of the export. It mirrors the Biomek **Deck Editor**'s layout of this deck. The deck is a grid of mounting holes addressed by **column letter** and **row number**, with the origin at the back-left corner: column `A` is the left edge and row `1` is the back of the deck, so the top row here is the back and the bottom row is the front (operator side). The Deck Editor does not label every hole — it labels one column per **deck plate** (hence `A`, then `F`, `M`, `T`, ... on an i5/i7, or `A`, `H`, `O`, `V` on an i3) and one row every 5 holes — and the grid below uses exactly those labeled columns and rows. Each ALP occupies one cell; an ALP whose body genuinely covers more than one deck plate (a trash slide, which overhangs the side of the deck) is drawn as one wide cell spanning them. The real export carries the cm coordinates (`x1 / y1`) plus each ALP group's `column / row` .

Row	A	F	M	T	AA	AH	AO
5	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]
10	[empty]	[P1]	[P6]	[P11]	[P16]	[empty]	[W1]
15	[empty]	[P2]	[P7]	[P12]	[P17]	[TR1]	]
20	[empty]	[P3]	[P8]	[P13]	[P18]	[empty]	
25	[empty]	[P4]	[P9]	[P14]	[P19]	[empty]	[empty]
30	[empty]	[P5]	[P10]	[P15]	[P20]	[empty]	[empty]

Deck (illustrative, resolved)

The block below mirrors the shape of `deckLayouts.span8` in the export, with `$ref` pointers resolved and non-load-bearing keys removed. Values shown (coordinates in cm, `alpType`, labware `class / properties.liquidtype / volumeType`, device `name`, characteristic and required keys) are copied verbatim from the export; the surrounding `labware` object is reduced to those fields, and `device` is collapsed from the export's device object to just its `name` string.

```
{
  "name": "Span8",
  "isDefault": true,
  "isReadOnly": false,
  "positions": {
    "p1": {
      "alpType": "Static1x1",
      "name": "P1",
      "x1": 11.23100000190735,
      "y1": 15.547999705505372,
      "z1": 15.875,
      "framed1": "Not",
```

```

    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p10": {
    "alpType": "Static1x3",
    "name": "P10",
    "x1": 25.81060000190735,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p11": {
    "alpType": "Static1x1",
    "name": "P11",
    "x1": 40.39020000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p12": {
    "alpType": "Static1x1",
    "name": "P12",
    "x1": 40.39020000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p13": {
    "alpType": "Static1x3",
    "name": "P13",
    "x1": 40.39020000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p14": {
    "alpType": "Static1x3",
    "name": "P14",
    "x1": 40.39020000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p15": {
    "alpType": "Static1x3",
    "name": "P15",

```

```
"x1": 40.39020000190736,
"y1": 57.20399963684082,
"z1": 15.875,
"framed1": "Not",
"permanentLabware": false,
"required": {
  "standard TiterPlate Size": null
},
"characteristics": {
  "span Can Load Tips": null
}
},
"p16": {
  "alpType": "Static1x1",
  "name": "P16",
  "x1": 54.96980000190736,
  "y1": 15.547999705505372,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p17": {
  "alpType": "Static1x1",
  "name": "P17",
  "x1": 54.96980000190736,
  "y1": 25.961999705505374,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p18": {
  "alpType": "Static1x3",
  "name": "P18",
  "x1": 54.96980000190736,
  "y1": 36.375999636840824,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p19": {
  "alpType": "Static1x3",
  "name": "P19",
  "x1": 54.96980000190736,
  "y1": 46.78999963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p2": {
  "alpType": "Static1x1",
  "name": "P2",
  "x1": 11.23100000190735,
  "y1": 25.961999705505374,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
}
```

```

},
"p20": {
  "alpType": "Static1x3",
  "name": "P20",
  "x1": 54.96980000190736,
  "y1": 57.20399963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p3": {
  "alpType": "Static1x3",
  "name": "P3",
  "x1": 11.23100000190735,
  "y1": 36.375999636840824,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p4": {
  "alpType": "Static1x3",
  "name": "P4",
  "x1": 11.23100000190735,
  "y1": 46.78999963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p5": {
  "alpType": "Static1x3",
  "name": "P5",
  "x1": 11.23100000190735,
  "y1": 57.20399963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p6": {
  "alpType": "Static1x1",
  "name": "P6",
  "x1": 25.81060000190735,
  "y1": 15.547999705505372,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p7": {
  "alpType": "Static1x1",
  "name": "P7",
  "x1": 25.81060000190735,
  "y1": 25.961999705505374,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  }
}

```

```

    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p8": {
    "alpType": "Static1x3",
    "name": "P8",
    "x1": 25.81060000190735,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p9": {
    "alpType": "Static1x3",
    "name": "P9",
    "x1": 25.81060000190735,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "tr1": {
    "alpType": "TrashRightSlide",
    "name": "TR1",
    "x1": 69.8573999909401,
    "y1": 30.147900088500975,
    "z1": 20.612,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "can Discard Tips": null,
      "disableFrameWithGripper": null,
      "disallow AutoTeach": null,
      "disallow ManualTeach": null,
      "disallow NormalTeach": null,
      "trash": null
    },
    "device": "Trash"
  },
  "w1": {
    "alpType": "WashStationSpan8",
    "name": "W1",
    "x1": 83.92169992294312,
    "y1": 15.342999946594238,
    "z1": 22.1361,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "no plates allowed": null
    },
    "characteristics": {
      "disableFrameWithGripper": null,
      "disallow AutoTeach": null,
      "disallow NormalTeach": null,
      "no Gripping Allowed": null,
      "vWash Station": null
    }
  }
}

```

Deck Layout: Biomek i7 DualMultichannel

- Source: `deckLayouts.dualMultichannel.positions` in the Biomek Instrument Settings export.
- Illustrative view: `$id / $ref` references are resolved inline, and hardware-internal keys (ALP `group` blocks, footprint spans, framing internals, Pod-2 coordinates, geometry-only fields, runtime state) are stripped. Only the per-position keys you need to read a deck remain.
- Pod2 note: Pod2 coordinates (`x2 / y2 / z2 / framed2` in the raw export) are omitted from the illustrative view. On stock decks they equal Pod1 because the deck is unframed (uncalibrated); on a framed deck they carry the Pod2 calibration.
- Full key reference: see Deck Layouts Format Specification.
- ALP types: see ALP Catalog.

Illustrative, not a literal export. This is a cleaned, readability-oriented view of the deck: `$id / $ref` references are resolved inline and non-position detail (ALP hardware groups, framing/geometry internals, Pod-2 coordinates, ...) is removed. In particular, every position in your real export carries a `group` object (a *position group*) — the shared definition of the ALP that position belongs to, linked across its positions via `$id / $ref`; it is per-ALP hardware data, not position placement, so it is dropped here. Your real export looks messier — it uses `$id / $ref` object sharing and carries additional keys — so treat this as a guide to the keys, not a byte-for-byte copy or deterministic parser output. See the deck-layout format spec for the full key reference.

Visual Layout

The grid below is a **derived aid** — it is not part of the export. It mirrors the Biomek **Deck Editor**'s layout of this deck. The deck is a grid of mounting holes addressed by **column letter** and **row number**, with the origin at the back-left corner: column `A` is the left edge and row `1` is the back of the deck, so the top row here is the back and the bottom row is the front (operator side). The Deck Editor does not label every hole — it labels one column per **deck plate** (hence `A`, then `F`, `M`, `T`, ... on an i5/i7, or `A`, `H`, `O`, `V` on an i3) and one row every 5 holes — and the grid below uses exactly those labeled columns and rows. Each ALP occupies one cell; an ALP whose body genuinely covers more than one deck plate (a trash slide, which overhangs the side of the deck) is drawn as one wide cell spanning them. The real export carries the cm coordinates (`x1 / y1`) plus each ALP group's `column / row`.

Row	A	F	M	T	AA	AH	AO	AV	BC	BJ	BQ
5	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]
10	[empty]	[WS1]	[TL1]	[P1]	[P6]	[P11]	[P16]	[P21]	[TL6]	[WS2]	[empty]
15	[TR1]	]		[TL2]	[P2]	[P7]	[P12]	[P17]	[P22]	[TL7]	[TR2]
20	[empty]	[empty]	[TL3]	[P3]	[P8]	[P13]	[P18]	[P23]	[TL8]	[empty]	[empty]
25	[empty]	[empty]	[TL4]	[P4]	[P9]	[P14]	[P19]	[P24]	[TL9]	[empty]	[empty]
30	[empty]	[empty]	[TL5]	[P5]	[P10]	[P15]	[P20]	[P25]	[TL10]	[empty]	[empty]

Deck (illustrative, resolved)

The block below mirrors the shape of `deckLayouts.dualMultichannel` in the export, with `$ref` pointers resolved and non-load-bearing keys removed. Values shown (coordinates in cm, `alpType`, `labware class / properties.liquidtype / volumeType`, `device name`, characteristic and required keys) are copied verbatim from the export; the surrounding `labware` object is reduced to those fields, and `device` is collapsed from the export's device object to just its `name` string.

```
{
  "name": "DualMultichannel",
  "isDefault": false,
  "isReadOnly": false,
  "positions": {
    "p1": {
      "alpType": "Static1x1",
      "name": "P1",
      "x1": 40.39020000190736,
      "y1": 15.547999705505372,
      "z1": 15.875,
      "framed1": "Not",
      "permanentLabware": false,
      "required": {
        "standard TiterPlate Size": null
      }
    }
  }
}
```

```

    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p10": {
    "alpType": "Static1x3",
    "name": "P10",
    "x1": 54.96980000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p11": {
    "alpType": "Static1x1",
    "name": "P11",
    "x1": 69.54940000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p12": {
    "alpType": "Static1x1",
    "name": "P12",
    "x1": 69.54940000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p13": {
    "alpType": "Static1x3",
    "name": "P13",
    "x1": 69.54940000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p14": {
    "alpType": "Static1x3",
    "name": "P14",
    "x1": 69.54940000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p15": {
    "alpType": "Static1x3",
    "name": "P15",
    "x1": 69.54940000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,

```

```

    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p16": {
    "alpType": "Static1x1",
    "name": "P16",
    "x1": 84.12900000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p17": {
    "alpType": "Static1x1",
    "name": "P17",
    "x1": 84.12900000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p18": {
    "alpType": "Static1x3",
    "name": "P18",
    "x1": 84.12900000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p19": {
    "alpType": "Static1x3",
    "name": "P19",
    "x1": 84.12900000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p2": {
    "alpType": "Static1x1",
    "name": "P2",
    "x1": 40.39020000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p20": {
    "alpType": "Static1x3",

```

```

    "name": "P20",
    "x1": 84.12900000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p21": {
    "alpType": "Static1x1",
    "name": "P21",
    "x1": 98.70860000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p22": {
    "alpType": "Static1x1",
    "name": "P22",
    "x1": 98.70860000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p23": {
    "alpType": "Static1x3",
    "name": "P23",
    "x1": 98.70860000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p24": {
    "alpType": "Static1x3",
    "name": "P24",
    "x1": 98.70860000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p25": {
    "alpType": "Static1x3",
    "name": "P25",
    "x1": 98.70860000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  }
}

```

```

    }
  },
  "p3": {
    "alpType": "Static1x3",
    "name": "P3",
    "x1": 40.39020000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p4": {
    "alpType": "Static1x3",
    "name": "P4",
    "x1": 40.39020000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p5": {
    "alpType": "Static1x3",
    "name": "P5",
    "x1": 40.39020000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p6": {
    "alpType": "Static1x1",
    "name": "P6",
    "x1": 54.96980000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p7": {
    "alpType": "Static1x1",
    "name": "P7",
    "x1": 54.96980000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p8": {
    "alpType": "Static1x3",
    "name": "P8",
    "x1": 54.96980000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {

```

```

    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p9": {
  "alpType": "Static1x3",
  "name": "P9",
  "x1": 54.96980000190736,
  "y1": 46.78999963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"tl1": {
  "alpType": "TipLoad1x1",
  "name": "TL1",
  "x1": 25.81060000190735,
  "y1": 15.547999705505372,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tl10": {
  "alpType": "TipLoad1x1",
  "name": "TL10",
  "x1": 113.28820000190736,
  "y1": 57.203999705505375,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tl2": {
  "alpType": "TipLoad1x1",
  "name": "TL2",
  "x1": 25.81060000190735,
  "y1": 25.961999705505374,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tl3": {
  "alpType": "TipLoad1x1",
  "name": "TL3",
  "x1": 25.81060000190735,
  "y1": 36.37599970550538,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tl4": {

```

```

"alpType": "TipLoad1x1",
"name": "TL4",
"x1": 25.81060000190735,
"y1": 46.78999970550537,
"z1": 16.129,
"framed1": "Not",
"permanentLabware": false,
"required": {
  "standard TiterPlate Size": null
},
"characteristics": {
  "multichannel Can Load Tips": null,
  "span Can Load Tips": null
}
},
"tl5": {
  "alpType": "TipLoad1x1",
  "name": "TL5",
  "x1": 25.81060000190735,
  "y1": 57.203999705505375,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tl6": {
  "alpType": "TipLoad1x1",
  "name": "TL6",
  "x1": 113.28820000190736,
  "y1": 15.547999705505372,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tl7": {
  "alpType": "TipLoad1x1",
  "name": "TL7",
  "x1": 113.28820000190736,
  "y1": 25.961999705505374,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tl8": {
  "alpType": "TipLoad1x1",
  "name": "TL8",
  "x1": 113.28820000190736,
  "y1": 36.37599970550538,
  "z1": 16.129,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tl9": {
  "alpType": "TipLoad1x1",
  "name": "TL9",
  "x1": 113.28820000190736,
  "y1": 46.78999970550537,
  "z1": 16.129,
  "framed1": "Not",

```

```

    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tr1": {
    "alpType": "TrashLeftSlide",
    "name": "TR1",
    "x1": 10.922000259399418,
    "y1": 30.147900088500975,
    "z1": 20.612,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "can Discard Tips": null,
      "disableFrameWithGripper": null,
      "disallow AutoTeach": null,
      "disallow ManualTeach": null,
      "disallow NormalTeach": null,
      "trash": null
    },
    "device": "Trash"
  },
  "tr2": {
    "alpType": "TrashRightSlide",
    "name": "TR2",
    "x1": 128.1757999909401,
    "y1": 30.147900088500975,
    "z1": 20.612,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "can Discard Tips": null,
      "disableFrameWithGripper": null,
      "disallow AutoTeach": null,
      "disallow ManualTeach": null,
      "disallow NormalTeach": null,
      "trash": null
    },
    "device": "Trash"
  },
  "ws1": {
    "alpType": "WashStation96",
    "name": "WS1",
    "x1": 10.33789996814728,
    "y1": 13.428300212097168,
    "z1": 8.5623,
    "framed1": "Not",
    "permanentLabware": true,
    "labware": {
      "class": "WashStation",
      "properties": {
        "liquidtype": "Water"
      }
    },
    "volumeType": "Known"
  },
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "disallow ManualTeach": null,
    "no Gripping Allowed": null,
    "wash Station": null
  }
},
"ws2": {
  "alpType": "WashStation96",
  "name": "WS2",
  "x1": 126.97469996814728,
  "y1": 13.428300212097168,
  "z1": 8.5623,
  "framed1": "Not",
  "permanentLabware": true,
  "labware": {
    "class": "WashStation",
    "properties": {

```

```
    "liquidtype": "Water"
  },
  "volumeType": "Known"
},
"required": {
  "standard TiterPlate Size": null
},
"characteristics": {
  "disallow ManualTeach": null,
  "no Gripping Allowed": null,
  "wash Station": null
}
}
}
```

† Table of contents

Deck Layout: Biomek i7 Hybrid

- Source: `deckLayouts.hybrid.positions` in the Biomek Instrument Settings export.
- Illustrative view: `$id / $ref` references are resolved inline, and hardware-internal keys (ALP `group` blocks, footprint spans, framing internals, Pod-2 coordinates, geometry-only fields, runtime state) are stripped. Only the per-position keys you need to read a deck remain.
- Pod2 note: Pod2 coordinates (`x2 / y2 / z2 / framed2` in the raw export) are omitted from the illustrative view. On stock decks they equal Pod1 because the deck is unframed (uncalibrated); on a framed deck they carry the Pod2 calibration.
- Full key reference: see Deck Layouts Format Specification.
- ALP types: see ALP Catalog.

Illustrative, not a literal export. This is a cleaned, readability-oriented view of the deck: `$id / $ref` references are resolved inline and non-position detail (ALP hardware groups, framing/geometry internals, Pod-2 coordinates, ...) is removed. In particular, every position in your real export carries a `group` object (a *position group*) — the shared definition of the ALP that position belongs to, linked across its positions via `$id / $ref`; it is per-ALP hardware data, not position placement, so it is dropped here. Your real export looks messier — it uses `$id / $ref` object sharing and carries additional keys — so treat this as a guide to the keys, not a byte-for-byte copy or deterministic parser output. See the deck-layout format spec for the full key reference.

Visual Layout

The grid below is a **derived aid** — it is not part of the export. It mirrors the Biomek **Deck Editor**'s layout of this deck. The deck is a grid of mounting holes addressed by **column letter** and **row number**, with the origin at the back-left corner: column `A` is the left edge and row `1` is the back of the deck, so the top row here is the back and the bottom row is the front (operator side). The Deck Editor does not label every hole — it labels one column per **deck plate** (hence `A`, then `F`, `M`, `T`, ... on an i5/i7, or `A`, `H`, `O`, `V` on an i3) and one row every 5 holes — and the grid below uses exactly those labeled columns and rows. Each ALP occupies one cell; an ALP whose body genuinely covers more than one deck plate (a trash slide, which overhangs the side of the deck) is drawn as one wide cell spanning them. The real export carries the cm coordinates (`x1 / y1`) plus each ALP group's `column / row`.

Row	A	F	M	T	AA	AH	AO	AV	BC	BJ	BQ
5	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]
10	[empty]	[WS1]	[TL1]	[P1]	[P6]	[P11]	[P16]	[P21]	[P26]	[empty]	[W1]
15	[TR1]	]	[TL2]	[P2]	[P7]	[P12]	[P17]	[P22]	[P27]	[TR2]	]
20	[empty]		[empty]	[TL3]	[P3]	[P8]	[P13]	[P18]	[P23]	[P28]	
25	[empty]	[empty]	[TL4]	[P4]	[P9]	[P14]	[P19]	[P24]	[P29]	[empty]	[empty]
30	[empty]	[empty]	[TL5]	[P5]	[P10]	[P15]	[P20]	[P25]	[P30]	[empty]	[empty]

Deck (illustrative, resolved)

The block below mirrors the shape of `deckLayouts.hybrid` in the export, with `$ref` pointers resolved and non-load-bearing keys removed. Values shown (coordinates in cm, `alpType`, labware `class / properties.liquidtype / volumeType`, device `name`, characteristic and required keys) are copied verbatim from the export; the surrounding `labware` object is reduced to those fields, and `device` is collapsed from the export's device object to just its `name` string.

```
{
  "name": "Hybrid",
  "isDefault": false,
  "isReadOnly": false,
  "positions": {
    "p1": {
      "alpType": "Static1x1",
      "name": "P1",
      "x1": 40.39020000190736,
      "y1": 15.547999705505372,
      "z1": 15.875,
      "framed1": "Not",
      "permanentLabware": false,
      "required": {
        "standard TiterPlate Size": null
      }
    }
  }
}
```

```

    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p10": {
    "alpType": "Static1x3",
    "name": "P10",
    "x1": 54.96980000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p11": {
    "alpType": "Static1x1",
    "name": "P11",
    "x1": 69.54940000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p12": {
    "alpType": "Static1x1",
    "name": "P12",
    "x1": 69.54940000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p13": {
    "alpType": "Static1x3",
    "name": "P13",
    "x1": 69.54940000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p14": {
    "alpType": "Static1x3",
    "name": "P14",
    "x1": 69.54940000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p15": {
    "alpType": "Static1x3",
    "name": "P15",
    "x1": 69.54940000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,

```

```

    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p16": {
    "alpType": "Static1x1",
    "name": "P16",
    "x1": 84.12900000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p17": {
    "alpType": "Static1x1",
    "name": "P17",
    "x1": 84.12900000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p18": {
    "alpType": "Static1x3",
    "name": "P18",
    "x1": 84.12900000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p19": {
    "alpType": "Static1x3",
    "name": "P19",
    "x1": 84.12900000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p2": {
    "alpType": "Static1x1",
    "name": "P2",
    "x1": 40.39020000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p20": {
    "alpType": "Static1x3",

```

```

    "name": "P20",
    "x1": 84.12900000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p21": {
    "alpType": "Static1x1",
    "name": "P21",
    "x1": 98.70860000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p22": {
    "alpType": "Static1x1",
    "name": "P22",
    "x1": 98.70860000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p23": {
    "alpType": "Static1x3",
    "name": "P23",
    "x1": 98.70860000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p24": {
    "alpType": "Static1x3",
    "name": "P24",
    "x1": 98.70860000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p25": {
    "alpType": "Static1x3",
    "name": "P25",
    "x1": 98.70860000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  }
}

```

```

    }
  },
  "p26": {
    "alpType": "Static1x1",
    "name": "P26",
    "x1": 113.28820000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p27": {
    "alpType": "Static1x1",
    "name": "P27",
    "x1": 113.28820000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p28": {
    "alpType": "Static1x3",
    "name": "P28",
    "x1": 113.28820000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p29": {
    "alpType": "Static1x3",
    "name": "P29",
    "x1": 113.28820000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p3": {
    "alpType": "Static1x3",
    "name": "P3",
    "x1": 40.39020000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p30": {
    "alpType": "Static1x3",
    "name": "P30",
    "x1": 113.28820000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {

```

```

        "standard TiterPlate Size": null
    },
    "characteristics": {
        "span Can Load Tips": null
    }
},
"p4": {
    "alpType": "Static1x3",
    "name": "P4",
    "x1": 40.39020000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
        "standard TiterPlate Size": null
    },
    "characteristics": {
        "span Can Load Tips": null
    }
},
"p5": {
    "alpType": "Static1x3",
    "name": "P5",
    "x1": 40.39020000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
        "standard TiterPlate Size": null
    },
    "characteristics": {
        "span Can Load Tips": null
    }
},
"p6": {
    "alpType": "Static1x1",
    "name": "P6",
    "x1": 54.96980000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
        "standard TiterPlate Size": null
    },
    "characteristics": {
        "span Can Load Tips": null
    }
},
"p7": {
    "alpType": "Static1x1",
    "name": "P7",
    "x1": 54.96980000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
        "standard TiterPlate Size": null
    },
    "characteristics": {
        "span Can Load Tips": null
    }
},
"p8": {
    "alpType": "Static1x3",
    "name": "P8",
    "x1": 54.96980000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
        "standard TiterPlate Size": null
    },
    "characteristics": {
        "span Can Load Tips": null
    }
},
"p9": {
    "alpType": "Static1x3",
    "name": "P9",
    "x1": 54.96980000190736,
    "y1": 46.78999963684082,

```

```

    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "tL1": {
    "alpType": "TipLoad1x1",
    "name": "TL1",
    "x1": 25.81060000190735,
    "y1": 15.547999705505372,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL2": {
    "alpType": "TipLoad1x1",
    "name": "TL2",
    "x1": 25.81060000190735,
    "y1": 25.961999705505374,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL3": {
    "alpType": "TipLoad1x1",
    "name": "TL3",
    "x1": 25.81060000190735,
    "y1": 36.37599970550538,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL4": {
    "alpType": "TipLoad1x1",
    "name": "TL4",
    "x1": 25.81060000190735,
    "y1": 46.78999970550537,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL5": {
    "alpType": "TipLoad1x1",
    "name": "TL5",
    "x1": 25.81060000190735,
    "y1": 57.203999705505375,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {

```

```

    "multichannel Can Load Tips": null,
    "span Can Load Tips": null
  }
},
"tr1": {
  "alpType": "TrashLeftSlide",
  "name": "TR1",
  "x1": 10.922000259399418,
  "y1": 30.147900088500975,
  "z1": 20.612,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "can Discard Tips": null,
    "disableFrameWithGripper": null,
    "disallow AutoTeach": null,
    "disallow ManualTeach": null,
    "disallow NormalTeach": null,
    "trash": null
  },
  "device": "Trash"
},
"tr2": {
  "alpType": "TrashRightSlide",
  "name": "TR2",
  "x1": 128.1757999909401,
  "y1": 30.147900088500975,
  "z1": 20.612,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "can Discard Tips": null,
    "disableFrameWithGripper": null,
    "disallow AutoTeach": null,
    "disallow ManualTeach": null,
    "disallow NormalTeach": null,
    "trash": null
  },
  "device": "Trash"
},
"w1": {
  "alpType": "WashStationSpan8",
  "name": "W1",
  "x1": 142.24009992294313,
  "y1": 15.342999946594238,
  "z1": 22.1361,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "no plates allowed": null
  },
  "characteristics": {
    "disableFrameWithGripper": null,
    "disallow AutoTeach": null,
    "disallow NormalTeach": null,
    "no Gripping Allowed": null,
    "vWash Station": null
  }
},
"ws1": {
  "alpType": "WashStation96",
  "name": "WS1",
  "x1": 10.33789996814728,
  "y1": 13.428300212097168,
  "z1": 8.5623,
  "framed1": "Not",
  "permanentLabware": true,
  "labware": {
    "class": "WashStation",
    "properties": {
      "liquidtype": "Water"
    },
    "volumeType": "Known"
  },
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "disallow ManualTeach": null,
    "no Gripping Allowed": null,

```

```
    "wash Station": null  
  }  
}  
}
```

† Table of contents

Deck Layout: Biomek i7 Multichannel

- Source: `deckLayouts.multichannel.positions` in the Biomek Instrument Settings export.
- Illustrative view: `$id / $ref` references are resolved inline, and hardware-internal keys (ALP `group` blocks, footprint spans, framing internals, Pod-2 coordinates, geometry-only fields, runtime state) are stripped. Only the per-position keys you need to read a deck remain.
- Pod2 note: Pod2 coordinates (`x2 / y2 / z2 / framed2` in the raw export) are omitted from the illustrative view. On stock decks they equal Pod1 because the deck is unframed (uncalibrated); on a framed deck they carry the Pod2 calibration.
- Full key reference: see Deck Layouts Format Specification.
- ALP types: see ALP Catalog.

Illustrative, not a literal export. This is a cleaned, readability-oriented view of the deck: `$id / $ref` references are resolved inline and non-position detail (ALP hardware groups, framing/geometry internals, Pod-2 coordinates, ...) is removed. In particular, every position in your real export carries a `group` object (a *position group*) — the shared definition of the ALP that position belongs to, linked across its positions via `$id / $ref`; it is per-ALP hardware data, not position placement, so it is dropped here. Your real export looks messier — it uses `$id / $ref` object sharing and carries additional keys — so treat this as a guide to the keys, not a byte-for-byte copy or deterministic parser output. See the deck-layout format spec for the full key reference.

Visual Layout

The grid below is a **derived aid** — it is not part of the export. It mirrors the Biomek **Deck Editor**'s layout of this deck. The deck is a grid of mounting holes addressed by **column letter** and **row number**, with the origin at the back-left corner: column `A` is the left edge and row `1` is the back of the deck, so the top row here is the back and the bottom row is the front (operator side). The Deck Editor does not label every hole — it labels one column per **deck plate** (hence `A`, then `F`, `M`, `T`, ... on an i5/i7, or `A`, `H`, `O`, `V` on an i3) and one row every 5 holes — and the grid below uses exactly those labeled columns and rows. Each ALP occupies one cell; an ALP whose body genuinely covers more than one deck plate (a trash slide, which overhangs the side of the deck) is drawn as one wide cell spanning them. The real export carries the cm coordinates (`x1 / y1`) plus each ALP group's `column / row`.

Row	A	F	M	T	AA	AH	AO	AV	BC	BJ	BQ
5	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]
10	[empty]	[WS1]	[TL1]	[P1]	[P6]	[P11]	[P16]	[P21]	[P26]	[empty]	[empty]
15	[TR1	]		[TL2]	[P2]	[P7]	[P12]	[P17]	[P22]	[P27]	[empty]
20	[empty]	[empty]	[TL3]	[P3]	[P8]	[P13]	[P18]	[P23]	[P28]	[empty]	[empty]
25	[empty]	[empty]	[TL4]	[P4]	[P9]	[P14]	[P19]	[P24]	[P29]	[empty]	[empty]
30	[empty]	[empty]	[TL5]	[P5]	[P10]	[P15]	[P20]	[P25]	[P30]	[empty]	[empty]

Deck (illustrative, resolved)

The block below mirrors the shape of `deckLayouts.multichannel` in the export, with `$ref` pointers resolved and non-load-bearing keys removed. Values shown (coordinates in cm, `alpType`, `labware class / properties.liquidtype / volumeType`, `device name`, `characteristic` and required keys) are copied verbatim from the export; the surrounding `labware` object is reduced to those fields, and `device` is collapsed from the export's device object to just its `name` string.

```
{
  "name": "Multichannel",
  "isDefault": true,
  "isReadOnly": false,
  "positions": {
    "p1": {
      "alpType": "Static1x1",
      "name": "P1",
      "x1": 40.39020000190736,
      "y1": 15.547999705505372,
      "z1": 15.875,
      "framed1": "Not",
      "permanentLabware": false,
      "required": {
        "standard TiterPlate Size": null
      }
    }
  }
}
```

```

    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p10": {
    "alpType": "Static1x3",
    "name": "P10",
    "x1": 54.96980000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p11": {
    "alpType": "Static1x1",
    "name": "P11",
    "x1": 69.54940000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p12": {
    "alpType": "Static1x1",
    "name": "P12",
    "x1": 69.54940000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p13": {
    "alpType": "Static1x3",
    "name": "P13",
    "x1": 69.54940000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p14": {
    "alpType": "Static1x3",
    "name": "P14",
    "x1": 69.54940000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p15": {
    "alpType": "Static1x3",
    "name": "P15",
    "x1": 69.54940000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,

```

```

    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p16": {
    "alpType": "Static1x1",
    "name": "P16",
    "x1": 84.12900000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p17": {
    "alpType": "Static1x1",
    "name": "P17",
    "x1": 84.12900000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p18": {
    "alpType": "Static1x3",
    "name": "P18",
    "x1": 84.12900000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p19": {
    "alpType": "Static1x3",
    "name": "P19",
    "x1": 84.12900000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p2": {
    "alpType": "Static1x1",
    "name": "P2",
    "x1": 40.39020000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p20": {
    "alpType": "Static1x3",

```

```

    "name": "P20",
    "x1": 84.12900000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p21": {
    "alpType": "Static1x1",
    "name": "P21",
    "x1": 98.70860000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p22": {
    "alpType": "Static1x1",
    "name": "P22",
    "x1": 98.70860000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p23": {
    "alpType": "Static1x3",
    "name": "P23",
    "x1": 98.70860000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p24": {
    "alpType": "Static1x3",
    "name": "P24",
    "x1": 98.70860000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p25": {
    "alpType": "Static1x3",
    "name": "P25",
    "x1": 98.70860000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  }
}

```

```

    }
  },
  "p26": {
    "alpType": "Static1x1",
    "name": "P26",
    "x1": 113.28820000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p27": {
    "alpType": "Static1x1",
    "name": "P27",
    "x1": 113.28820000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p28": {
    "alpType": "Static1x3",
    "name": "P28",
    "x1": 113.28820000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p29": {
    "alpType": "Static1x3",
    "name": "P29",
    "x1": 113.28820000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p3": {
    "alpType": "Static1x3",
    "name": "P3",
    "x1": 40.39020000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p30": {
    "alpType": "Static1x3",
    "name": "P30",
    "x1": 113.28820000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {

```

```

    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p4": {
  "alpType": "Static1x3",
  "name": "P4",
  "x1": 40.39020000190736,
  "y1": 46.78999963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p5": {
  "alpType": "Static1x3",
  "name": "P5",
  "x1": 40.39020000190736,
  "y1": 57.20399963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p6": {
  "alpType": "Static1x1",
  "name": "P6",
  "x1": 54.96980000190736,
  "y1": 15.547999705505372,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p7": {
  "alpType": "Static1x1",
  "name": "P7",
  "x1": 54.96980000190736,
  "y1": 25.961999705505374,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p8": {
  "alpType": "Static1x3",
  "name": "P8",
  "x1": 54.96980000190736,
  "y1": 36.375999636840824,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p9": {
  "alpType": "Static1x3",
  "name": "P9",
  "x1": 54.96980000190736,
  "y1": 46.78999963684082,

```

```

    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "tL1": {
    "alpType": "TipLoad1x1",
    "name": "TL1",
    "x1": 25.81060000190735,
    "y1": 15.547999705505372,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL2": {
    "alpType": "TipLoad1x1",
    "name": "TL2",
    "x1": 25.81060000190735,
    "y1": 25.961999705505374,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL3": {
    "alpType": "TipLoad1x1",
    "name": "TL3",
    "x1": 25.81060000190735,
    "y1": 36.37599970550538,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL4": {
    "alpType": "TipLoad1x1",
    "name": "TL4",
    "x1": 25.81060000190735,
    "y1": 46.78999970550537,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "multichannel Can Load Tips": null,
      "span Can Load Tips": null
    }
  },
  "tL5": {
    "alpType": "TipLoad1x1",
    "name": "TL5",
    "x1": 25.81060000190735,
    "y1": 57.203999705505375,
    "z1": 16.129,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {

```

```

        "multichannel Can Load Tips": null,
        "span Can Load Tips": null
    }
},
"tr1": {
    "alpType": "TrashLeftSlide",
    "name": "TR1",
    "x1": 10.922000259399418,
    "y1": 30.147900088500975,
    "z1": 20.612,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
        "standard TiterPlate Size": null
    },
    "characteristics": {
        "can Discard Tips": null,
        "disableFrameWithGripper": null,
        "disallow AutoTeach": null,
        "disallow ManualTeach": null,
        "disallow NormalTeach": null,
        "trash": null
    },
    "device": "Trash"
},
"ws1": {
    "alpType": "WashStation96",
    "name": "WS1",
    "x1": 10.33789996814728,
    "y1": 13.428300212097168,
    "z1": 8.5623,
    "framed1": "Not",
    "permanentLabware": true,
    "labware": {
        "class": "WashStation",
        "properties": {
            "liquidtype": "Water"
        },
        "volumeType": "Known"
    },
    "required": {
        "standard TiterPlate Size": null
    },
    "characteristics": {
        "disallow ManualTeach": null,
        "no Gripping Allowed": null,
        "wash Station": null
    }
}
}
}
}

```

Deck Layout: Biomek i7 Span8

- Source: `deckLayouts.span8.positions` in the Biomek Instrument Settings export.
- Illustrative view: `$id / $ref` references are resolved inline, and hardware-internal keys (ALP `group` blocks, footprint spans, framing internals, Pod-2 coordinates, geometry-only fields, runtime state) are stripped. Only the per-position keys you need to read a deck remain.
- Pod2 note: Pod2 coordinates (`x2 / y2 / z2 / framed2` in the raw export) are omitted from the illustrative view. On stock decks they equal Pod1 because the deck is unframed (uncalibrated); on a framed deck they carry the Pod2 calibration.
- Full key reference: see Deck Layouts Format Specification.
- ALP types: see ALP Catalog.

Illustrative, not a literal export. This is a cleaned, readability-oriented view of the deck: `$id / $ref` references are resolved inline and non-position detail (ALP hardware groups, framing/geometry internals, Pod-2 coordinates, ...) is removed. In particular, every position in your real export carries a `group` object (a *position group*) — the shared definition of the ALP that position belongs to, linked across its positions via `$id / $ref`; it is per-ALP hardware data, not position placement, so it is dropped here. Your real export looks messier — it uses `$id / $ref` object sharing and carries additional keys — so treat this as a guide to the keys, not a byte-for-byte copy or deterministic parser output. See the deck-layout format spec for the full key reference.

Visual Layout

The grid below is a **derived aid** — it is not part of the export. It mirrors the Biomek **Deck Editor**'s layout of this deck. The deck is a grid of mounting holes addressed by **column letter** and **row number**, with the origin at the back-left corner: column **A** is the left edge and row **1** is the back of the deck, so the top row here is the back and the bottom row is the front (operator side). The Deck Editor does not label every hole — it labels one column per **deck plate** (hence **A**, then **F**, **M**, **T**, ... on an i5/i7, or **A**, **H**, **O**, **V** on an i3) and one row every 5 holes — and the grid below uses exactly those labeled columns and rows. Each ALP occupies one cell; an ALP whose body genuinely covers more than one deck plate (a trash slide, which overhangs the side of the deck) is drawn as one wide cell spanning them. The real export carries the cm coordinates (`x1 / y1`) plus each ALP group's `column / row`.

Row	A	F	M	T	AA	AH	AO	AV	BC	BJ	BQ
5	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]	[empty]
10	[empty]	[empty]	[empty]	[P1]	[P6]	[P11]	[P16]	[P21]	[P26]	[empty]	[W1]
15	[empty]	[empty]	[empty]	[P2]	[P7]	[P12]	[P17]	[P22]	[P27]	[TR1]	]
20	[empty]	[empty]	[empty]	[P3]	[P8]	[P13]	[P18]	[P23]	[P28]	[empty]	
25	[empty]	[empty]	[empty]	[P4]	[P9]	[P14]	[P19]	[P24]	[P29]	[empty]	[empty]
30	[empty]	[empty]	[empty]	[P5]	[P10]	[P15]	[P20]	[P25]	[P30]	[empty]	[empty]

Deck (illustrative, resolved)

The block below mirrors the shape of `deckLayouts.span8` in the export, with `$ref` pointers resolved and non-load-bearing keys removed. Values shown (coordinates in cm, `alpType`, labware `class / properties.liquidtype / volumeType`, device `name`, characteristic and required keys) are copied verbatim from the export; the surrounding `labware` object is reduced to those fields, and `device` is collapsed from the export's device object to just its `name` string.

```
{
  "name": "Span8",
  "isDefault": false,
  "isReadOnly": false,
  "positions": {
    "p1": {
      "alpType": "Static1x1",
      "name": "P1",
      "x1": 40.39020000190736,
      "y1": 15.547999705505372,
      "z1": 15.875,
      "framed1": "Not",
      "permanentLabware": false,
      "required": {
        "standard TiterPlate Size": null
      }
    }
  }
}
```

```

    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p10": {
    "alpType": "Static1x3",
    "name": "P10",
    "x1": 54.96980000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p11": {
    "alpType": "Static1x1",
    "name": "P11",
    "x1": 69.54940000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p12": {
    "alpType": "Static1x1",
    "name": "P12",
    "x1": 69.54940000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p13": {
    "alpType": "Static1x3",
    "name": "P13",
    "x1": 69.54940000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p14": {
    "alpType": "Static1x3",
    "name": "P14",
    "x1": 69.54940000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p15": {
    "alpType": "Static1x3",
    "name": "P15",
    "x1": 69.54940000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,

```

```

    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p16": {
    "alpType": "Static1x1",
    "name": "P16",
    "x1": 84.12900000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p17": {
    "alpType": "Static1x1",
    "name": "P17",
    "x1": 84.12900000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p18": {
    "alpType": "Static1x3",
    "name": "P18",
    "x1": 84.12900000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p19": {
    "alpType": "Static1x3",
    "name": "P19",
    "x1": 84.12900000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p2": {
    "alpType": "Static1x1",
    "name": "P2",
    "x1": 40.39020000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p20": {
    "alpType": "Static1x3",

```

```
"name": "P20",
"x1": 84.12900000190736,
"y1": 57.20399963684082,
"z1": 15.875,
"framed1": "Not",
"permanentLabware": false,
"required": {
  "standard TiterPlate Size": null
},
"characteristics": {
  "span Can Load Tips": null
}
},
"p21": {
  "alpType": "Static1x1",
  "name": "P21",
  "x1": 98.70860000190736,
  "y1": 15.547999705505372,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p22": {
  "alpType": "Static1x1",
  "name": "P22",
  "x1": 98.70860000190736,
  "y1": 25.961999705505374,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p23": {
  "alpType": "Static1x3",
  "name": "P23",
  "x1": 98.70860000190736,
  "y1": 36.375999636840824,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p24": {
  "alpType": "Static1x3",
  "name": "P24",
  "x1": 98.70860000190736,
  "y1": 46.78999963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p25": {
  "alpType": "Static1x3",
  "name": "P25",
  "x1": 98.70860000190736,
  "y1": 57.20399963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
}
```

```

    }
  },
  "p26": {
    "alpType": "Static1x1",
    "name": "P26",
    "x1": 113.28820000190736,
    "y1": 15.547999705505372,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p27": {
    "alpType": "Static1x1",
    "name": "P27",
    "x1": 113.28820000190736,
    "y1": 25.961999705505374,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p28": {
    "alpType": "Static1x3",
    "name": "P28",
    "x1": 113.28820000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p29": {
    "alpType": "Static1x3",
    "name": "P29",
    "x1": 113.28820000190736,
    "y1": 46.78999963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p3": {
    "alpType": "Static1x3",
    "name": "P3",
    "x1": 40.39020000190736,
    "y1": 36.375999636840824,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "p30": {
    "alpType": "Static1x3",
    "name": "P30",
    "x1": 113.28820000190736,
    "y1": 57.20399963684082,
    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {

```

```

    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p4": {
  "alpType": "Static1x3",
  "name": "P4",
  "x1": 40.39020000190736,
  "y1": 46.78999963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p5": {
  "alpType": "Static1x3",
  "name": "P5",
  "x1": 40.39020000190736,
  "y1": 57.20399963684082,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p6": {
  "alpType": "Static1x1",
  "name": "P6",
  "x1": 54.96980000190736,
  "y1": 15.547999705505372,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p7": {
  "alpType": "Static1x1",
  "name": "P7",
  "x1": 54.96980000190736,
  "y1": 25.961999705505374,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p8": {
  "alpType": "Static1x3",
  "name": "P8",
  "x1": 54.96980000190736,
  "y1": 36.375999636840824,
  "z1": 15.875,
  "framed1": "Not",
  "permanentLabware": false,
  "required": {
    "standard TiterPlate Size": null
  },
  "characteristics": {
    "span Can Load Tips": null
  }
},
"p9": {
  "alpType": "Static1x3",
  "name": "P9",
  "x1": 54.96980000190736,
  "y1": 46.78999963684082,

```

```

    "z1": 15.875,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "span Can Load Tips": null
    }
  },
  "tR1": {
    "alpType": "TrashRightSlide",
    "name": "TR1",
    "x1": 128.175799909401,
    "y1": 30.147900088500975,
    "z1": 20.612,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "standard TiterPlate Size": null
    },
    "characteristics": {
      "can Discard Tips": null,
      "disableFrameWithGripper": null,
      "disallow AutoTeach": null,
      "disallow ManualTeach": null,
      "disallow NormalTeach": null,
      "trash": null
    },
    "device": "Trash"
  },
  "w1": {
    "alpType": "WashStationSpan8",
    "name": "W1",
    "x1": 142.24009992294313,
    "y1": 15.342999946594238,
    "z1": 22.1361,
    "framed1": "Not",
    "permanentLabware": false,
    "required": {
      "no plates allowed": null
    },
    "characteristics": {
      "disableFrameWithGripper": null,
      "disallow AutoTeach": null,
      "disallow NormalTeach": null,
      "no Gripping Allowed": null,
      "vWash Station": null
    }
  }
}
}
}

```

Project Items Format Specification

Planned / future format — not a current Biomek export. Biomek has **no JSON export for project items** today; its JSON formats cover only methods (import and export) and instrument settings (export only). The catalogs described here are a **forward-looking preview** built from the default project configurations, not something the released software emits — treat them as reference, not as a guide to authoring importable project items today.

These are the defaults, and project items are broader than they look. The companion `catalog-*.md` files are *derived summaries* of the default Biomek project configurations; the values they show are the **defaults** included with Biomek. Project items include labware classes, labware patterns, liquid types, tip classes, and **pipetting techniques and templates**. Many customers maintain their own project items in addition to the defaults.

Purpose

These catalogs give a readable, per-category summary of the project-item definitions Biomek includes by default — the labware classes, tip classes, liquid types, well patterns, pipetting techniques, and templates a method can reference. A project-item definition is reusable metadata: methods instantiate and reference it at runtime (many labware instances can share one labware class).

Biomek itself has no JSON export for project items. The catalogs are therefore a derived summary, not an importable export, and — apart from the technique catalog, which mirrors the real technique-serializer shape — they do not follow the envelope conventions of Biomek's own JSON (`format / formatVersion` , `$id / $ref` , and so on).

Companion Catalogs

The defaults are broken out by category into the six per-category catalog files below. Each catalog aggregates one category across the default instrument projects, so labware classes, tip classes, techniques, and so on each live in one place:

Catalog	Shape	What it lists
<code>catalog-labware-classes.md</code>	prose + a table per item type	the default labware classes
<code>catalog-labware-patterns.md</code>	prose + reference form	the default 384-well quadrant patterns
<code>catalog-liquid-types.md</code>	prose	the default liquid types (Water, Serum)
<code>catalog-tip-classes.md</code>	prose + one table	the default tip classes
<code>catalog-techniques.md</code>	one JSON block per technique	the default pipetting techniques
<code>catalog-templates.md</code>	prose	the default pipetting templates

Item shape and conventions

The technique catalog, which lists items as JSON, uses a **flat per-item block** — there is no project/category envelope. The labware and tip-class catalogs describe the same per-item shape but present it as tables. Each item carries the fields relevant to its category:

- itemName** — the item's key (for example `T1025F` , `SeptaFluted`).
- hardwareCompatibility** — present on every cataloged item (see below).
- properties** — the method-authoring-relevant values for the item (per-category sets below).
- Labware classes additionally carry **itemType** (`TiterPlate` , `TipBox` , `TubeRack` , `Reservoir` , `Lid` , Or `Reservation`), **characteristics** (presence flags such as `Can Pipette` and `Moveable`), and **stackableOnLabwareClasses** (the classes this class may stack on, including itself when it self-stacks). Tip classes and techniques carry none of these three. In `catalog-labware-classes.md` these three surface as the per-type sections, the per-section prose notes, and the **Stacks on** table column respectively.

Techniques are the exception to the flat-item convention: the technique catalog emits the real serialized technique object — top-level `name` (not `itemName`), the technique's own nested blocks (aspirate / dispense / mix / context / general) rather than a `properties` wrapper, and `hardwareCompatibility` shown as a catalog annotation above each block rather than a field inside it.

Liquid types and templates are documented as **prose** — liquid behavior is better viewed in the Liquid Type editor, and templates are step-tree definitions best viewed in the Template Editor. Labware patterns are documented as prose plus the **by-name reference form** used in a step (`localPattern: false` + `referencedPattern`).

hardwareCompatibility

`hardwareCompatibility` is **pod-based**: a list drawn from `Multichannel`, `Span-8`, and `Fixed-8`. Chassis size does not affect it, and a hybrid instrument supports the union of its pods. It is a catalog annotation, **derived** from which default projects include the item — not an intrinsic, importable field. Each pod tag is derived from the single-pod default project that uses that pod:

Default single-pod project	Pod
i3	Fixed-8
i5 Multichannel	Multichannel
i5 Span-8	Span-8

An item's `hardwareCompatibility` is the set of those pods whose project includes the item, or `["any"]` when all three do. The i7 Hybrid project (which has both a Multichannel and a Span-8 pod) is **not** used to add pod tags, because membership there cannot distinguish which pod an item is for.

Properties by category

Each item emits only the keys its own definition carries.

Category	properties keys
Labware Classes	<code>Description</code> , <code>PartNumber</code> , <code>DefaultLidClass</code> , <code>Height</code> , <code>XSpan</code> , <code>YSpan</code> , <code>WellsX</code> , <code>WellsY</code> , <code>WellStyle</code> , <code>WellVolume</code> , <code>WellDepth</code> , <code>WellXSpacing</code> , <code>WellYSpacing</code> , <code>Wells(0).MaxVolume</code> (the maximum volume of the representative well, index 0; the key name mirrors the source property path), plus <code>Refills</code> (Reservoir classes) and <code>RequiresPiercing</code> (septum-capped tube-rack classes). The class type is carried in <code>itemType</code> , not repeated in <code>properties</code> . These are the keys that appear across labware classes; a given class emits only the subset its type carries — e.g. Reservation classes emit only <code>Description</code> / <code>Height</code> / <code>XSpan</code> / <code>YSpan</code> / <code>WellsX</code> / <code>WellsY</code> , Lid classes omit <code>DefaultLidClass</code> and well-depth/max-volume keys, and Reservoir classes omit most well-geometry keys. <code>catalog-labware-classes.md</code> carries the method-authoring-relevant subset of these as table columns — <code>Wells(0).MaxVolume</code> as the max-volume column, <code>WellsX</code> / <code>WellsY</code> as the wells column, <code>WellStyle</code> as the well-shape column, <code>DefaultLidClass</code> as the default-lid column — and omits the pure-geometry keys.
Tip Classes	<code>AirCapacity</code> , <code>Capacity</code> , <code>Filtered</code> , <code>Fixed</code> , <code>LLS</code> , <code>Piercing</code> , <code>TipHeight</code> , <code>UseSystemTubing</code> . Every tip class emits all eight. <code>catalog-tip-classes.md</code> carries the first six as table columns and states the three uniform ones — <code>Fixed</code> , <code>Piercing</code> , <code>UseSystemTubing</code> — in prose, since only <code>Fixed100</code> and <code>SeptaFluted</code> differ.
Techniques	the real serialized technique object (<code>_biomekType: "technique"</code> plus the technique's nested aspirate / dispense / mix / context blocks) — see the technique catalog for the shape and a worked example

Reading notes:

- `Refills` applies to `Reservoir`-type classes and no other, and is the **Refills** column in the reservoir quick reference; `RequiresPiercing` applies only to the septum-capped `BCSeptaTubeRack_*` classes, noted in prose in the Tube Racks section.
- In the underlying source data a liquid type's `isDefault` marks the project default (Water) and is absent, not `false`, on the others; the liquid catalog is prose-only and does not surface this field directly.
- The technique catalog shows a few techniques in full and the rest with their identifying fields only, to keep the page compact; open a technique in the Technique Editor for fully resolved values.

Notes

- Project items are definitions, not runtime instances; methods consume them and instantiate labware/tip/liquid usage dynamically.
- Instrument-specific projects vary in which classes, patterns, and techniques they provide.

Catalog: Labware Classes

The names here are real and usable. These entries are the **default project items included with Biomek**: a method references them **by name**, and these are the valid names to use. Project items are not a file you author or import. A specific project may add its own, so treat this as the default set, not an exhaustive list.

Every unique Beckman default labware class is listed once, deduplicated across the three single-pod default projects. The **Pods** column in each table records `hardwareCompatibility`, which is **pod-based** (Multichannel / Span-8 / Fixed-8); chassis size does not affect it, and a hybrid instrument supports the union of its pods. The field is tentative: it is a catalog annotation derived from which default projects include the item (mapped to their pods), not necessarily an intrinsic field of the forthcoming format. A Pods value of `any` means all three default projects include the class.

The **Pods column is a membership hint, not a guarantee that the pod can use the labware**. It says the class is included in that pod's default project; it does not promise the pod can reach the position you place it at, or that the pod can access that labware type at all. A class marked `any` can still fail with `Cannot access labware of type <class> with <pod>.`, and a class marked for one pod family can still be unreachable where you put it. Reach is a property of the deck position and the pod — see [Reachability and Access](#).

Scope and derivation

- Category:** Labware Classes (`Labware Classes`).
- Sources:** the three single-pod default projects (`Biomek_i3Project`, `Biomek_i5Project-MC`, `Biomek_i5Project-Span`).
- Pod mapping used to derive `hardwareCompatibility`:** each pod tag is derived from the single-pod default project that uses that pod (`Fixed-8` from `Biomek_i3Project`, `Multichannel` from `Biomek_i5Project-MC`, `Span-8` from `Biomek_i5Project-Span`); an item's compatibility is the set of those pods whose project includes it, or `["any"]` when all three do. The `Biomek_i7Project` (Hybrid) is not used to add pod tags — it carries both pods, so membership there cannot distinguish which pod an item is for. Excluding it costs no coverage: its labware classes are exactly the union of the three single-pod projects.

This catalog covers six item types: reservoirs, titer plates, tip boxes, tube racks, lids, and reservations. Reservoirs come first and get the most space because their addressing model differs from every other type — a reservoir is a list of sections, not a well grid.

Reservoirs

A reservoir labware class is **not** a WellsX-by-WellsY grid. It is modeled as a `Sections[]` **array**, and each section is the addressable unit for pipetting. A section is addressed by its **1-based position** in the type's section list. When authoring a pipetting step that targets a reservoir:

- The `selectionInfo / pattern` boolean comArray length equals the reservoir's **section count** (one flag per section position).
- Each flag selects (`true`) or deselects (`false`) the section at that position.
- Section ordering is **author-defined** in the Labware Type Editor's Reservoir Builder (sorting mode + manual Section ID), so section 1 is not necessarily the leftmost cavity.

For the full numbering rule see Reservoirs and multi-section troughs (in *Biomek JSON Authoring Guide*) in [Introduction-to-Pipetting-Steps.md](#). For deck placement see the sample deck layouts.

Quick reference

Class	Sections	Max volume per section (µL)	Refills	Top-section depth (cm)	Bottom shape	Pods	Description
<code>AgilentReservoir</code>	1	300000	No	3.922	Flat	any	Agilent Seahorse Microplates (formerly Innovative Microplate): single cavity polypropylene

Class	Sections	Max volume per section (µL)	Refills	Top-section depth (cm)	Bottom shape	Pods	Description
							reservoir, 300ml, 96 pyramids base geometry, 44mm height.
BCFullReservoir	1	200000	No	2.159	V-shaped	Span-8	Beckman Coulter reservoir with baffles used to hold bulk reagents for dispensing by the Span-8 Pod
BCUpsideDownTipBoxLid	1	160000	No	1.71	Flat	any	Upside down lid from a Beckman Coulter tip box used as a reservoir
CirculatingReservoir	1	250000	Yes	3.78	Flat	Multichannel, Span-8	Circulating reservoir for use with the ReservoirTipBox ALP
DrainableRefillableReservoir	1	71606	No	1.905	Flat	Multichannel, Span-8	Beckman Coulter drainable and refillable reservoir
ModularReservoir	4	40700	No	2.479	V-shaped	any	Beckman Coulter reservoir that may be customized with added sections
Reservoir	1	110000	No	1.453	Flat	any	Reservoir used with the Multimek

All seven carry `Can Pipette` and `Standard TiterPlate Size`. Only `AgilentReservoir`, `BCFullReservoir`, and `Reservoir` also carry `Moveable`; a `Move Labware` step cannot reposition the other four. **Refills** marks a reservoir replenished from an external source during the run, which among the defaults is only `CirculatingReservoir`. **Top-section depth** is the Z-span of the section's upper, straight-walled portion; each class has a further bottom section beneath it, so this figure is not the full distance to the cavity floor. `ModularReservoir`'s default configuration is four equal quarter-sections arranged left to right.

Worked example: Transfer step items targeting a reservoir

Aspirating from a single-section reservoir (`BCFullReservoir`) — the `pattern` and `selectionInfo` arrays each have **1** element (one per section):

```
// Source item inside a Transfer step's "items" array
{
  "labwareClass": "BCFullReservoir",
  "position": "P3",
  "liquidType": "Well Contents",
  "volume": "200",
  // ... other Transfer-item keys ...
  "pattern": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true] },
  "selectionInfo": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true] }
}
```

For a 4-section `ModularReservoir`, select only section 2 by setting the second flag:

```
{
  "labwareClass": "ModularReservoir",
  "position": "P3",
  "liquidType": "Well Contents",
  "volume": "100",
  // pattern length = 4 (one per section):
  "pattern": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [false, true, false, false] },
  "selectionInfo": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [false, true, false, false] }
}
```

Best practice: aspirate from reservoirs bottom-relative, not liquid-relative. Reservoir aspiration is conventionally referenced to the section bottom (`overrideHeight: true`, `heightFrom: 1 = "Bottom"`, with a small positive `height` above the floor). Fixed-8 and Multichannel pods do not perform liquid-level sensing (only Span-8 does — see the LLS note in Catalog: Tip Classes), so a liquid-relative move (`heightFrom: 0 = "Liquid"`) over a reservoir on those pods has no surface to reference and fails. Bottom-referenced also composes correctly with the section geometry documented here (the max-volume and top-section-depth columns above). For the enclosing item

shape and other height keys see Transfer (in *Biomek JSON Authoring Guide*); to have the engine track reservoir starting/dead volumes so heights and volume checks stay accurate, declare the section volumes in Instrument Setup (in *Biomek JSON Authoring Guide*).

```
// Source item aspirating from a reservoir, bottom-referenced:
{
  "labwareClass": "Reservoir",
  "position": "P3",
  "liquidType": "Well Contents",
  "volume": "150",
  "overrideHeight": true,
  "heightFrom": 1,           // 1 = "Bottom"; 0 = "Liquid" would require LLS (Span-8 only)
  "height": 1.0,            // mm above the section bottom
  "pattern": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true] },
  "selectionInfo": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true] }
}
```

Titer Plates

TiterPlate labware classes model an addressable WellsX × WellsY grid. Numeric and class-name values below are reproduced exactly from each class's default project-item properties; Description text is reproduced with terminal punctuation normalized. Per-well physical dimensions (heights, spans, spacings) live in the labware class definition, viewable in the Labware Type Editor, and are omitted from this table.

Class name	Wells	Max vol (μL)	Well shape	Default lid	Stacks on	Pods	Description
AB384WellReactionPlate	24×16	40.0	Round	TiterplateLid	self	any	384 well thermal cycler style plate
AB_0661	12×8	2200.0	Rectangle	—	self	any	ABGene 0661 (individual) and 0778 (bulk) deepwell storage plates.
AB_0765	12×8	830.0	Round	—	self	any	Abgene 0765 or 0865 polypropylene microplate, half as tall as a standard deep well plate, with 96 conical-bottomed wells.
AB_0932	12×8	2070.0	Rectangle	—	self	any	AbGene 0932 deepwell storage plate.
AB_1127	12×8	1200.0	Rectangle	—	self, Axygen_P96450VC	any	AbGene 1127, 96 well plate, 1.2ml square well with round bottom.
Axygen_P96450VC	12×8	500.0	Round	—	self	any	Axygen P-96-450VC V-bottom

Class name	Wells	Max vol (µL)	Well shape	Default lid	Stacks on	Pods	Description
							microplate, clear, sterile.
BCDeep96Round	12×8	1317.9726	Round	TiterplateLid	—	any	Beckman Coulter microplate with 96 deep, round-bottomed wells.
BCFlat96	12×8	362.76	Round	TiterplateLid	self	any	Beckman Coulter microplate with 96 round, flat-bottomed wells.
Bio_RadPCR384	24×16	38.0	Round	—	self	any	Bio-Rad Hard-Shell 384-well PCR Plate (formerly MJ Research): thin-wall, skirted.
Bio_RadPCR96	12×8	186.0	Round	—	self	any	BioRad HardShell Low Profile Fully Skirted PCR Plate.
CorningCostar12	4×3	6900.0	Round	TiterplateLid	CorningCostarLid	any	Corning Costar Cell Culture Microplate, 12 wells.
CorningCostar24	6×4	3480.0	Round	TiterplateLid	CorningCostarLid	any	Corning Costar Cell Culture Microplate, 24 wells.
CorningCostar48	8×6	1620.0	Round	TiterplateLid	CorningCostarLid	any	Corning Costar Cell Culture Microplate, 48 wells.
CorningCostar6	3×2	16800.0	Round	TiterplateLid	CorningCostarLid	any	Corning Costar Cell Culture Microplate, 6 wells.
CostarFlat384Square	24×16	112.0	Rectangle	CostarFlat384SquareLid	self	any	Costar microplate with 384 square, flat-

Class name	Wells	Max vol (μL)	Well shape	Default lid	Stacks on	Pods	Description
							bottomed wells.
Eppendorf_twintec96_FullSkirt	12×8	220.0	Round	—	self	any	Eppendorf twin.tec PCR Plate 96, skirted.
Greiner384ConePP	24×16	130.0	Rectangle	Greiner384Lid	self	any	Greiner polypropylene microplate with 384 square, V-bottomed wells.
Greiner384ConePPDeep	24×16	240.0	Rectangle	Greiner384Lid	self	any	Greiner polypropylene microplate with 384 round, deep, V-bottomed wells.
Greiner96ConePS	12×8	230.0	Round	Greiner384Lid	self	any	Greiner polystyrene microplate with 96 round, V-bottomed wells.
Greiner96RoundDeep	12×8	1200.0	Round	—	—	any	Greiner polypropylene microplate with 96 deep, round, round-bottomed wells.
Greiner96RoundDeepSquare	12×8	2300.0	Rectangle	—	—	any	Greiner polypropylene microplate with 96 deep, square wells. The product is specified v-bottom, but the labware definer cannot express a v-bottom, so this class is defined with a round bottom (tested as acceptable).
Greiner96RoundPS	12×8	300.0	Round	Greiner384Lid	self	any	Greiner polystyrene

Class name	Wells	Max vol (µL)	Well shape	Default lid	Stacks on	Pods	Description
							microplate with 96 round, U-bottomed wells.
GreinerCELLSTAR12	4×3	6550.0	Round	TiterplateLid	GreinerCELLSTARLid	any	Greiner 12 Well Cell Culture Multiwell Plate.
GreinerCELLSTAR24	6×4	3300.0	Round	TiterplateLid	GreinerCELLSTARLid	any	Greiner 24 Well Cell Culture Multiwell Plate.
GreinerCELLSTAR48	8×6	1700.0	Round	TiterplateLid	GreinerCELLSTARLid	any	Greiner 48 Well Cell Culture Multiwell Plate.
GreinerCELLSTAR6	3×2	16400.0	Round	TiterplateLid	GreinerCELLSTARLid	any	Greiner 6 Well Cell Culture Multiwell Plate.
ILS_2mLDeepSquare	12×8	2200.0	Rectangle	—	self	any	Irish Life Science microplate with 96 deep, square, round-bottomed wells, 2.2 mL capacity. The external wall taper does not support an undergrip for moving stacks; when stacking, adjust the gripper offset to pick up above the skirt. Dead volume 10 µL per well.
BCDeep96Square	12×8	2200.0	Rectangle	—	—	Multichannel, Span-8	Beckman Coulter microplate with 96 deep, square, V-bottomed wells.

Class name	Wells	Max vol (μL)	Well shape	Default lid	Stacks on	Pods	Description
BCI_12_Strip	12×8	344.0	Round	titerplatelid	—	Multichannel, Span-8	Beckman Coulter microplate with 96 round, flat-bottomed wells.
GreinerFlat1536Square	48×32	12.0	Rectangle	GreinerFlat1536SquareLid	self	Multichannel	Greiner microplate with 1536 square, flat-bottomed wells.
WashStation	12×8	1000000.0	Round	—	—	Multichannel, Span-8	—
WashStation384	24×16	1000000.0	Round	—	—	Multichannel	—
WashStationSpan8	1×8	1000000.0	Round	—	—	Span-8	—

All titer plates carry `Can Pipette` and `Moveable` except `BCDeep96Square`, `WashStation`, `WashStation384`, and `WashStationSpan8`, which carry `Can Pipette` but lack `Moveable`. A class without `Moveable` cannot be repositioned by a `Move Labware` step; targeting one fails at `enqueue` with `Labware` of type `<class>` cannot be moved. All but the three wash stations also carry `Standard TiterPlate Size`; the wash stations carry `Do Not Report` instead, so they do not appear in reports. The `Stacks on` column lists the classes an instance may be placed on top of; `self` means another instance of its own class.

Tip Boxes

A tip box class is what a pipetting step's `tipLocation` names. Each default box holds exactly one tip class; the box name is the tip class name with `T` replaced by `BC`. Tip capacity, filtering, LLS capability and bore live in `Catalog: Tip Classes`.

The suffixes carry over from the tip class: the number is the tip's nominal capacity in μL, `_F` is filtered, `_LLS` is the LLS-capable variant, `_WB` is wide-bore, and `_384` is a 384-format box for a 384-channel head. See `Catalog: Tip Classes` for the full decoding and the per-tip properties.

All 24 default boxes are Beckman Coulter, use lid class `TipBoxLid`, have round wells that each hold one tip, carry the `Moveable` and `Standard TiterPlate Size` characteristics (a tip box does not carry `Can Pipette`), and are not stackable on other labware. Nine also carry `Do Not Report`: `BC190F`, `BC190F_LLS`, `BC190F_WB`, `BC230`, `BC230_WB`, `BC40F`, `BC50F`, `BC80`, and `BC90`.

Tip box class	Holds tip class	Tips	Pods
BC1025F	T1025F	96	any
BC1025F_LLS	T1025F_LLS	96	any
BC1025F_WB	T1025F_WB	96	any
BC1070	T1070	96	any
BC1070_LLS	T1070_LLS	96	any
BC1070_WB	T1070_WB	96	any
BC190F	T190F	96	any
BC190F_LLS	T190F_LLS	96	any
BC190F_WB	T190F_WB	96	any
BC230	T230	96	any

Tip box class	Holds tip class	Tips	Pods
BC230_LLS	T230_LLS	96	any
BC230_WB	T230_WB	96	any
BC40F	T40F	96	any
BC40F_LLS	T40F_LLS	96	any
BC50F	T50F	96	any
BC50F_LLS	T50F_LLS	96	any
BC80	T80	96	any
BC80_LLS	T80_LLS	96	any
BC90	T90	96	any
BC90_LLS	T90_LLS	96	any
BC25F_384	T25F_384	384	Multichannel
BC30_384	T30_384	384	Multichannel
BC40F_384	T40F_384	384	Multichannel
BC50_384	T50_384	384	Multichannel

Tube Racks

Tube racks come in two deck footprints. The `Matrix96_*` and `SmallTubeRack_*` classes occupy a standard microplate footprint, about 12.8 × 8.5 cm. The eight `BCTubeRack_*` and `BCSeptaTubeRack_*` classes are oversize at 13.716 × 26.67 cm, roughly three microplate footprints deep, and sit on the dedicated `TubeRack` ALP, which provides a single position that does not allow gripping.

Height matters too. Most microplates are under about 4.5 cm tall. The oversize racks run 7.66 to 11.587 cm, and three of the four `SmallTubeRack_*` classes are standard-footprint but tall — 8.0 cm for the 10 mm and 12 mm racks, 10.5 cm for the 13 mm rack. Only the two `Matrix96_*` racks, at 3.556 and 4.55 cm, and `SmallTubeRack_Microfuge`, at 3.95 cm, stay in plate territory. Those same two `Matrix96_*` classes are also the only ones of the fourteen that carry `Moveable`; for the rest a `Move` Labware step cannot reposition them.

All tube racks carry `Can Pipette`. The four `BCSeptaTubeRack_*` classes are septum-capped and require piercing tips. For every `BCTubeRack_*` and `BCSeptaTubeRack_*` class the well geometry assumes the inner diameter noted in the table, and 2 mm is subtracted from the nominal tube height for height and volume calculations. The four `SmallTubeRack_*` classes are 24-position racks built on BCI 373661 with class-specific inserts and tubes.

Class name	Tubes	Max vol/tube (µL)	Footprint	Height (cm)	Moveable	Pods	Notes
<code>Matrix96_1400uL</code>	12×8	1380.0	Standard plate	4.55	Yes	any	Thermo Scientific Matrix 1.4 mL 2D Barcoded Storage Tubes
<code>Matrix96_750uL</code>	12×8	750.0	Standard plate	3.556	Yes	any	Thermo Scientific Matrix 0.75 mL 2D barcoded storage tubes; height assumes septa in place, so remove septa when using non-piercing tips
<code>SmallTubeRack_Microfuge</code>	6×4	1700.0	Standard plate	3.95	No	any	Inserts BCI 373696, tubes BCI 356090
<code>SmallTubeRack_10mm</code>	6×4	4200.0	Standard plate	8.0	No	any	Inserts BCI 373699, tubes VWR 60825-538
<code>SmallTubeRack_12mm</code>	6×4	6200.0	Standard plate	8.0	No	any	Inserts BCI 373698, tubes VWR 60825-550

Class name	Tubes	Max vol/tube (µL)	Footprint	Height (cm)	Moveable	Pods	Notes
SmallTubeRack_13mm	6×4	10000.0	Standard plate	10.5	No	any	Inserts BCI 373697, tubes VWR 60825-571
BCTubeRack_10mm	10×16	3552.0	Oversize	7.66	No	Multichannel, Span-8	10 mm tubes, 75 mm; 8 mm assumed inner diameter
BCTubeRack_12mm	8×16	5523.0	Oversize	7.66	No	Multichannel, Span-8	12 mm tubes, 75 mm; 10 mm assumed inner diameter
BCTubeRack_13mm	8×12	9043.0	Oversize	10.16	No	Multichannel, Span-8	13 mm tubes, 100 mm; 11 mm assumed inner diameter
BCTubeRack_15_5mm	6×8	13562.0	Oversize	10.16	No	Multichannel, Span-8	15.5 mm tubes, 100 mm; 13.5 mm assumed inner diameter
BCSeptaTubeRack_13x75mm	8×12	6668.0	Oversize	9.087	No	Span-8	13 mm / 75 mm, 11 mm assumed inner diameter
BCSeptaTubeRack_13x100mm	8×12	9043.0	Oversize	11.587	No	Span-8	13 mm / 100 mm, 11 mm assumed inner diameter
BCSeptaTubeRack_15_5x75mm	6×8	9983.0	Oversize	9.087	No	Span-8	15.5 mm / 75 mm, 13.5 mm assumed inner diameter
BCSeptaTubeRack_15_5x100mm	6×8	13562.0	Oversize	11.587	No	Span-8	15.5 mm / 100 mm, 13.5 mm assumed inner diameter

Lids

All eight lid classes are 1×1 with `wellStyle: None` and zero well volume, and carry `Moveable` and `Standard TiterPlate Size`. All carry `Do Not Report` except `NuncLid`. `CostarFlat384SquareLid`, `Greiner384Lid`, and `GreinerFlat1536SquareLid` each name their own class as a stacking target, shown below as `self`. A blank `Stacks on` cell does not mean the lid is unusable: `TiterplateLid` and `TipBoxLid` are assigned through the plate's or box's default-lid setting rather than through a stacking list.

Class name	Stacks on	Pods	Description
CorningCostarLid	CorningCostar12 , CorningCostar24 , CorningCostar48 , CorningCostar6	any	Lid for Corning Costar cell culture plates (6-, 12-, 24-, 48-, 96-well)
CostarFlat384SquareLid	self	any	Lid from a Costar 384 microplate
Greiner384Lid	self	any	Lid from a Greiner 384 microplate
GreinerCELLSTARLid	GreinerCELLSTAR12 , GreinerCELLSTAR24 , GreinerCELLSTAR48 , GreinerCELLSTAR6	any	Lid for Greiner CELLSTAR cell culture plates (6-, 12-, 24-, 48-, 96-well)
NuncLid	—	any	Clear universal plate lid, no corners cut.
TipBoxLid	—	any	Lid from a Beckman Coulter tip box
TiterplateLid	—	any	Lid from a Beckman Coulter microplate
GreinerFlat1536SquareLid	self	Multichannel	Lid from a Greiner 1536 microplate

Reservations

Two reservation classes hold a deck position without providing pipettable wells. `SwapSpace` reserves a position for temporary storage during labware movement; `TipLoad` reserves a position for tip loading when tip boxes are provided from an external device. Both are 12×8 on a standard titer-plate footprint, work on any pod, and carry `Do Not Report`. Neither carries `Can Pipette` or `Moveable`.

Reservations are normally placed and managed by the software, not by a method author: you typically should **not** author a reservation class in an Instrument Setup `deckItems` list. Treat them as classes you may see in an export rather than ones you place by hand.

† [Table of contents](#)

Catalog: Labware Patterns

These are the well-selection patterns (**Well Patterns**) included in Beckman's default projects. Biomek has no standalone JSON export for project items today, but you do not need one to *use* a default pattern: a step references it by name (see **Using a pattern in a step** below). The `hardwareCompatibility` note is **pod-based** (Multichannel / Span-8 / Fixed-8); chassis size does not affect it, and a hybrid instrument supports the union of its pods. It is a catalog annotation derived from the three single-pod default projects: each pod tag is derived from the single-pod default project that uses that pod (`Fixed-8` from `Biomek_i3Project` , `Multichannel` from `Biomek_i5Project-MC` , `Span-8` from `Biomek_i5Project-Span`); a pattern's compatibility is the set of those pods whose project includes it, or `["any"]` when all three do. The `Biomek_i7Project` (Hybrid) is not used to add pod tags.

The default patterns

All four default patterns are **quadrant interleave selections** on a 24×16 (384-well) plate. Each one picks the 96 wells that a single 96-well plate maps to when four 96-well plates are condensed into one 384-well plate. They differ only in which quadrant they select:

Pattern	Selects (24 columns × 16 rows, 1-indexed)
UL_quad_384	Upper-left interleave — the 96 wells at odd rows × odd columns
UR_quad_384	Upper-right interleave — odd rows × even columns
LL_quad_384	Lower-left interleave — even rows × odd columns
LR_quad_384	Lower-right interleave — even rows × even columns

Hardware compatibility: all four are `["Multichannel", "Span-8"]`. They are included in the `Biomek_i5Project-MC` and `Biomek_i5Project-Span` single-pod default projects; the `Fixed-8` default project (`Biomek_i3Project`) carries no patterns. (The Hybrid `Biomek_i7Project` also carries them, but it is not used to add pod tags.)

Using a pattern in a step

Steps that take a well selection (for example `Transfer`) let you either embed a selection inline or reference one of these default patterns **by name**. To use a default pattern, set `localPattern` to `false` and name it in `referencedPattern` :

```
{
  "localPattern": false,
  "referencedPattern": "UL_quad_384"
}
```

With `localPattern` set to `false` , the step resolves the named pattern from the project's Well Patterns, so you do not spell out the 384-well mask yourself. (On export the editor also emits empty `pattern` and `selectionInfo` arrays alongside these keys for round-tripping; they are ignored when a pattern is referenced by name.) To supply your own selection instead, set `localPattern` to `true` and provide the mask inline — see the `Transfer` and `Define Pattern` step documentation for that form.

† Table of contents

Catalog: Liquid Types

Biomek includes two default liquid types, **Water** and **Serum**, but most customers define their own liquid types for their assays; open the **Liquid Type editor** in the Biomek software to view or edit the full per-liquid pipetting detail.

[↑ Table of contents](#)

Catalog: Techniques

This document describes only the default techniques that are included with Biomek Software. A given Biomek system likely has additional techniques developed by users which are preferred for use over the defaults. Users should provide the names of available techniques on their systems to LLMs for use in method authoring.

The names here are real and usable; only the JSON *format* is forward-looking. Biomek has **no JSON export for project items** today, so the payloads below are the planned/future shape of the technique-provider JSON — not authored files. But the catalog documents the **actual default pipetting techniques included with Biomek** — the named entries in the project's Technique Browser that a pipetting step selects via `prototype` or auto-select — and these are the valid technique names to reference. A specific project may add its own, so treat this as the default set, not an exhaustive list.

Unlike the other project-item catalogs, each technique here is shown in the **real serialized JSON shape Biomek produces for a technique** — the same `_biomekType: "technique"` payload that appears when a pipetting step embeds an inline `customPrototype`. `hardwareCompatibility` is **pod-based** (Multichannel / Span-8 / Fixed-8); chassis size does not affect it, and a hybrid instrument supports the union of its pods. Techniques are pod-family-specific by name convention (`F8 ...` / `MC ...` / `S8 ...`) and by their internal `context.pod` list, so no default technique reaches all three pods and `["any"]` never appears here.

Scope and derivation

- **Category:** Pipetting Techniques. Techniques are *not* one of the formal project-item categories the default projects carry alongside labware classes, labware patterns, liquid types, tip classes, and pipetting templates; they live in the project's technique library instead, reachable in the Biomek software via **Utilities > Technique Browser** and **Utilities > Technique Editor**, and are surfaced in method JSON either by name (`prototype`) or as an inline object (`customPrototype`) with `_biomekType: "technique"`.
- **Sources:** the four default projects (`Biomek_i3Project` , `Biomek_i5Project-MC` , `Biomek_i5Project-Span` , `Biomek_i7Project`) and the technique definitions each one installs into its Technique Browser.
- **Pod mapping used to derive `hardwareCompatibility`:** each pod tag comes from the single-pod project that owns that pod family — Fixed-8 from `Biomek_i3Project` , Multichannel from `Biomek_i5Project-MC` , Span-8 from `Biomek_i5Project-Span` . The `Biomek_i7Project` (Hybrid) project includes duplicates of the Multichannel and Span-8 techniques but is not used to add tags.
- All default technique names are pod-family-prefixed and the corresponding `context.pod` list confines each technique to that pod family, so every entry here resolves to a single pod (`["Fixed-8"]` , `["Multichannel"]` , or `["Span-8"]`).

Serialized JSON shape

Each technique object is emitted with:

- A `_biomekType: "technique"` discriminator at the top level.
- Nested plain-dictionary blocks — `aspirate` , `context` , `dispense` , `general` , `mix` — that carry no `_biomekType` (plain dictionaries are the "empty type" and elide the tag to keep the JSON concise, as documented in `Method-JSON-Structure.md`).
- Top-level scalars `name` , `readOnly` , and `template` (the read-only name of the pipetting template this technique binds to).
- Keys are camelCased on write and compared case-insensitively on read. `C__Blowout` in the source object surfaces as `c__Blowout` in JSON; `Aspirate` becomes `aspirate` ; `Name` becomes `name` . Keys are sorted alphabetically (case-insensitive) within each block. Some default techniques carry fully lowercased key names (for example `minimumvolume` / `maximumvolume`) inherited from their source declarations while others use `minimumVolume` / `maximumVolume` ; because keys are compared case-insensitively on read, the printed variation is cosmetic, not a typo.
- `context.*` fields carry the auto-select match criteria the technique is scoped to — `pod` , and any of `head` , `syringeType` , `tips` , `labware` , `minimumVolume` , `maximumVolume` , `rank` , `autoSelection` . Each is a JSON array (a variant list) even when it contains a single value.

The `customPrototype` payload embedded on a Multichannel Aspirate step in a real exported method — see the JSON snippet on the Multichannel Aspirate step page — is a direct example of this shape.

Items

Full technique payloads — one exemplar per pod family

These three entries are shown in full. Every other default technique in the section below uses the same top-level shape as the exemplar for its pod family; the exact `general` -block key set differs between Fixed-8 and Multichannel/Span-8 (see the note under the `MC P60` example). The compact entries omit the `aspirate` / `dispense` / `mix` / `general` blocks for space and list only the identifying `context` , `name` , `template` , and `readOnly` fields — which capture the per-technique auto-select scope and template pairing.

F8 Medium — Fixed-8 default pipetting technique

hardwareCompatibility: ["Fixed-8"]

```
{
  "_biomekType": "technique",
  "aspirate": {
    "c_Blowout": true,
    "c_FollowLiquid": true,
    "c_Height": -2.0,
    "c_HeightFrom": 0,
    "c_Mix": false,
    "c_MixAspirateFrom": 0,
    "c_MixAspirateHeight": 0.0,
    "c_MixAspirateSpeed": 100.0,
    "c_MixCount": 1.0,
    "c_MixDispenseFrom": 0,
    "c_MixDispenseHeight": 0.0,
    "c_MixDispenseSpeed": 100.0,
    "c_MixVolume": 10.0,
    "c_Speed": 100.0,
    "c_TipTouch": true,
    "c_TrailingAirGap": false
  },
  "context": {
    "maximumVolume": [230.0],
    "minimumVolume": [10.0],
    "pod": ["Fixed8"]
  },
  "dispense": {
    "c_Blowout": true,
    "c_FollowLiquid": true,
    "c_Height": 0.0,
    "c_HeightFrom": 0,
    "c_Mix": false,
    "c_MixAspirateFrom": 0,
    "c_MixAspirateHeight": 0.0,
    "c_MixAspirateSpeed": 100.0,
    "c_MixCount": 1.0,
    "c_MixDispenseFrom": 0,
    "c_MixDispenseHeight": 0.0,
    "c_MixDispenseSpeed": 100.0,
    "c_MixVolume": 10.0,
    "c_Speed": 100.0,
    "c_TipTouch": false
  },
  "general": {
    "c_aspiratedelay": 0.0,
    "c_aspirespeed": 50.0,
    "c_blowoutdelay": 0.0,
    "c_blowoutvolume": 15.0,
    "c_CalibrationOffset": 2.0,
    "c_CalibrationSlope": 1.03,
    "c_CDConductivity": "1",
    "c_CDConductivityInherit": 2,
    "c_CDErrorAutoRecover": 2,
    "c_CDHeight": "0",
    "c_CDHeightFrom": 0,
    "c_CDRepeat": "1",
    "c_CDSpeed": 0.0,
    "c_ConditioningExcess": 0.0,
    "c_ConditioningSectionCount": 0.0,
    "c_ConditioningSectionVolume": 0.0,
    "c_Conductivity": "3500",
    "c_ConductivityInherit": 2,
    "c_cutoffvelocity": 150.0,
    "c_DetectionSpeed": 20.0,
    "c_dispensedelay": 150.0,
    "c_dispensespeed": 25.0,
    "c_EmptyTips": false,
    "c_LLSAccept": false,
  }
}
```

```

    "c__LLSAcceptValue": "0",
    "c__llserrorautorecover": 2,
    "c__llserrorautoretry": "0",
    "c__LLSFail": false,
    "c__LLSFailValue": "0",
    "c__LLSInitial": true,
    "c__LLSInitialHeight": "0",
    "c__LLSInitialHeightFrom": 2,
    "c__LLSRepeat": "1",
    "c__MinimumHeight": 0.2,
    "c__overrideliqidtypesettings": true,
    "c__PiercedSpeedLimit": 20.0,
    "c__PiercingExitHeight": 2.0,
    "c__PiercingExitSpeed": 20.0,
    "c__PiercingFinalHeight": -5.0,
    "c__PiercingFinalSpeed": 20.0,
    "c__PiercingInitialHeight": 2.0,
    "c__Prewet": false,
    "c__prewetdelay": 0.0,
    "c__prewetoverage": 0.0,
    "c__tiptouchdelay": 0.0,
    "c__tiptouchheight": -1.0,
    "c__tiptouchheightfrom": 2.0,
    "c__tiptouchspeed": 100.0,
    "c__tiptouchtheta": 90.0,
    "c__trailingairgapvolume": 0.0
  },
  "mix": {
    "c__Blowout": true,
    "c__FollowLiquid": false,
    "c__Mix": true,
    "c__MixAspirateFrom": 1,
    "c__MixAspirateHeight": 2.0,
    "c__MixAspirateSpeed": 100.0,
    "c__MixDispenseFrom": 1,
    "c__MixDispenseHeight": 2.0,
    "c__MixDispenseSpeed": 100.0,
    "c__Speed": 100.0,
    "c__TipTouch": true
  },
  "name": "F8 Medium",
  "readOnly": false,
  "template": "F8 Pipetting"
}

```

MC P60 — Multichannel default, 384-head

hardwareCompatibility: ["Multichannel"]

```

{
  "_biomekType": "technique",
  "aspirate": {
    "c__Blowout": true,
    "c__FollowLiquid": true,
    "c__Height": -2.0,
    "c__HeightFrom": 0,
    "c__Mix": false,
    "c__MixAspirateFrom": 1,
    "c__MixAspirateHeight": 0.0,
    "c__MixAspirateSpeed": 100.0,
    "c__MixCount": 1.0,
    "c__MixDispenseFrom": 1,
    "c__MixDispenseHeight": 0.0,
    "c__MixDispenseSpeed": 100.0,
    "c__MixVolume": 10.0,
    "c__Speed": 10.0,
    "c__TipTouch": false,
    "c__TrailingAirGap": false
  },
  "context": {
    "head": ["MC384_60"],
    "pod": ["Pod96"],
    "rank": [50]
  },
  "dispense": {
    "c__Blowout": true,
    "c__FollowLiquid": true,
    "c__Height": -2.0,
    "c__HeightFrom": 0,
    "c__Mix": false,
    "c__MixAspirateFrom": 1,
    "c__MixAspirateHeight": 2.0,
    "c__MixAspirateSpeed": 100.0,

```

```

    "c_MixCount": 4.0,
    "c_MixDispenseFrom": 1,
    "c_MixDispenseHeight": 5.0,
    "c_MixDispenseSpeed": 100.0,
    "c_MixVolume": 20.0,
    "c_Speed": 10.0,
    "c_TipTouch": true
  },
  "general": {
    "c_CalibrationOffset": 0.0,
    "c_CalibrationSlope": 1.0,
    "c_CDConductivity": "1",
    "c_CDConductivityInherit": 2,
    "c_CDErrorAutoRecover": 2,
    "c_CDHeight": "0",
    "c_CDHeightFrom": 0,
    "c_CDRepeat": "1",
    "c_CDSpeed": 30.0,
    "c_ConditioningExcess": 0.0,
    "c_ConditioningSectionCount": 0.0,
    "c_ConditioningSectionVolume": 0.0,
    "c_Conductivity": "2500",
    "c_ConductivityInherit": 0,
    "c_DetectionSpeed": 100.0,
    "c_EmptyTips": false,
    "c_LLSAccept": false,
    "c_LLSAcceptValue": "0",
    "c_llserrorautorecover": 2,
    "c_llserrorautoretry": "0",
    "c_LLSFail": false,
    "c_LLSFailValue": "0",
    "c_LLSInitial": true,
    "c_LLSInitialHeight": "0",
    "c_LLSInitialHeightFrom": 0,
    "c_LLSRepeat": "1",
    "c_MinimumHeight": 0.2,
    "c_overrideliqidtypesettings": false,
    "c_PiercedSpeedLimit": 20.0,
    "c_PiercingExitHeight": 2.0,
    "c_PiercingExitSpeed": 20.0,
    "c_PiercingFinalHeight": -5.0,
    "c_PiercingFinalSpeed": 20.0,
    "c_PiercingInitialHeight": 2.0,
    "c_Prewet": false
  },
  "mix": {
    "c_Blowout": true,
    "c_FollowLiquid": false,
    "c_Mix": true,
    "c_MixAspirateFrom": 1,
    "c_MixAspirateHeight": 3.0,
    "c_MixAspirateSpeed": 100.0,
    "c_MixDispenseFrom": 1,
    "c_MixDispenseHeight": 3.0,
    "c_MixDispenseSpeed": 100.0,
    "c_Speed": 10.0,
    "c_TipTouch": true
  },
  "name": "MC P60",
  "readOnly": false,
  "template": "MC Pipetting"
}

```

Note: the Multichannel and Span-8 `general` blocks do not carry the free-standing `c__aspiratedelay`, `c__aspiratespeed`, `c__blowoutdelay`, `c__blowoutvolume`, `c__cutoffvelocity`, `c__dispensedelay`, `c__dispensespeed`, `c__prewetdelay`, `c__prewetoverage`, `c__tiptouchdelay`, `c__tiptouchheight`, `c__tiptouchheightfrom`, `c__tiptouchspeed`, `c__tiptouchtheta`, or `c__trailingairgapvolume` keys that Fixed-8 techniques do — those parameters are supplied by liquid-type context when `c__overrideliqidtypesettings` is `false`.

S8 1000 Medium — Span-8 default, 1000 syringes, mid volume

hardwareCompatibility: ["Span-8"]

```

{
  "_biomekType": "technique",
  "aspirate": {
    "c_Blowout": true,

```

```

    "c_FollowLiquid": true,
    "c_Height": -2.0,
    "c_HeightFrom": 0,
    "c_Mix": false,
    "c_MixAspirateFrom": 0,
    "c_MixAspirateHeight": 0.0,
    "c_MixAspirateSpeed": 100.0,
    "c_MixCount": 1.0,
    "c_MixDispenseFrom": 0,
    "c_MixDispenseHeight": 0.0,
    "c_MixDispenseSpeed": 100.0,
    "c_MixVolume": 10.0,
    "c_Speed": 5.0,
    "c_TipTouch": false,
    "c_TrailingAirGap": true
  },
  "context": {
    "minimumVolume": [25.0],
    "pod": ["Pod8Span"],
    "syringeType": ["SYR1000", "SYR2500", "SYR5000"]
  },
  "dispense": {
    "c_Blowout": true,
    "c_FollowLiquid": false,
    "c_Height": 1.5,
    "c_HeightFrom": 1,
    "c_Mix": false,
    "c_MixAspirateFrom": 0,
    "c_MixAspirateHeight": 0.0,
    "c_MixAspirateSpeed": 100.0,
    "c_MixCount": 1.0,
    "c_MixDispenseFrom": 0,
    "c_MixDispenseHeight": 0.0,
    "c_MixDispenseSpeed": 100.0,
    "c_MixVolume": 10.0,
    "c_Speed": 5.0,
    "c_TipTouch": false
  },
  "general": {
    "c_CalibrationOffset": 0.0,
    "c_CalibrationSlope": 1.0,
    "c_CDConductivity": "1",
    "c_CDConductivityInherit": 2,
    "c_CDErrorAutoRecover": 2,
    "c_CDHeight": "0",
    "c_CDHeightFrom": 0,
    "c_CDRepeat": "1",
    "c_CDSpeed": 30.0,
    "c_ConditioningExcess": 0.0,
    "c_ConditioningSectionCount": 0.0,
    "c_ConditioningSectionVolume": 0.0,
    "c_Conductivity": "3500",
    "c_ConductivityInherit": 2,
    "c_DetectionSpeed": 20.0,
    "c_EmptyTips": false,
    "c_LLSAccept": false,
    "c_LLSAcceptValue": "0",
    "c_llserrorautorecover": 2,
    "c_llserrorautoretry": "1",
    "c_LLSFail": false,
    "c_LLSFailValue": "0",
    "c_LLSInitial": true,
    "c_LLSInitialHeight": "0",
    "c_LLSInitialHeightFrom": 2,
    "c_LLSRepeat": "1",
    "c_MinimumHeight": 1.5,
    "c_overrideLiquidTypesettings": false,
    "c_PiercedSpeedLimit": 20.0,
    "c_PiercingExitHeight": 2.0,
    "c_PiercingExitSpeed": 20.0,
    "c_PiercingFinalHeight": -5.0,
    "c_PiercingFinalSpeed": 20.0,
    "c_PiercingInitialHeight": 2.0,
    "c_Prewet": false
  },
  "mix": {
    "c_Blowout": true,
    "c_FollowLiquid": false,
    "c_Mix": true,
    "c_MixAspirateFrom": 1,
    "c_MixAspirateHeight": 3.0,
    "c_MixAspirateSpeed": 100.0,
    "c_MixDispenseFrom": 1,
    "c_MixDispenseHeight": 3.0,
    "c_MixDispenseSpeed": 100.0,
    "c_Speed": 5.0,

```

```

    "c_TipTouch": false
  },
  "name": "S8 1000 Medium",
  "readOnly": false,
  "template": "S8 Pipetting"
}

```

Compact entries — remaining defaults

Each entry below shows the identifying top-level fields (`_biomekType` , `context` , `name` , `readOnly` , `template`) — the auto-select scope and template binding. The `aspirate` , `dispense` , `mix` , and `general` blocks are omitted for space; they follow the same key set as the pod-family exemplar above and carry values tuned to the technique (open the technique in the Technique Editor to see the resolved values on the General / Aspirate / Dispense / Mix / Calibration / Liquid Level Sensing / Clot Detection / Piercing tabs).

Fixed-8 techniques (`hardwareCompatibility`: ["Fixed-8"])

All Fixed-8 techniques are included in `Biomek_i3Project` only.

F8 High

```

{
  "_biomekType": "technique",
  "context": {
    "minimumVolume": [100.0],
    "pod": ["Fixed8"]
  },
  "name": "F8 High",
  "readOnly": false,
  "template": "F8 Pipetting"
}

```

F8 Low

```

{
  "_biomekType": "technique",
  "context": {
    "maximumVolume": [90.0],
    "pod": ["Fixed8"]
  },
  "name": "F8 Low",
  "readOnly": false,
  "template": "F8 Pipetting"
}

```

F8 Medium TipDipDispense

```

{
  "_biomekType": "technique",
  "context": {
    "maximumVolume": [230.0],
    "minimumVolume": [10.0],
    "pod": ["Fixed8"]
  },
  "name": "F8 Medium TipDipDispense",
  "readOnly": false,
  "template": "F8 Pipetting TipDipDispense"
}

```

F8 Medium TopDispense

```

{
  "_biomekType": "technique",
  "context": {
    "maximumVolume": [230.0],
    "minimumVolume": [10.0],
    "pod": ["Fixed8"]
  },
  "name": "F8 Medium TopDispense",
  "readOnly": false,

```

```
"template": "F8 Pipetting"
}
```

F8 MultiDispense

```
{
  "_biomekType": "technique",
  "context": {
    "autoSelection": ["DoNotAutoSelect"],
    "pod": ["Fixed8"],
    "rank": [50]
  },
  "name": "F8 MultiDispense",
  "readOnly": false,
  "template": "F8 Pipetting"
}
```

Fixed-8 pipetting steps (Aspirate/Dispense/Mix) never auto-select a technique — the step code throws if `autoSelectPrototype` is `true`, and otherwise requires an explicit `prototype` (or `customPrototype`). This is enforced in the step itself, independent of any technique's `DoNotAutoSelect` /auto-selection flag (those only affect the shared auto-select resolver used by the other pod families).

Multichannel techniques (`hardwareCompatibility: ["Multichannel"]`)

All Multichannel techniques are included in `Biomek_i5Project-MC` and `Biomek_i7Project`.

MC

```
{
  "_biomekType": "technique",
  "context": {
    "head": ["MC96_1200", "MC96_300"],
    "pod": ["Pod96"],
    "rank": [50]
  },
  "name": "MC",
  "readOnly": false,
  "template": "MC Pipetting"
}
```

MC Active Wash

```
{
  "_biomekType": "technique",
  "context": {
    "labware": ["WashStation", "WashStation384"],
    "pod": ["Pod96"],
    "rank": [25]
  },
  "name": "MC Active Wash",
  "readOnly": false,
  "template": "MC Active Washing"
}
```

MC Low

```
{
  "_biomekType": "technique",
  "context": {
    "head": ["MC96_1200", "MC96_300"],
    "maximumVolume": [5.0],
    "pod": ["Pod96"],
    "rank": [25]
  },
  "name": "MC Low",
  "readOnly": false,
  "template": "MC Low-Volume Pipetting"
}
```

MC MultiDispense

```
{
  "_biomekType": "technique",
  "context": {
    "autoSelection": ["DoNotAutoSelect"],
    "pod": ["Pod96"],
    "rank": [50]
  },
  "name": "MC MultiDispense",
  "readOnly": false,
  "template": "MC Pipetting"
}
```

MC P60 Low

```
{
  "_biomekType": "technique",
  "context": {
    "head": ["MC384_60"],
    "maximumVolume": [5.0],
    "pod": ["Pod96"],
    "rank": [25]
  },
  "name": "MC P60 Low",
  "readOnly": false,
  "template": "MC Low-Volume Pipetting"
}
```

MC P300 High

```
{
  "_biomekType": "technique",
  "context": {
    "head": ["MC96_300"],
    "minimumVolume": [50.0],
    "pod": ["Pod96"],
    "rank": [25]
  },
  "name": "MC P300 High",
  "readOnly": false,
  "template": "MC Pipetting"
}
```

MC P1200 High

```
{
  "_biomekType": "technique",
  "context": {
    "head": ["MC96_1200"],
    "minimumVolume": [150.0],
    "pod": ["Pod96"],
    "rank": [25]
  },
  "name": "MC P1200 High",
  "readOnly": false,
  "template": "MC Pipetting"
}
```

The Multichannel head identifiers describe which multichannel head is installed on the pod: MC96_1200 and MC96_300 are the 96-format 1200 uL and 300 uL heads; MC384_60 is the 384-format 60 uL head. Auto-select uses head to scope which volume-range techniques apply on a given assembly.

Span-8 techniques (hardwareCompatibility: ["Span-8"])

All Span-8 techniques are included in Biomek_i5Project-Span and Biomek_i7Project .

```
{
  "_biomekType": "technique",
  "context": {
    "minimumVolume": [5.0],
    "pod": ["Pod8Span"],
    "syringeType": ["SYR250", "SYR500"]
  },
  "name": "S8 250",
  "readOnly": false,
  "template": "S8 Pipetting"
}
```

S8 250 Low

```
{
  "_biomekType": "technique",
  "context": {
    "maximumVolume": [5.0],
    "pod": ["Pod8Span"],
    "syringeType": ["SYR250", "SYR500"]
  },
  "name": "S8 250 Low",
  "readOnly": false,
  "template": "S8 Pipetting"
}
```

S8 1000 Low

```
{
  "_biomekType": "technique",
  "context": {
    "maximumVolume": [25.0],
    "pod": ["Pod8Span"],
    "syringeType": ["SYR1000", "SYR2500", "SYR5000"]
  },
  "name": "S8 1000 Low",
  "readOnly": false,
  "template": "S8 Pipetting"
}
```

S8 1000 High

```
{
  "_biomekType": "technique",
  "context": {
    "minimumVolume": [500.0],
    "pod": ["Pod8Span"],
    "syringeType": ["SYR1000", "SYR2500", "SYR5000"]
  },
  "name": "S8 1000 High",
  "readOnly": false,
  "template": "S8 Pipetting"
}
```

S8 Active Wash

```
{
  "_biomekType": "technique",
  "context": {
    "labware": ["WashStation", "WashStation384", "WashStationSpan8"],
    "pod": ["Pod8Span"]
  },
  "name": "S8 Active Wash",
  "readOnly": false,
  "template": "S8 Active Washing"
}
```

S8 Clot Detection

```
{
  "_biomekType": "technique",
```

```

"context": {
  "autoSelection": ["DoNotAutoSelect"],
  "pod": ["Pod8Span"],
  "tips": [
    "Fixed100",
    "SeptaFluted",
    "T1025F_LLS",
    "T1070_LLS",
    "T190F_LLS",
    "T230_LLS",
    "T40F_LLS",
    "T50F_LLS",
    "T80_LLS",
    "T90_LLS"
  ]
},
"name": "S8 Clot Detection",
"readOnly": false,
"template": "S8 Clot Detection"
}

```

S8 MultiDispense

```

{
  "_biomekType": "technique",
  "context": {
    "autoSelection": ["DoNotAutoSelect"],
    "pod": ["Pod8Span"]
  },
  "name": "S8 MultiDispense",
  "readOnly": false,
  "template": "S8 Pipetting"
}

```

S8 SeptaFluted

```

{
  "_biomekType": "technique",
  "context": {
    "labware": [
      "BCSeptaTubeRack_13x100mm",
      "BCSeptaTubeRack_13x75mm",
      "BCSeptaTubeRack_15_5x100mm",
      "BCSeptaTubeRack_15_5x75mm"
    ],
    "pod": ["Pod8Span"],
    "tips": ["SeptaFluted"]
  },
  "name": "S8 SeptaFluted",
  "readOnly": false,
  "template": "S8 Septa Piercing"
}

```

Span-8 syringe identifiers (SYR250 , SYR500 , SYR1000 , SYR2500 , SYR5000) name the installed syringe size on the Span-8 pod's 8 channels. Auto-select uses `syringeType` to scope which pipetting-volume techniques apply on a given assembly.

Catalog: Pipetting Templates

This document describes only the default templates that are included with Biomek Software. A given Biomek system likely has additional templates developed by users which are preferred for use over the defaults. Users should provide the names of available templates on their systems to LLMs for use in method editing.

The names here are real and usable. Templates live inside the project's template library (installed alongside techniques) rather than in any exported file, so this catalog documents the **default pipetting templates included with Biomek** as prose — and these are the valid template names to reference. A specific project may add its own, so treat this as the default set, not an exhaustive list.

`hardwareCompatibility` is **pod-based** (Multichannel / Span-8 / Fixed-8); chassis size does not affect it, and a hybrid instrument supports the union of its pods. Every default template is pod-family-specific by name convention (`F8 ...` / `MC ...` / `S8 ...`) and by the templates its bound techniques carry, so no default template reaches all three pods and `["any"]` never appears here.

Scope and derivation

- **Category:** Pipetting Templates. There is no project-item export today, so templates are not written to any file — they live only inside the project (installed alongside techniques) and are edited in the Biomek software via **Utilities > Pipetting Template Editor**. Method JSON references a template only indirectly, through the `template` field of the technique that binds it.
- **Sources:** the four default projects (`Biomek_i3Project` , `Biomek_i5Project-MC` , `Biomek_i5Project-Span` , `Biomek_i7Project`) and the pipetting-template definitions each one installs into its Pipetting Template Editor.
- **Pod mapping used to derive `hardwareCompatibility`:** each pod tag comes from the single-pod project that owns that pod family — Fixed-8 from `Biomek_i3Project` , Multichannel from `Biomek_i5Project-MC` , Span-8 from `Biomek_i5Project-Span` . The `Biomek_i7Project` (Hybrid) project includes duplicates of the Multichannel and Span-8 templates but is not used to add tags.
- All default template names are pod-family-prefixed and the corresponding techniques bind them within that pod family, so every entry here resolves to a single pod (`["Fixed-8"]` , `["Multichannel"]` , or `["Span-8"]`).
- **Technique compatibility:** each template lists the default techniques whose `template` field points at it. See `catalog-techniques.md` for the technique payloads (each entry includes a `template` field that names the template it binds).

Method-JSON reach

Method JSON references techniques, not templates. No pipetting step, and no `_biomekType` value, refers to a template — you cannot author, edit, or select a template from method JSON, and you cannot change the template a technique binds from method JSON either (the technique's `template` field is a read-only property of the technique). The runtime path is:

1. The pipetting step names a technique (via `prototype` , or auto-select, or an inline `customPrototype` payload).
2. The technique's `template` field names the pipetting template it binds.
3. The template's ordered sequence of pipetting actions executes, reading its parameter values from `C__` context variables the step and technique populate at run time.

To change the physical procedure, edit the template in the Pipetting Template Editor, or pick a technique whose template already does what you need. See `Pipetting-Techniques-and-Templates.md` for the full technique/template model, the `C__` variable list, and the auto-select algorithm.

Items

Fixed-8 templates (`hardwareCompatibility: ["Fixed-8"]`)

All Fixed-8 pipetting templates are included in `Biomek_i3Project` only.

F8 Pipetting

- **What it does:** the general-purpose Fixed-8 pipetting recipe — the aspirate/dispense/mix operation tree used by the great majority of Fixed-8 techniques. Each operation begins with an axes move to the pipetting position, then executes an air-gap, a mix, an aspirate/dispense, a trailing air-gap, and an optional tip-touch — all parameterized on `C__` variables so the bound technique's General/Aspirate/Dispense/Mix tab values drive the actual motion.
- **Technique compatibility:** bound by `F8 High`, `F8 Low`, `F8 Medium`, `F8 Medium TopDispense`, and `F8 MultiDispense`.
- **Where to see the full recipe:** open `F8 Pipetting` in **Utilities > Pipetting Template Editor** in the Biomek software; that view is authoritative for the action sequence, variable bindings, and any conditional guards.

F8 Pipetting TipDipDispense

- **What it does:** a Fixed-8 pipetting variant that adds a post-blowout "tip dip" — a short dispense-only move to 0 mm from the liquid, guarded on the labware having a known/nominal liquid volume — to knock residual droplets off the tip on the way out. Otherwise mirrors `F8 Pipetting`.
- **Technique compatibility:** bound by `F8 Medium TipDipDispense`.
- **Where to see the full recipe:** open `F8 Pipetting TipDipDispense` in **Utilities > Pipetting Template Editor** in the Biomek software.

Multichannel templates (`hardwareCompatibility: ["Multichannel"]`)

All Multichannel pipetting templates are included in `Biomek_i5Project-MC` and `Biomek_i7Project`.

MC Pipetting

- **What it does:** the general-purpose Multichannel pipetting recipe — the aspirate, dispense, and mix operation tree that carries the great majority of 96- and 384-format multichannel work. All motion parameters read from `C__` variables so the bound technique's tab values drive the actual behavior.
- **Technique compatibility:** bound by `MC`, `MC MultiDispense`, `MC P60`, `MC P300 High`, and `MC P1200 High`.
- **Where to see the full recipe:** open `MC Pipetting` in **Utilities > Pipetting Template Editor** in the Biomek software.

MC Low-Volume Pipetting

- **What it does:** the Multichannel low-volume pipetting variant, tuned for reliable aspirate/dispense at very small volumes (single-digit uL). Uses a hard-coded blowout speed rather than reading `C__AspirateSpeed` for the post-aspirate air gap, to keep ejection dynamics predictable near the head's low-volume floor. Otherwise mirrors the `MC Pipetting` structure.
- **Technique compatibility:** bound by `MC Low` and `MC P60 Low`.
- **Where to see the full recipe:** open `MC Low-Volume Pipetting` in **Utilities > Pipetting Template Editor** in the Biomek software.

MC Active Washing

- **What it does:** the Multichannel wash-station recipe. Aspirate is blocked — aspirating from a wash station is not supported — and dispense targets the wash-station waste area rather than a well, so mix and tip-touch settings on the technique's Dispense tab are disregarded by design.
- **Technique compatibility:** bound by `MC Active Wash`.
- **Where to see the full recipe:** open `MC Active Washing` in **Utilities > Pipetting Template Editor** in the Biomek software.

Span-8 templates (`hardwareCompatibility: ["Span-8"]`)

All Span-8 pipetting templates are included in `Biomek_i5Project-Span` and `Biomek_i7Project`.

S8 Pipetting

- **What it does:** the general-purpose Span-8 pipetting recipe — the aspirate, dispense, and mix operation tree that drives most Span-8 work with disposable or fixed tips. All motion parameters read from `C__` variables so the bound technique's tab values drive the actual behavior.
- **Technique compatibility:** bound by `S8 250`, `S8 250 Low`, `S8 1000 Low`, `S8 1000 Medium`, `S8 1000 High`, and `S8 MultiDispense`.
- **Where to see the full recipe:** open `S8 Pipetting` in **Utilities > Pipetting Template Editor** in the Biomek software.

S8 Active Washing

- **What it does:** the Span-8 wash-station recipe. Aspirate is blocked — aspirating from a wash station is not supported — and dispense targets the wash-station waste area rather than a well, so mix and tip-touch settings on the technique's Dispense tab are disregarded by design.
- **Technique compatibility:** bound by `S8 Active Wash`.
- **Where to see the full recipe:** open `S8 Active Washing` in **Utilities > Pipetting Template Editor** in the Biomek software.

S8 Clot Detection

- **What it does:** the Span-8 clot-detection recipe — an `S8 Pipetting`-style tree augmented to run the clot-detection routine during aspirate, using the technique's Clot Detection tab settings (`C__CDConductivity` , `C__CDHeight` , `C__CDRepeat` , `C__CDSpeed` , `C__CDErrorAutoRecover` , and related keys). Only meaningful with an LLS-capable tip, and typically paired with the tip list that `S8 Clot Detection` scopes.
- **Technique compatibility:** bound by `S8 Clot Detection`.
- **Where to see the full recipe:** open `S8 Clot Detection` in **Utilities > Pipetting Template Editor** in the Biomek software.

S8 Septa Piercing

- **What it does:** the Span-8 septum-piercing recipe used with fluted septum-piercing tips on septum-capped tube racks. Tip touch is disabled by the template because no lateral movement is allowed while a tip is inside a septum tube; the mix heights on the Aspirate and Dispense tabs are also disregarded by the template for the same reason; and the template does not follow the liquid level, to avoid extra motions inside the tube.
- **Technique compatibility:** bound by `S8 SeptaFluted`.
- **Where to see the full recipe:** open `S8 Septa Piercing` in **Utilities > Pipetting Template Editor** in the Biomek software.

Note on user-defined templates

Users can create their own pipetting templates in **Utilities > Pipetting Template Editor** alongside the defaults listed above. This catalog covers only the Beckman defaults; the installed template set on a specific instrument may include additional user-defined templates that are outside the scope of this reference.

Catalog: Tip Classes

The names here are real and usable. These entries are the **default project items included with Biomek**: a method references them **by name**, and these are the valid names to use. Project items are not a file you author or import. A specific project may add its own, so treat this as the default set, not an exhaustive list.

Every unique Beckman default tip class is listed once, deduplicated across the three single-pod default projects. The **Pods** column records `hardwareCompatibility`, which is **pod-based** (Multichannel / Span-8 / Fixed-8); chassis size does not affect it, and a hybrid instrument supports the union of its pods. The field is tentative: it is a catalog annotation derived from which default projects include the item (mapped to their pods), not necessarily an intrinsic field of the forthcoming format. A Pods value of `any` means all three default projects include the class. **It is a membership hint, not a capability guarantee** — it does not promise a given pod can reach the position you place the labware at.

Note on LLS. The per-tip-class `LLS` boolean records whether the tip itself is LLS-capable. **Liquid-level sensing is only supported by the Span-8 pod**; Fixed-8 and Multichannel pods do not perform LLS regardless of the tip's `LLS` value. An LLS-capable tip class showing Pods `any` means the tip class is included in all three single-pod default projects — not that LLS itself works across all pods.

Operational consequence. A non-LLS tip class (`LLS: false`, e.g. `T190F`, `T50F`) cannot be paired with an LLS-enabled Span-8 technique — the stock auto-selected Span-8 techniques set `c__LLSInitial: true`, so with non-LLS tips the enqueue fails with `Cannot liquid level sense with tip type <class>`. To fix, either pick an LLS-capable tip class (`_LLS` variant), select a technique whose `c__LLSInitial` is `false`, or override height on the pipetting step (`overrideHeight: true` with `heightFrom` set to `Bottom` or `Top`).

Scope and derivation

- **Category:** Tip Classes (`Tip Classes`).
- **Sources:** the three single-pod default projects (`Biomek_i3Project`, `Biomek_i5Project-MC`, `Biomek_i5Project-Span`).
- **Pod mapping used to derive `hardwareCompatibility`**: each pod tag is derived from the single-pod default project that uses that pod (`Fixed-8` from `Biomek_i3Project`, `Multichannel` from `Biomek_i5Project-MC`, `Span-8` from `Biomek_i5Project-Span`); a tip's compatibility is the set of those pods whose project includes it, or `["any"]` when all three do. The `Biomek_i7Project` (Hybrid) is not used to add pod tags — it carries both pods, so membership there cannot distinguish which pod a tip is for. Excluding it costs no coverage: its 26 tip classes are exactly the 26 in the union of the three single-pod projects.

Naming scheme

Disposable tip class names decode as `T<capacity>[F][_LLS][_WB][_384]` :

Element	Meaning
<code>T<number></code>	Nominal capacity in μL . <code>T190F</code> holds 190 μL ; <code>T1070</code> holds 1070 μL . Capacity is inclusive of liquid and trailing air gap.
<code>F</code>	Filtered (aerosol-resistant). Present exactly when <code>Filtered: true</code> .
<code>_LLS</code>	LLS-capable variant. Present exactly when <code>LLS: true</code> on a disposable tip.
<code>_WB</code>	Wide-bore variant, intended for viscous liquids and cell suspensions.
<code>_384</code>	384-format tip for a 384-channel head on a Multichannel pod.

Consequences of the scheme when picking a class by name:

- Filtered and unfiltered variants at the same tip length are separate classes with different capacities. `T190F` (190 μL) and `T230` (230 μL) share a tip height of 5.1435 cm; `T40F` / `T80`, `T50F` / `T90`, and `T1025F` / `T1070` pair the same way at their own lengths.
- A `_WB` class carries exactly the same published property values as its base class — capacity, filtering, LLS, tip height and air capacity are all identical. The difference is the orifice: a `_WB` tip narrows to a 0.0991 cm radius against 0.0635 cm on its base class. That geometry is not one of the properties below, so the two are told apart by name alone.

- `_LLS` and `_WB` variants exist only for 96-format tips. The four `_384` classes have neither.
- The two fixed Span-8 tips, `Fixed100` and `SeptaFluted`, do not follow this scheme. Both are `Fixed: true`, LLS-capable, and draw from system tubing; `SeptaFluted` is the piercing tip used with septa-capped tube racks.

Items

Each of these tip classes has a matching `BC*` tip box; see Tip Boxes for the box names and the box-to-tip pairing.

All 24 disposable classes are unfixed, non-piercing, and do not draw from system tubing. `Fixed100` and `SeptaFluted` are the two fixed Span-8 tips: both draw from system tubing and are LLS-capable, and `SeptaFluted` is the only piercing tip in the set.

Capacity is the maximum liquid plus *trailing* air gap the tip will hold, and excludes the leading air gap.

Class	Capacity (µL)	Filtered	LLS	Tip height (cm)	Pods
T1025F	1025.0	Yes	No	10.9855	any
T1025F_LLS	1025.0	Yes	Yes	10.9855	any
T1025F_WB	1025.0	Yes	No	10.9855	any
T1070	1070.0	No	No	10.9855	any
T1070_LLS	1070.0	No	Yes	10.9855	any
T1070_WB	1070.0	No	No	10.9855	any
T190F	190.0	Yes	No	5.1435	any
T190F_LLS	190.0	Yes	Yes	5.1435	any
T190F_WB	190.0	Yes	No	5.1435	any
T230	230.0	No	No	5.1435	any
T230_LLS	230.0	No	Yes	5.1435	any
T230_WB	230.0	No	No	5.1435	any
T40F	40.0	Yes	No	3.8252	any
T40F_LLS	40.0	Yes	Yes	3.8252	any
T50F	50.0	Yes	No	5.1359	any
T50F_LLS	50.0	Yes	Yes	5.1359	any
T80	80.0	No	No	3.8252	any
T80_LLS	80.0	No	Yes	3.8252	any
T90	90.0	No	No	5.1359	any
T90_LLS	90.0	No	Yes	5.1359	any
T25F_384	25.0	Yes	No	3.3147	Multichannel
T30_384	30.0	No	No	3.3147	Multichannel
T40F_384	40.0	Yes	No	4.3104	Multichannel
T50_384	50.0	No	No	4.3104	Multichannel
Fixed100	93.0 (see note)	No	Yes	10.185	Span-8
SeptaFluted	37.0 (see note)	No	Yes	11.938	Span-8

Note: Fixed tips (`Fixed100`, `SeptaFluted`) enable drawing liquid into the Span-8 pod's tubing. Depending on the installed tubing and syringe, the volume varies. The given volumes are for large volume tubing, assuming no liquid is drawn into the tubing. Refer to the Biomek i5 and Biomek i7 Instructions for Use for more information (B54473).

